Moon SI Analyzer

A full Documentation of Moon Analyzer v1.0.1

Islam Tarek Moawad Eid (With Claude)



Moon SI Analyzer

A full Documentation of Moon Analyzer v1.0.1

by

Islam Tarek Moawad Eid (With Claude)

Author: Islam Tarek Moawad Eid (IslamHUFE)
Faculty: Faculty of Engineering, Capital University





Summary

Moon SI Analyzer is an open-source desktop application for the analysis of high-speed interconnect channels from their measured or simulated S-parameters. It reads Touchstone files (`.s2p`, `.s4p` and files with any number of ports through an explicit port mapping) and turns them into the quantities a signal-integrity engineer needs to judge a channel: insertion and return loss, impedance, group delay, power gain and stability, noise behaviour, time-domain reflection and transmission, a statistical eye diagram with equalization, bathtub curves and BER contours, eye-mask compliance, crosstalk metrics, equivalent circuits, rational (vector-fitting) models with SPICE export, and TRL de-embedding.

The program is written in Python. The numerical layer (NumPy, SciPy, scikit-rf) is completely separated from the presentation layer (Tkinter and Matplotlib), so every algorithm can be executed and verified without a graphical interface. A random-pattern engine filters long data sequences with the measured transfer function in the frequency domain, accumulates the received waveform in a two-dimensional histogram and draws it like a persistence oscilloscope. Block-wise processing keeps the memory consumption constant while millions of symbols are observed.

This document is the official technical reference of the software. For each feature it states the purpose, derives or quotes the governing equations, lists the assumptions, and reproduces the exact Python code that implements it; all listings are extracted automatically from the source file, so they cannot deviate from the program. The reference covers the architecture (Chapter 2), the network-parameter foundations (Chapter 3), frequency-domain (Chapter 4) and time-domain analysis (Chapter 5), the eye-diagram engine (Chapter 6), statistical channel assessment (Chapter 7), modelling, calibration and conditioning (Chapter 8), the graphical interface (Chapter 9), installation and deployment (Chapter 10), the verification campaign (Chapter 11) and the limitations of the models (Chapter 12). Appendices list every top-level function and every user-interface control.

A suite of 34 automated checks compares the implementation against independent references such as scikit-rf, closed-form solutions and direct circuit solutions; all checks pass (Chapter 11). The verification campaign also exposed one real inaccuracy, a step discontinuity introduced when a measurement does not start at DC, which was corrected and is documented in that chapter.

Contents

Summary i

Nomenclature vii

1 Introduction 1
- 1.1 Motivation and Problem Statement . . . 1
- 1.2 Objectives . . . 1
- 1.3 Scope and Intended Users . . . 2
- 1.4 Feature Overview . . . 2
- 1.5 Design Principles . . . 2
- 1.6 Organization of this Document . . . 3
- 1.7 Conventions . . . 3

2 System Architecture and Software Design 5
- 2.1 Technology Stack . . . 5
- 2.2 Layered Architecture . . . 5
- 2.3 Structure of the Source File . . . 6
- 2.4 The Data Model . . . 7
- 2.5 Composition of the Application Class . . . 7
- 2.6 Life Cycle of an Analysis . . . 8
- 2.7 Concurrency Model . . . 10
- 2.8 Performance and Memory Design . . . 11
- 2.9 Robustness . . . 12
- 2.10 Persistent State . . . 12

3 Touchstone Files and Network-Parameter Foundations 14
- 3.1 Scattering Parameters . . . 14
- 3.2 Reading Touchstone Files . . . 14
- 3.3 Selecting the Channel from an N-Port File . . . 16
- 3.4 Reference Impedance and Impedance from Reflection . . . 16
- 3.5 Conversion to Other Network Representations . . . 17
- 3.6 Mixed-Mode S-Parameters . . . 18
- 3.7 Noise Parameters in the File . . . 19

4 Frequency-Domain Analysis 21
- 4.1 S-Parameter, Loss and Impedance Views . . . 21
- 4.2 RF Metrics . . . 22
- 4.3 Group Delay and Phase Linearity . . . 23
- 4.4 Terminations . . . 24
- 4.5 Power Gains . . . 24
- 4.6 Stability Factors . . . 25
- 4.7 Noise Parameters and Noise Figure . . . 27
- 4.8 The Smith Chart . . . 28
- 4.9 Stability, Gain and Noise Circles . . . 29
- 4.10 Impedance Matching with L-Sections . . . 30

5 Time-Domain Analysis 33
- 5.1 From Frequency Data to a Time Response . . . 33
  - 5.1.1 The low-pass transform . . . 33
  - 5.1.2 Grid construction . . . 33
  - 5.1.3 Interpolation and DC handling . . . 34

5.1.4 Windowing and edge-rate limitation . . . 34
5.2 Impedance Profile and Distance Axis . . . 35
5.3 Locating Discontinuities . . . 37
5.4 Extrapolation to DC and Resampling . . . 37
5.5 Time-Domain Gating . . . 38
5.6 Causality Check . . . 39

6 The Eye-Diagram Engine 40
6.1 Overview of the Algorithm . . . 40
6.2 Signal Types . . . 40
6.2.1 Pseudo-random binary sequences . . . 41
6.3 Filtering with the Measured Channel . . . 42
6.3.1 Interpolation of the channel and band limitation . . . 42
6.3.2 Finite transmitter edge rate . . . 43
6.4 Equalization . . . 43
6.4.1 Transmitter feed-forward equalizer (FFE) . . . 43
6.4.2 Receiver continuous-time linear equalizer (CTLE) . . . 43
6.4.3 Pulse response and cursors . . . 44
6.4.4 Decision-feedback equalizer (DFE) . . . 44
6.5 Terminated Channel . . . 44
6.6 Threshold Detection, Alignment and Jitter . . . 45
6.6.1 Decision thresholds from the data . . . 45
6.6.2 Zero-crossing detection . . . 45
6.6.3 Alignment of the eye . . . 46
6.6.4 Jitter metrics . . . 46
6.7 Folding and the Two Density Arrays . . . 46
6.7.1 Folding . . . 46
6.7.2 Sample histogram . . . 46
6.7.3 Display density with trace interpolation . . . 46
6.7.4 Constant memory . . . 47
6.8 Statistical Eye Height . . . 47
6.9 The Complete Engine . . . 47
6.10 Display of the Eye . . . 51

7 Statistical Assessment of Channel Quality 54
7.1 Bit-Error Probability from the Eye Density . . . 54
7.1.1 Model . . . 54
7.1.2 Horizontal bathtub . . . 54
7.1.3 Random jitter . . . 54
7.1.4 Vertical bathtub and BER contour . . . 55
7.1.5 Eye opening at a target BER . . . 55
7.2 Eye Masks . . . 56
7.3 Limit Lines . . . 57
7.4 Insertion-Loss Deviation . . . 58
7.5 COM-Like Budget . . . 58
7.6 Passivity Enforcement . . . 61
7.7 Crosstalk . . . 61

8 Modelling, Calibration and Conditioning 64
8.1 Per-Unit-Length Parameters of a Uniform Line . . . 64
8.2 Lumped π Equivalent Circuit . . . 65
8.3 Vector Fitting and SPICE Export . . . 67
8.4 TRL Calibration and De-embedding . . . 68
8.5 Cascading and De-embedding with Known Networks . . . 70
8.6 Conditioning Tools . . . 71

9 The Graphical User Interface 72
9.1 Window Layout . . . 72

9.2 Theming . . . 73
9.3 The Tab-Building Pattern . . . 73
9.4 Plots and Interaction . . . 74
9.4.1 Embedding Matplotlib . . . 74
9.4.2 Markers . . . 75
9.4.3 Summary text . . . 75
9.4.4 Key performance indicators . . . 76
9.5 File Menu, Recent Files and Port Mapping . . . 76
9.6 Sessions and Exports . . . 77
9.7 Batch Processing . . . 79
9.8 The Logo . . . 79
9.9 Entry Point . . . 81

10 Installation, Usage and Deployment 82
10.1 Requirements . . . 82
10.2 Installation . . . 82
10.3 Running the Program . . . 82
10.4 Using the Numerical Layer as a Library . . . 83
10.5 A Typical Session . . . 83
10.6 Typical Tasks . . . 83
10.7 Packaging for Distribution . . . 84
10.8 Files Created by the Program . . . 84
10.9 Troubleshooting . . . 84

11 Verification and Validation 86
11.1 Method . . . 86
11.2 Results . . . 86
11.3 Defects Found During Verification and Development . . . 88
11.4 Regression Procedure . . . 89
11.5 What Has Not Been Validated . . . 89

12 Assumptions, Limitations and Future Work 90
12.1 Global Assumptions . . . 90
12.2 Limitations by Analysis . . . 90
12.3 Interpreting Results Responsibly . . . 91
12.4 Future Work . . . 91

A API Reference 93
A.1 Classes . . . 95
A.2 Naming Conventions of the Interface Code . . . 96
A.3 Tab Registry . . . 96
A.4 Keys of the Eye Result . . . 97

B User-Interface Reference 98
B.1 Main Control Cards . . . 98
B.2 Advanced Tab Controls . . . 98
B.3 Quality & EQ and Tools Controls . . . 99
B.4 Keyboard and Mouse . . . 99

List of Figures

1.1 Main window of Moon SI Analyzer. 3

2.1 Two-layer architecture of Moon SI Analyzer. 6
2.2 Sequence of events when a Touchstone file is loaded. 10

3.1 Mixed-mode analysis of a four-port differential channel. 19

4.1 Smith chart with the S11 trace coloured by frequency. 29

5.1 Time-domain gating of a reflection response. 39

6.1 Processing chain of the eye engine for one block of symbols. 40
6.2 Eye diagram of an NRZ signal through a measured channel. 52
6.3 PAM4 eye with three sub-eyes and their thresholds. 53
6.4 Effect of the trace-interpolated display density on steep edges. 53

7.1 Pulse response, bathtub curves and BER contour. 55
7.2 Crosstalk analysis of a four-lane connector. 62

8.1 Per-unit-length parameters and π-model comparison. 67
8.2 Raw and TRL-corrected responses of a DUT. 70

9.1 Batch processing of several channels. 80

List of Tables

1.1 Capabilities of Moon SI Analyzer and where they are documented. . . . 2
1.2 Sections of the source file `si_analyzer.py`. . . . 4

2.1 Third-party components and their roles. . . . 5
2.2 Responsibilities of the application class and its mixins. . . . 8
2.3 Asymptotic cost of the main algorithms (N_f frequency points, N_b symbols per block, O samples per UI, N_{blk} blocks, B voltage bins). . . . 12

6.1 Signal types supported by the eye engine. . . . 41

9.1 Tabs of the main window and the chapters that describe them. . . . 72

10.1 Software versions used for the verification run. . . . 82

11.1 Results of the automated verification suite (`tools/validate.py`). . . . 86

A.1 Top-level functions of `si_analyzer.py` (parameters shown without annotations and defaults). . . . 93
A.2 Classes of the program. . . . 96
A.3 Naming conventions in the mixins. . . . 96
A.4 Analysis tabs: figure name, draw method and group. . . . 96
A.5 Keys of the eye-engine result. . . . 97

B.1 Controls of the three cards above the notebook. . . . 98
B.2 Controls inside the Advanced tab. . . . 99
B.3 Controls of the Quality & EQ and Tools tabs. . . . 99

Nomenclature

Abbreviations

ABCD	Chain (transmission) matrix of a two-port
ACR	Attenuation-to-crosstalk ratio
BER	Bit-error ratio
CSV	Comma-separated values
CTLE	Continuous-time linear equalizer
DC	Direct current (zero frequency)
DFE	Decision-feedback equalizer
DUT	Device under test
FEXT	Far-end crosstalk
FFE	Feed-forward equalizer
FFT / IFFT	(Inverse) fast Fourier transform
GUI	Graphical user interface
ICN	Integrated crosstalk noise
IL	Insertion loss
ILD	Insertion-loss deviation
ISI	Inter-symbol interference
JSON	JavaScript object notation
KPI	Key performance indicator
LFSR	Linear-feedback shift register
MAG	Maximum available gain
MSG	Maximum stable gain
NEXT	Near-end crosstalk
NF	Noise figure
NRZ	Non-return-to-zero (two-level signalling, PAM2)
PAM-L	Pulse-amplitude modulation with L levels
PRBS	Pseudo-random binary sequence
PSACR / PSNEXT / PSFEXT	Power-sum ACR / NEXT / FEXT
RJ	Random jitter
RL	Return loss
RLGC	Resistance, inductance, conductance, capacitance per unit length
RX / TX	Receiver / transmitter
SI	Signal integrity
SPICE	Simulation program with integrated circuit emphasis
SVD	Singular value decomposition
TDR / TDT	Time-domain reflectometry / transmissometry
TRL	Thru-reflect-line calibration
UI	Unit interval (symbol period)
VF	Vector fitting
VSWR	Voltage standing-wave ratio

Symbols

$f,\ \omega$	Frequency and angular frequency, $\omega = 2\pi f$
Z_0	Real reference impedance of a port (normally $50\,\Omega$)
$\mathbf{S}$, S_{ij}	Scattering matrix and its elements (response at port i to excitation at port j)
$\mathbf{Z},\ \mathbf{Y}$	Impedance and admittance matrices
$\mathbf{T}$	Scattering-transfer matrix
Γ	Reflection coefficient
$\Gamma_s,\ \Gamma_L$	Source and load reflection coefficients
$\Gamma_{in},\ \Gamma_{out}$	Input and output reflection coefficients with terminations
τ_g	Group delay
$\gamma = \alpha + j\beta$	Complex propagation constant
Z_c	Characteristic impedance of a line
$\varepsilon_{r,\mathrm{eff}}$	Effective relative permittivity
$\sigma_{\max}(\mathbf{S})$	Largest singular value of $\mathbf{S}$
$K,\ \Delta,\ \mu,\ \mu'$	Stability factors (Rollett, determinant, Edwards–Sinsky)
$G_T,\ G_A,\ G_P$	Transducer, available and operating power gain
$F,\ F_{\min},\ R_n,\ \Gamma_{opt}$	Noise factor, minimum noise factor, equivalent noise resistance, optimum source reflection
$\rho(t)$	Reflection step response in a TDR measurement
t_r	10–90 % rise time
T	Unit interval (symbol period)
R_b	Data (bit) rate
β_s	Number of information bits per symbol
O	Oversampling factor (samples per UI)
N_b	Number of symbols per processing block
A	Peak signal amplitude, $A = V_{pp}/2$
h_k	Pulse-response cursor at kT from the main cursor
$Q(x)$	Gaussian tail probability, $Q(x) = \frac{1}{2}\,\mathrm{erfc}(x/\sqrt{2})$
$\sigma_n,\ \sigma_j$	Voltage-noise and random-jitter standard deviations

1

Introduction

1.1. Motivation and Problem Statement

The data rates of serial links have grown by orders of magnitude, while the printed-circuit boards, connectors, cables and packages that carry them have changed far more slowly. At multi-gigabit rates a channel stops behaving like a set of wires and starts behaving like a distributed, frequency-dependent, lossy transmission-line network. Whether a link works depends on the interaction of attenuation, dispersion, reflections, crosstalk and the equalization built into the transmitter and receiver [**hall2009**, **bogatin2018**].

Engineers describe such a channel by its scattering parameters (S-parameters), measured with a vector network analyzer or computed by an electromagnetic solver and stored in Touchstone files [**touchstone2**]. A Touchstone file is only data. Turning it into a decision ("is the insertion loss acceptable at the Nyquist frequency?", "where is the impedance discontinuity?", "does the eye pass the mask with this equalizer?") requires a chain of transformations: conversion between network representations, transformation to the time domain, synthesis of a data pattern, filtering, statistics and visualization. Commercial tools provide this chain but are closed, expensive and difficult to inspect; ad-hoc scripts are transparent but rarely complete, tested or documented.

Moon SI Analyzer addresses this gap. It is an open-source application that provides the complete chain in one coherent tool, in which every assumption is stated and every algorithm is available as readable Python code.

1.2. Objectives

The objectives of the software, and therefore of the design decisions documented in this reference, are:

1. **Completeness.** Offer, for a Touchstone channel, the analyses that signal-integrity practice relies on, from basic S-parameter plots to equalized statistical eye diagrams and de-embedding.
2. **Physical faithfulness.** Prefer models that reproduce reality (finite transmitter edge rate, terminations, inter-symbol interference, noise and jitter) over idealized ones, and make every simplification explicit in the interface and in this document.
3. **Numerical honesty.** Never silently produce a number that its model cannot support: results are labelled with their assumptions (for example, the "COM-like" budget is explicitly not a standards-compliant COM), singular points yield `NaN` instead of exceptions, and the verification suite (Chapter 11) checks the implementation against independent references.
4. **Transparency.** Keep the numerical layer in plain, documented functions that can be read, tested and reused without the graphical interface.
5. **Scalability.** Process millions of symbols and thousands of frequency points with constant memory and acceptable run time on an ordinary computer.

6. **Usability.** Present the results in a consistent dark-themed interface with live parameter changes, markers, undo, session files, batch processing and export.

1.3. Scope and Intended Users

The software is intended for signal-integrity and PCB engineers, for researchers who compare channel models with measurements, and for students who want to see how a Touchstone file turns into an eye diagram. It analyses *linear, time-invariant* channels. Non-linear transmitter or receiver circuits, clock-and-data recovery, and full link-level BER simulation with adaptive equalizers are outside its scope (Chapter 12 lists all limitations).

1.4. Feature Overview

Table 1.1 lists the capabilities of the program and the chapter that documents each of them.

Table 1.1: Capabilities of Moon SI Analyzer and where they are documented.

Capability	Interface location	Chapter
Touchstone reading, N-port mapping, noise block	File menu	3
S-parameter, loss, impedance and Smith-chart views	Main tabs	4
Group delay, gains, stability factors, RF metrics	Advanced	4
Parameter conversions (Z, Y, ABCD, T, H, G)	Advanced	3
Mixed-mode S-parameters	Advanced	3
TDR/TDT, impedance profile, discontinuity finder	Advanced	5
Time-domain gating, DC extrapolation, resampling	Tools	5
Statistical eye with 12 signal types	Eye & jitter	6
TX FFE, RX CTLE, DFE, terminations, TX rise time	Eye cards	6
Pulse response, bathtub, BER contour, eye masks	Quality & EQ	7
Limit lines, ILD, COM-like budget, causality check	Quality & EQ	7
Crosstalk (NEXT, FEXT, ICN) from N-port files	Quality & EQ	7
RLGC and π-model extraction	Advanced	8
Vector fitting and SPICE export	Tools	8
TRL de-embedding, cascade, renormalization	Advanced, Tools	8
Batch processing, CSV/HTML/PNG export, sessions	Tools, File	9

1.5. Design Principles

Four principles shaped the implementation and recur throughout this document.

Layered separation. Pure numerical functions never import the graphical toolkit; user-interface code only calls them and draws the results. This is what makes the automated verification of Chapter 11 possible.

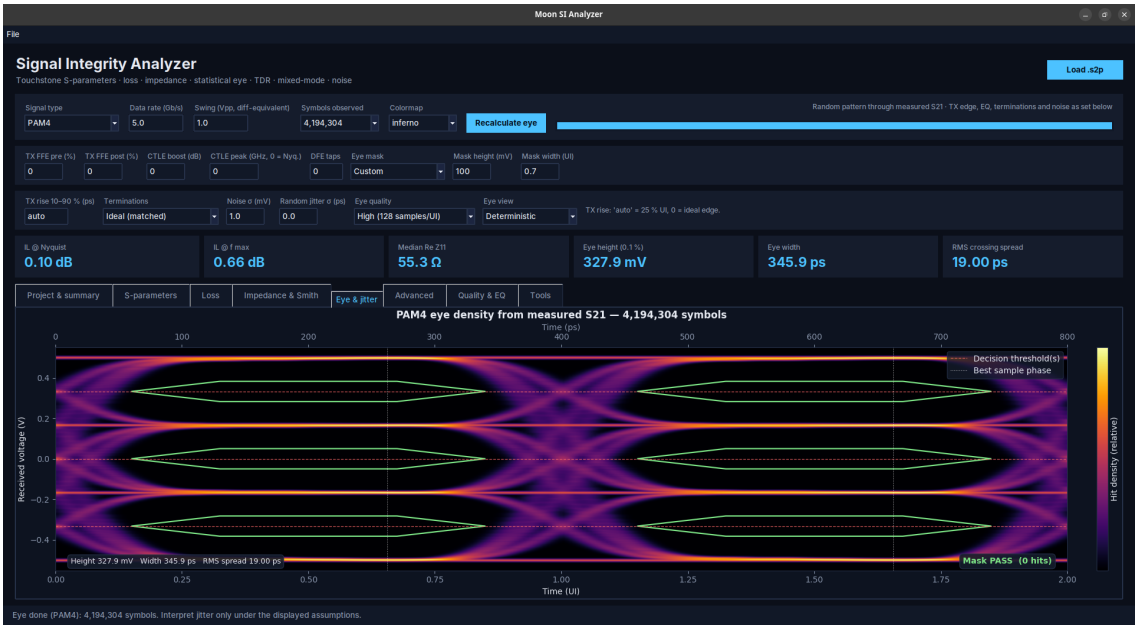


Figure 1.1: Main window of Moon SI Analyzer.

Arrays instead of loops. Frequency sweeps are handled as NumPy arrays of shape $(N_f, 2, 2)$, and linear algebra is batched across frequency. Python-level loops appear only over a handful of items (ports, thresholds, taps).

Constant memory. The eye engine processes independent blocks and accumulates only a histogram of fixed size; the number of observed symbols can therefore grow into the millions without growing memory (Section 6.7).

Explicit assumptions. Every analysis states what it does not model, in the interface, in the source docstrings and in Chapter 12.

1.6. Organization of this Document

Chapter 2 presents the architecture and the data model. Chapter 3 defines the network-parameter foundations on which everything else rests. Chapters 4 and 5 cover frequency- and time-domain analysis, Chapter 6 the eye-diagram engine, and Chapter 7 the statistical assessment built on top of it. Chapter 8 explains modelling, calibration and conditioning tools. Chapter 9 documents the graphical implementation, Chapter 10 installation and use, Chapter 11 the verification, and Chapter 12 the limitations and future work. Appendix A is a function reference, and Appendix B lists every control of the interface.

1.7. Conventions

- Matrices are bold ($\mathbf{S}$), scalars italic, and $j = \sqrt{-1}$. Logarithms in decibel formulas are base 10.
- Unless stated otherwise the reference impedance Z_0 is real and equal on all ports.
- Source-code listings are extracted verbatim from `si_analyzer.py` by an automated generator. Methods are shown without their enclosing class indentation. The caption above each listing names the extracted symbol.
- Figures marked *PLACEHOLDER* are to be replaced with screenshots or diagrams; the placeholder text states exactly what to capture. A checklist of all placeholders is provided in the file `FIGURES_TODO.md`.
- Table 1.2 gives the structure of the source file that the listings refer to.

Table 1.2: Sections of the source file `si_analyzer.py`.

Lines	Count	Section
40–166	127	Dark palette
167–237	71	Signal types for the eye diagram
238–595	358	Eye / jitter engine (block-wise, constant memory, any number of symbols)
596–898	303	Advanced analysis helpers (pure numpy, no GUI)
899–1538	640	Advanced tabs (mixin for the main window)
1539–1672	134	Extra helpers: causality, gating, passivity, crosstalk weighting, circles, matching
1673–2548	876	Extra tabs (mixin): ’Quality & EQ’ and ’Tools’
2549–2937	389	Ultimate additions: terminations, discontinuities, conditioning, batch, preferences
2938–3640	703	GUI
Total: 3640 lines		

2

System Architecture and Software Design

This chapter describes how the program is organized: the technology stack, the layering that separates mathematics from presentation, the central data model, the composition of the application class from mixins, the life cycle of an analysis, the threading model and the strategies used for memory, performance and robustness.

2.1. Technology Stack

Moon SI Analyzer is a single-file Python program. Table 2.1 lists the libraries it relies on and the role of each.

Table 2.1: Third-party components and their roles.

Component	Role
Python ≥ 3.10	Language and standard library (`tkinter`, `threading`, `dataclasses`, `json`, `csv`, `functools`)
NumPy [**harris2020**]	Array arithmetic, FFT, batched linear algebra, SVD
SciPy [**virtanen2020**]	Gaussian filtering, special functions (`erfc`), peak finding, cubic interpolation
scikit-rf [**arsenovic2022**]	Touchstone parsing, network cascading, renormalization, TRL calibration, vector fitting
Matplotlib [**hunter2007**]	All plots, embedded in Tk through the `TkAgg` backend
Tkinter / ttk	Windows, widgets, dialogs, menus, themed styling
Pillow	Loading and scaling of `logo.png` (installed together with Matplotlib)

2.2. Layered Architecture

The program has two layers (Figure 2.1).

Numerical layer. Pure functions and constants that take NumPy arrays and return NumPy arrays or dictionaries: Touchstone reading, parameter conversions, mixed-mode transformation, group delay, stability, gains, time-domain transforms, the eye engine, BER statistics, masks, circuit extraction, conditioning and crosstalk weighting. They import neither `tkinter` nor `matplotlib.pyplot`.

Presentation layer. One application class assembled from three mixins. It owns the widgets, reads their values, calls the numerical layer and draws the results on Matplotlib figures.

The separation has practical consequences. The whole numerical layer can be imported and exercised in an environment without a display (this is how the verification suite of Chapter 11 works), it can be reused as a library (Section 10.4), and a defect in a plot cannot corrupt a number.

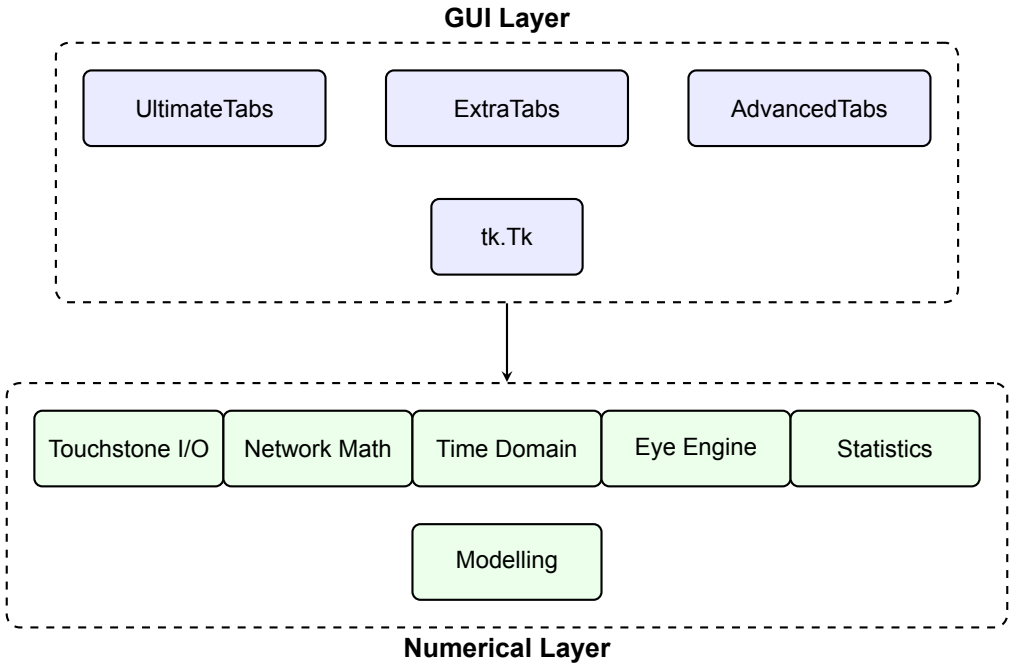


Figure 2.1: Two-layer architecture of Moon SI Analyzer.

2.3. Structure of the Source File

The source file is organized top-down in the order shown in Table 1.2 (Chapter 1): visual constants, the data model and reader, the signal definitions, the eye engine, the analysis helpers, the three mixins, and finally the application class and the entry point. Dependencies only point upwards, so reading the file from the top never requires a forward reference to a later definition (except for calls resolved at run time).

The visual constants define one dark palette used by every widget and plot, and configure Matplotlib globally so that all figures share the same look:

Source: `si_analyzer.py` — `si_analyzer.py`, lines 42–78

```
BG = "#0b0f19"
PANEL = "#111827"
CARD = "#151c2c"
CARD_HI = "#1c2540"
BORDER = "#263149"
FG = "#e6eaf2"
MUTED = "#8b97ad"
ACCENT = "#4cc2ff"
ACCENT2 = "#ff9f5a"
GREEN = "#7ee787"
PURPLE = "#d2a8ff"
RED = "#ff6b6b"
PLOT_BG = "#0d1320"
AX_BG = "#0a0e17"
GRID = "#243049"

# Smith-chart trace: blue at the lowest frequency -> red at the highest.
FREQ_CMAP = LinearSegmentedColormap.from_list("freq_blue_red", ["#2f7bff", "#ff3b3b"])

mpl.rcParams.update({
    "figure.facecolor": PLOT_BG,
```

```
    "axes.facecolor": AX_BG,
    "axes.edgecolor": BORDER,
    "axes.labelcolor": FG,
    "axes.titlecolor": FG,
    "axes.titleweight": "bold",
    "axes.titlesize": 12,
    "xtick.color": MUTED,
    "ytick.color": MUTED,
    "text.color": FG,
    "grid.color": GRID,
    "grid.alpha": 0.6,
    "legend.facecolor": CARD,
    "legend.edgecolor": BORDER,
    "legend.labelcolor": FG,
    "savefig.facecolor": PLOT_BG,
    "lines.linewidth": 1.6,
```

2.4. The Data Model

All analyses start from one object, `ChannelData`. Its fields `s` and `z0` always describe the *active two-port*, the network that the main tabs analyse. For a two-port file this is simply the file content. For files with more ports, the active two-port is derived according to a port mapping (Section 3.3): either two single-ended ports, or the differential block $\mathbf{S}_{dd}$ of two port pairs. In the second case the underlying 4×4 matrix is kept in `s_full` so that the mixed-mode tab can show all four mode blocks.

Source: `si_analyzer.py` — `ChannelData`

```
@dataclass
class ChannelData:
    """`s` / `z0` always hold the active 2-port (for a 4-port file: the differential Sdd
        block)."""
    path: Path
    frequency_hz: np.ndarray
    s: np.ndarray
    z0: np.ndarray
    ports: int = 2
    s_full: np.ndarray | None = None
    z0_full: np.ndarray | None = None
    noise: dict[str, Any] | None = None
    pairing: str = "1-2 / 3-4"
    note: str = ""
```

The frequency axis is stored in hertz, the S-parameters as an array of shape $(N_f, 2, 2)$ and the reference impedances as $(N_f, 2)$. The optional `noise` dictionary holds the noise parameters of a two-port file (Section 4.7); `note` is a human-readable description of the mapping used, which appears in the project summary.

2.5. Composition of the Application Class

The application window is a class assembled from three mixins:

$$\texttt{SIAnalyzerApp} = \texttt{UltimateTabs} + \texttt{ExtraTabs} + \texttt{AdvancedTabs} + \texttt{tk.Tk}.$$

Each mixin groups one family of features and contributes methods only; it keeps no constructor and relies on attributes that the final class provides. This keeps the individual files regions small and lets features be added without growing a single monolithic class. Table 2.2 summarizes the responsibilities.

The constructor of the final class shows the order of initialization: window geometry, state variables, logo, then the construction of the interface:

Source: `si_analyzer.py` — `SIAnalyzerApp.__init__`

```
def __init__(self) -> None:
    super().__init__()
```

Table 2.2: Responsibilities of the application class and its mixins.

Class	Responsibility
`AdvancedTabs`	Group delay, gains and stability, parameter conversions, RF metrics, mixed-mode, equivalent circuit, noise, TDR/Z-profile and TRL de-embedding (the “Advanced” tab); shared helpers such as field builders and placeholders
`ExtraTabs`	Equalization settings, masks, bathtub and BER contour, limit lines, ILD/COM/causality, crosstalk, file comparison, cascade/de-embedding, vector fitting, gating, Smith extras and exports (the “Quality & EQ” and “Tools” tabs)
`UltimateTabs`	File menu, preferences, port-mapping dialog, conditioning tools and batch processing
`SIAnalyzerApp`	Main window, styling, the control cards, the main S-parameter, loss, impedance and eye tabs, loading, KPIs, summary text

```
    self.title(APP_TITLE)
    self.geometry("1320x880")
    self.minsize(1000, 700)
    self.configure(bg=BG)
    self.channel: ChannelData | None = None
    self.eye_result: dict[str, Any] | None = None
    self._eye_state: dict[str, Any] | None = None
    self._smith_cbar: Any = None
    self._load_logo()
    self._build_ui()
    self._set_status("File␣ ␣Open␣(Ctrl+O)␣to␣load␣a␣Touchstone␣file.␣Double-click␣a␣plot␣to␣
        drop␣a␣marker;␣right␣double-click␣clears.")
```

2.6. Life Cycle of an Analysis

When a file is opened, the following sequence runs (Figure 2.2):

1. `load_s2p` calls `read_touchstone` (Chapter 3); if the file has a port count that needs a mapping, a dialog is shown first.
2. The history of undo states is cleared, the file is added to the recent-files list and the eye result is discarded.
3. `_refresh_analysis` redraws the main plots and calls `_refresh_advanced` and `_refresh_extra`, which redraw every analysis tab. Each tab is drawn inside its own `try`/`except` block, so that a failure in one analysis shows an error message in its tab and leaves the others intact.
4. The statistical eye is started in a worker thread (Section 2.7); when it finishes, `_after_eye` updates the tabs that depend on it (bathtub and quality).

Source: `si_analyzer.py` — `SIAnalyzerApp.load_s2p`

```
def load_s2p(self, path: str | Path, port_map: tuple | None = None, force_map: bool = False)
    -> None:
    try:
        if force_map and port_map is None:
            m = re.search(r"\.s(\d+)p$", str(path).lower())
            port_map = self._ask_port_map(int(m.group(1)) if m else 4, path)
            if port_map is None:
                return self._set_status("Load␣cancelled.")
        try:
            channel = read_touchstone(path, self.pair_var.get(), port_map)
        except NeedPortMap as need:
            port_map = self._ask_port_map(need.nports, path)
            if port_map is None:
                return self._set_status("Load␣cancelled.")
            channel = read_touchstone(path, self.pair_var.get(), port_map)
```

```
        self.channel = channel
        self._history.clear()
        self._remember(path)
        self.eye_result = None
        self._refresh_analysis()
        self._set_status(f"Loaded {self.channel.path.name}: {len(self.channel.frequency_hz)} points, "
                         f"{self.channel.frequency_hz[0]/1e9:.4g}–{self.channel.frequency_hz[-1]/1e9:.4g} GHz.")
    except Exception as exc:
        messagebox.showerror("Could not load Touchstone file", str(exc))
        self._set_status("Load failed.")
```

Source: `si_analyzer.py` — `SIAnalyzerApp._refresh_analysis`

```
def _refresh_analysis(self) -> None:
    if self.channel is None:
        return
    d = self.channel
    f_ghz = d.frequency_hz / 1e9
    colors = {"S11": ACCENT, "S21": ACCENT2, "S12": GREEN, "S22": PURPLE}
    self.ax_s_mag.clear()
    self.ax_s_phase.clear()
    for row, col, name in ((0, 0, "S11"), (1, 0, "S21"), (0, 1, "S12"), (1, 1, "S22")):
        values = d.s[:, row, col]
        ls = "--" if name in ("S12", "S22") else "-"
        self.ax_s_mag.plot(f_ghz, 20 * np.log10(np.maximum(np.abs(values), 1e-15)), ls, color=colors[name], label=name)
        self.ax_s_phase.plot(f_ghz, np.unwrap(np.angle(values)) * 180 / np.pi, ls, color=colors[name], label=name)
    self.ax_s_mag.set_ylabel("Magnitude (dB)")
    self.ax_s_phase.set_ylabel("Unwrapped phase (°)")
    self.ax_s_phase.set_xlabel("Frequency (GHz)")
    self.ax_s_mag.set_title("Two-port S-parameters")
    for ax in (self.ax_s_mag, self.ax_s_phase):
        ax.grid(True)
        ax.legend(ncol=4, loc="best")
    self.canvas_s.draw_idle()

    self._draw_loss()

    self.ax_z.clear()
    z11 = impedance_from_reflection(d.s[:, 0, 0], np.real(d.z0[:, 0]))
    z22 = impedance_from_reflection(d.s[:, 1, 1], np.real(d.z0[:, 1]))
    self.ax_z.plot(f_ghz, np.real(z11), color=ACCENT, label="Re Z11")
    self.ax_z.plot(f_ghz, np.real(z22), color=PURPLE, label="Re Z22")
    self.ax_z.plot(f_ghz, np.abs(z11), "--", color=ACCENT2, alpha=0.85, label="|Z11|")
    self.ax_z.set_title("Input impedance from S11 / S22")
    self.ax_z.set_xlabel("Frequency (GHz)")
    self.ax_z.set_ylabel("Impedance Ω()")
    self.ax_z.grid(True)
    self.ax_z.legend()
    self._draw_smith(self.ax_smith, d.s[:, 0, 0], f_ghz)
    self.canvas_z.draw_idle()

    self._refresh_advanced()
    self._refresh_extra()
    self._update_kpis()
    self._update_summary()
    self._recalculate_eye(silent=True)
```

Source: `si_analyzer.py` — `AdvancedTabs._refresh_advanced`

```
def _refresh_advanced(self) -> None:
    self.adv_summary = {}
    for name in ("tdr", "gd", "gains", "params", "rf", "mm", "eq", "noise"):
        try:
            getattr(self, f"_draw_{name}")()
        except Exception as exc:  # keep the other tabs alive if one analysis fails
            self._placeholder(name, f"{name} analysis failed:\n{exc}")
```

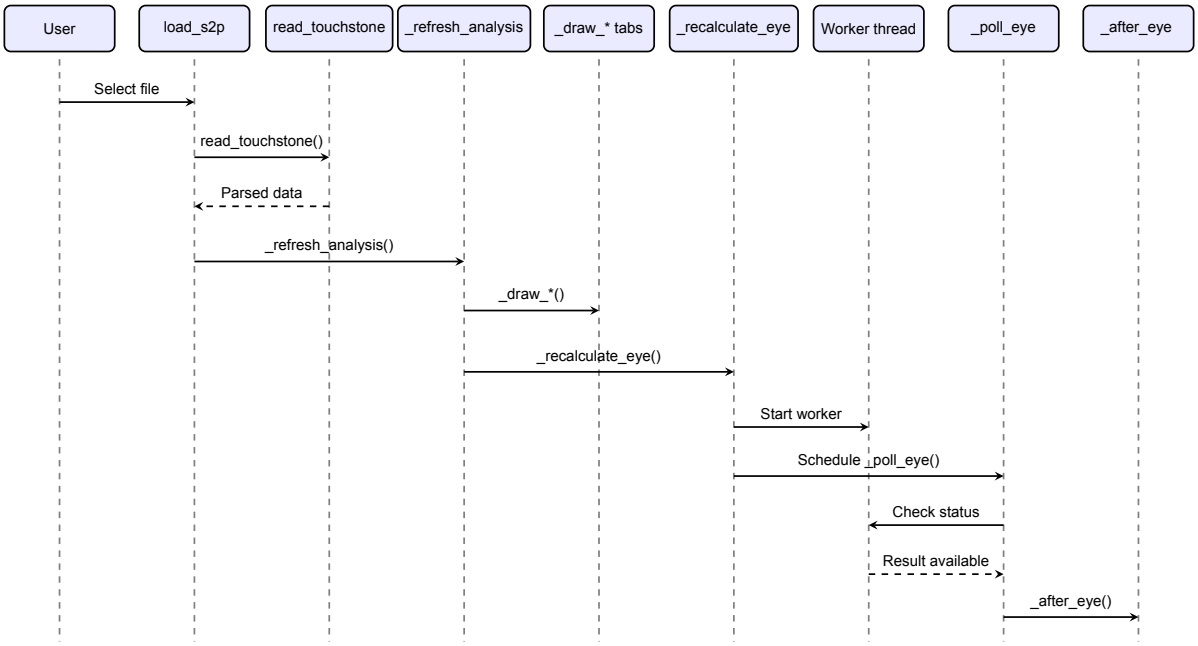


Figure 2.2: Sequence of events when a Touchstone file is loaded.

2.7. Concurrency Model

Generating an eye over hundreds of thousands of symbols takes seconds, which would freeze a single-threaded interface. The eye computation therefore runs in a daemon worker thread, under three rules:

- The worker never touches a widget. It only writes into a plain dictionary (`progress`, `result`, `error`, `done`).
- The Tk main loop polls that dictionary every $80\,\mathrm{ms}$ through `after`, updates the progress bar and, when the worker is done, draws the result on the main thread.
- A second request is ignored while a run is in progress, which prevents overlapping computations.

All eye settings are read on the main thread *before* the worker starts, so the worker operates on a consistent snapshot even if the user changes a field meanwhile.

Source: `si_analyzer.py` — `SIAnalyzerApp._recalculate_eye`

```
def _recalculate_eye(self, silent: bool = False) -> None:
    if self.channel is None:
        if not silent:
            messagebox.showinfo("Load␣channel␣first", "Load␣a␣.s2p␣file␣before␣calculating␣an
                ␣eye␣diagram.")
        return
    if self._eye_state is not None and not self._eye_state["done"]:
        return  # a run is already in progress
    try:
        rate = float(self.rate_var.get())
        swing = float(self.swing_var.get())
        bits = int(self.bits_var.get().replace(",", "").replace("_", "").strip())
    except ValueError:
        if not silent:
            messagebox.showerror("Invalid␣input", "Data␣rate,␣swing,␣and␣symbol␣count␣must␣be
                ␣numbers.")
        return
    signal = self.signal_var.get()
    eq = self._eq_settings()

    freq = self.channel.frequency_hz
    if self.term_var.get().startswith("Zs/ZL"):
        gs, gl = self._terminations()
```

```
        s21 = terminated_response(self.channel.s, gs, gl)
    else:
        s21 = self.channel.s[:, 1, 0]
    state: dict[str, Any] = {"done": False, "progress": 0.0, "result": None, "error": None, "
        silent": silent}
    self._eye_state = state

    def work() -> None:
        try:
            state["result"] = make_eye_and_jitter(
                freq, s21, rate, swing, total_bits=bits, signal=signal,
                progress=lambda p: state.__setitem__("progress", p), **eq,
            )
        except Exception as exc:  # reported on the Tk thread
            state["error"] = exc
        state["done"] = True

    self.eye_btn.configure(state="disabled")
    self.progress.configure(value=0.0)
    self._set_status(f"Calculating␣{signal}␣eye␣over␣{bits:,}␣…symbols")
    threading.Thread(target=work, daemon=True).start()
    self.after(80, self._poll_eye)
```

Source: `si_analyzer.py` — `SIAnalyzerApp._poll_eye`

```
def _poll_eye(self) -> None:
    state = self._eye_state
    if state is None:
        return
    self.progress.configure(value=state["progress"])
    if not state["done"]:
        self.after(80, self._poll_eye)
        return
    self.eye_btn.configure(state="normal")
    self.progress.configure(value=1.0 if state["error"] is None else 0.0)
    if state["error"] is not None:
        if not state["silent"]:
            messagebox.showerror("Eye␣analysis␣failed", str(state["error"]))
        self._set_status(f"Eye␣analysis␣failed:␣{state['error']}")
        return
    self.eye_result = state["result"]
    self._redraw_eye()
    self._draw_loss()
    self._update_kpis()
    self._update_summary()
    self._after_eye()
    self._set_status(f"Eye␣done␣({self.eye_result['signal']}):␣{self.eye_result['bits']:,}␣
        symbols.␣"
                     "Interpret␣jitter␣only␣under␣the␣displayed␣assumptions.")
```

2.8. Performance and Memory Design

Three techniques keep the program responsive:

Batched linear algebra. Conversions such as $\mathbf{S} \to \mathbf{Z}$ solve N_f small systems in a single call to `numpy.linalg.solve`, with a per-point fallback only when a singular matrix occurs (function `_solve`).

Block-wise eye accumulation. Memory is proportional to the block length and the histogram size, not to the number of symbols (Section 6.7).

Caching. The period of each maximal-length PRBS sequence is computed once (`functools.lru_cache`).

Table 2.3 gives the asymptotic cost of the main operations.

The FFT is the Cooley–Tukey algorithm [**cooley1965**] as implemented in NumPy.

Table 2.3: Asymptotic cost of the main algorithms (N_f frequency points, N_b symbols per block, O samples per UI, N_{blk} blocks, B voltage bins).

Operation	Time	Memory
Parameter conversion	$O(N_f)$	$O(N_f)$
Time response (TDR/TDT)	$O(M \log M)$, $M = 8192$	$O(M)$
Eye engine, one block	$O(N_b O \log(N_b O))$	$O(N_b O)$
Eye engine, total	$N_{blk}\times$ one block	$O(O\,B)$ histogram
BER contour	$O(O\,B\,n_y)$	$O(O\,n_y)$

2.9. Robustness

Numerical singularities are an everyday fact in network analysis: a lossless short makes $\mathbf{I} - \mathbf{S}$ singular, a perfectly matched port makes a ratio of zero by zero. The program treats them as data, not as errors. The solver wrapper returns `NaN` for singular frequency points and the plotting code skips non-finite values:

Source: `si_analyzer.py` — `_solve`

```
def _solve(a: np.ndarray, b: np.ndarray) -> np.ndarray:
    """Batched␣linear␣solve␣that␣returns␣NaN␣for␣singular␣frequency␣points␣instead␣of␣raising
        ."""
    try:
        return np.linalg.solve(a, b)
    except np.linalg.LinAlgError:
        out = np.full(np.broadcast(a, b).shape, np.nan, dtype=complex)
        for i in range(a.shape[0]):
            try:
                out[i] = np.linalg.solve(a[i], b[i])
            except np.linalg.LinAlgError:
                pass
        return out
```

In addition, input validation rejects files with fewer than three frequency points, non-increasing frequencies and non-finite parameters (Section 3.2), and each analysis tab reports its own failure message rather than aborting the interface.

2.10. Persistent State

Three mechanisms keep state between actions or sessions.

Preferences. The most recently used folder and the list of the last eight files are saved to `~/.si_analyzer.json`. A failure to read or write this file is silently ignored; the program never depends on it.

Sessions. A session file (JSON) stores all control values and the path of the channel file (Section 9.6).

Undo history. Every operation that modifies the active channel (cascade, de-embedding, conditioning, gating, passivity enforcement, TRL) first stores a deep copy of the current channel; up to ten states are kept.

Source: `si_analyzer.py` — `load_prefs`

```
def load_prefs() -> dict[str, Any]:
    try:
        import json
        return json.loads(PREFS_PATH.read_text(encoding="utf-8"))
    except Exception:
        return {}
```

Source: `si_analyzer.py` — `save_prefs`

```
def save_prefs(prefs: dict[str, Any]) -> None:
    try:
        import json
```

```
        PREFS_PATH.write_text(json.dumps(prefs, indent=2), encoding="utf-8")
    except Exception:
        pass
```

Source: `si_analyzer.py` — `ExtraTabs._push_history`

```
def _push_history(self) -> None:
    import copy
    if self.channel is not None:
        self._history.append(copy.deepcopy(self.channel))
        del self._history[:-10]
```

Source: `si_analyzer.py` — `ExtraTabs._undo`

```
def _undo(self) -> None:
    if not self._history:
        messagebox.showinfo("Nothing to undo", "No earlier channel is stored.")
        return
    self.channel = self._history.pop()
    self.eye_result = None
    self._refresh_analysis()
    self._set_status("Restored the previous channel.")
```

3

Touchstone Files and Network-Parameter Foundations

Everything the program computes starts from a set of S-parameters. This chapter defines the quantities, explains how files are read and validated, how a channel is selected from a multi-port file, and how the same network is expressed in the other common representations (impedance, admittance, chain, transfer, hybrid) and in mixed-mode form.

3.1. Scattering Parameters

For an n-port network with real reference impedances Z_{0i}, the incident and reflected power waves at port i are defined from the port voltage V_i and current I_i as [**kurokawa1965**, **pozar2012**]

$$a_i = \frac{V_i + Z_{0i} I_i}{2\sqrt{Z_{0i}}}, \qquad b_i = \frac{V_i - Z_{0i} I_i}{2\sqrt{Z_{0i}}}, \tag{3.1}$$

and the scattering matrix relates them linearly:

$$\mathbf{b} = \mathbf{S}\,\mathbf{a}. \tag{3.2}$$

S_{ij} is therefore the wave leaving port i when port j is driven and all other ports are terminated in their reference impedance. For a two-port, S_{11} and S_{22} are reflection coefficients and S_{21}, S_{12} are transmission coefficients.

3.2. Reading Touchstone Files

A Touchstone file consists of an option line (frequency unit, parameter type, number format, reference resistance), comment lines starting with `!`, and data lines [**touchstone2**]. The number format can be magnitude/angle (MA), dB/angle (DB) or real/imaginary (RI). The program delegates parsing to scikit-rf [**arsenovic2022**], which returns the frequency axis in hertz, the S-matrices and the per-port reference impedances; the program then adds its own validation:

- at least three frequency points, strictly increasing and finite;
- all S-parameter values finite;
- a port count of at least two.

Files whose port count is neither 2 nor 4 raise the exception `NeedPortMap`, which the interface answers by opening the port-mapping dialog (Section 3.3).

Source: *`si_analyzer.py` — `NeedPortMap`*

```
class NeedPortMap(Exception):
    """Raised when a file has a port count that needs a user-chosen mapping."""

    def __init__(self, nports: int) -> None:
        super().__init__(f"{nports}-port file needs a port mapping")
        self.nports = nports
```

Source: `si_analyzer.py` — `read_touchstone`

```
def read_touchstone(path: str | Path, pairing: str = "1-2 / 3-4", port_map: tuple | None =
    None) -> ChannelData:
    """Load an N-port Touchstone file through scikit-rf.

    2-port files load directly; 4-port files default to the differential Sdd block of `
    pairing`.
    Any other file (or an explicit `port_map`) selects the channel by mapping:
    ("se", in_port, out_port) or ("diff", in+, in-, out+, out-), 0-based.
    """
    file_path = Path(path)
    network = rf.Network(str(file_path))
    n = network.nports
    if n < 2:
        raise ValueError("A Touchstone file with at least 2 ports is required.")
    f = np.asarray(network.f, dtype=float)
    s = np.asarray(network.s, dtype=complex)
    z0 = np.asarray(network.z0, dtype=complex)
    if len(f) < 3 or not np.all(np.isfinite(f)) or np.any(np.diff(f) <= 0):
        raise ValueError("The Touchstone file must contain at least three increasing
            frequency points.")
    if not np.all(np.isfinite(s)):
        raise ValueError("The S-parameter file contains non-finite values.")
    if z0.ndim == 1:
        z0 = np.repeat(z0[:, None], n, axis=1)
    if port_map is None and n not in (2, 4):
        raise NeedPortMap(n)
    noise = None
    if n == 2:
        try:
            noise = parse_touchstone_noise(file_path)
        except Exception:
            noise = None
    if port_map is not None:
        if port_map[0] == "se":
            i, j = int(port_map[1]), int(port_map[2])
            if i == j or max(i, j) >= n or min(i, j) < 0:
                raise ValueError("Invalid single-ended port selection.")
            sel = [i, j]
            return ChannelData(file_path, f, s[:, sel][:, :, sel], z0[:, sel], ports=n,
                z0_full=z0, noise=noise,
                               note=f"single-ended, port {i + 1} → port {j + 1}")
        order = [int(q) for q in port_map[1:5]]
        if len(set(order)) != 4 or max(order) >= n or min(order) < 0:
            raise ValueError("Invalid differential port selection.")
        sub = s[:, order][:, :, order]
        z_d = 2.0 * np.column_stack([z0[:, order[0]], z0[:, order[2]]])
        o = [q + 1 for q in order]
        return ChannelData(file_path, f, mixed_mode(sub, "1-2 / 3-4")["sdd"], z_d, ports=n,
            s_full=sub, z0_full=z0[:, order],
                           pairing="1-2 / 3-4", note=f"differential Sdd, ports ({o[0]},{o
                               [1]}) → ({o[2]},{o[3]})")
    if n == 4:
        mm = mixed_mode(s, pairing)
        z0_diff = 2.0 * np.column_stack([z0[:, 0], z0[:, 2]])
        return ChannelData(file_path, f, mm["sdd"], z0_diff, ports=4, s_full=s, z0_full=z0,
            pairing=pairing)
    return ChannelData(file_path, f, s, z0, ports=2, noise=noise)
```

3.3. Selecting the Channel from an N-Port File

Real measurements often contain more ports than a channel needs: a four-port file of a differential link, an eight-port file of a connector with four lanes. The program offers three ways to obtain the active two-port:

Two-port file. Used as it is.

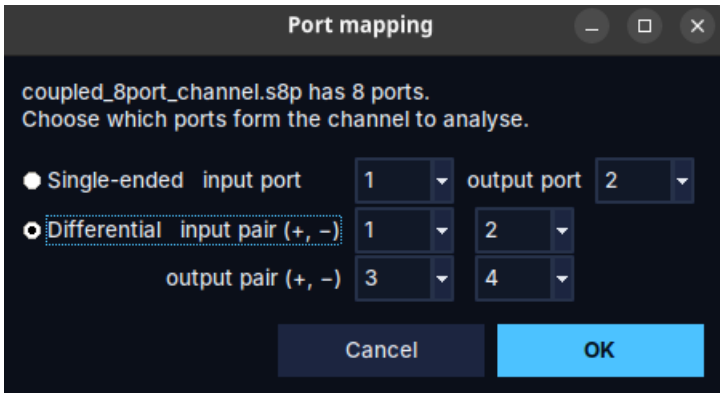
Four-port file, default. The differential transmission $\mathbf{S}_{dd}$ of two port pairs (Section 3.6), using the pairing chosen in the Mixed-mode tab.

Explicit mapping. The user picks either *single-ended*: an input port i and an output port j, giving the sub-matrix

$$\mathbf{S}_2 = \begin{bmatrix} S_{ii} & S_{ij} \\ S_{ji} & S_{jj} \end{bmatrix}, \tag{3.3}$$

or *differential*: four ports $(p_1^+, p_1^-, p_2^+, p_2^-)$, from which the 4×4 sub-matrix is extracted and converted to mixed-mode form.

For the differential case the reference impedance of the resulting differential ports is $2Z_0$, the standard value for a pair of single-ended lines of impedance Z_0. The dialog is part of the interface (Chapter 9).



3.4. Reference Impedance and Impedance from Reflection

A one-port reflection coefficient Γ measured against a reference impedance Z_0 corresponds to the impedance

$$Z = Z_0 \frac{1 + \Gamma}{1 - \Gamma}, \tag{3.4}$$

which is undefined for $\Gamma = 1$ (an ideal open). The function guards this singularity and returns `NaN` where $|1 - \Gamma| < 10^{-12}$. The same relation is used for the “Re Z_{11}” indicator, the impedance plot, and, with a time-dependent Γ, for the TDR impedance profile (Chapter 5).

Source: `si_analyzer.py` — `impedance_from_reflection`

```
def impedance_from_reflection(gamma: np.ndarray, reference_z: np.ndarray) -> np.ndarray:
    """Convert a one-port reflection coefficient to impedance."""
    denominator = 1.0 - gamma
    with np.errstate(divide="ignore", invalid="ignore"):
        z = reference_z * (1.0 + gamma) / denominator
    z[np.abs(denominator) < 1e-12] = np.nan + 1j * np.nan
    return z
```

Changing the reference impedance of a network (for example from $50\,\Omega$ to $100\,\Omega$ for a differential channel) is a re-expression of the same physical network with respect to different port normalizations. The program offers it in the Conditioning tool and delegates the computation to scikit-rf, which implements the power-wave renormalization [**kurokawa1965**].

3.5. Conversion to Other Network Representations

For a two-port with equal real reference impedance Z_0 the following representations are computed from $\mathbf{S}$ at every frequency. The impedance and admittance matrices are

$$\mathbf{Z} = Z_0\,(\mathbf{I}-\mathbf{S})^{-1}(\mathbf{I}+\mathbf{S}), \tag{3.5}$$

$$\mathbf{Y} = \frac{1}{Z_0}\,(\mathbf{I}+\mathbf{S})^{-1}(\mathbf{I}-\mathbf{S}) = \mathbf{Z}^{-1}. \tag{3.6}$$

Both exist only if $\mathbf{I}\mp\mathbf{S}$ is non-singular; points where it is singular (for example a lossless open or short) are returned as `NaN`. The chain (ABCD) matrix, defined by $\begin{bmatrix}V_1\\I_1\end{bmatrix} = \begin{bmatrix}A & B\\C & D\end{bmatrix}\begin{bmatrix}V_2\\-I_2\end{bmatrix}$, follows from [**pozar2012**]

$$A = \frac{(1+S_{11})(1-S_{22})+S_{12}S_{21}}{2S_{21}}, \qquad B = Z_0\,\frac{(1+S_{11})(1+S_{22})-S_{12}S_{21}}{2S_{21}}, \tag{3.7}$$

$$C = \frac{1}{Z_0}\,\frac{(1-S_{11})(1-S_{22})-S_{12}S_{21}}{2S_{21}}, \qquad D = \frac{(1-S_{11})(1+S_{22})+S_{12}S_{21}}{2S_{21}}. \tag{3.8}$$

The scattering-transfer matrix $\mathbf{T}$ relates the waves at port 1 to those at port 2, $\begin{bmatrix}b_1\\a_1\end{bmatrix} = \mathbf{T}\begin{bmatrix}a_2\\b_2\end{bmatrix}$:

$$\mathbf{T} = \frac{1}{S_{21}}\begin{bmatrix}-\det\mathbf{S} & S_{11}\\ -S_{22} & 1\end{bmatrix}. \tag{3.9}$$

Its useful property is that the $\mathbf{T}$-matrix of a cascade is the product of the individual matrices, which Chapter 8 uses for de-embedding. Finally the hybrid matrices are obtained from the admittance matrix,

$$\mathbf{H} = \begin{bmatrix}\dfrac{1}{Y_{11}} & -\dfrac{Y_{12}}{Y_{11}}\\[2mm] \dfrac{Y_{21}}{Y_{11}} & \dfrac{\det\mathbf{Y}}{Y_{11}}\end{bmatrix}, \qquad \mathbf{G} = \mathbf{H}^{-1}. \tag{3.10}$$

The conventions are printed in the title of the Parameters tab, because $\mathbf{T}$, $\mathbf{H}$ and $\mathbf{G}$ are defined differently in different textbooks. When the two ports have different reference impedances the average $Z_0 = \tfrac{1}{2}(Z_{01}+Z_{02})$ is used and the title states it.

Source: `si_analyzer.py` — `convert_parameters`

```
def convert_parameters(s: np.ndarray, z0: float) -> dict[str, np.ndarray]:
    """S -> Z, Y, ABCD, T, H, G for a 2-port with equal real reference impedance z0."""
    n = s.shape[0]
    eye = np.broadcast_to(np.eye(2), (n, 2, 2))
    z = z0 * _solve(eye - s, eye + s)
    y = _solve(eye + s, eye - s) / z0
    s11, s12, s21, s22 = s[:, 0, 0], s[:, 0, 1], s[:, 1, 0], s[:, 1, 1]
    a = np.empty_like(s)
    t = np.empty_like(s)
    h = np.empty_like(s)
    with np.errstate(all="ignore"):
        den = 2.0 * s21
        a[:, 0, 0] = ((1 + s11) * (1 - s22) + s12 * s21) / den
        a[:, 0, 1] = z0 * ((1 + s11) * (1 + s22) - s12 * s21) / den
        a[:, 1, 0] = ((1 - s11) * (1 - s22) - s12 * s21) / den / z0
        a[:, 1, 1] = ((1 - s11) * (1 + s22) + s12 * s21) / den
        det_s = s11 * s22 - s12 * s21
        t[:, 0, 0] = -det_s / s21
        t[:, 0, 1] = s11 / s21
        t[:, 1, 0] = -s22 / s21
```

```
        t[:, 1, 1] = 1.0 / s21
        y11, y12, y21, y22 = y[:, 0, 0], y[:, 0, 1], y[:, 1, 0], y[:, 1, 1]
        h[:, 0, 0] = 1.0 / y11
        h[:, 0, 1] = -y12 / y11
        h[:, 1, 0] = y21 / y11
        h[:, 1, 1] = (y11 * y22 - y12 * y21) / y11
    g = _solve(h, eye)
    return {"S": s, "Z": z, "Y": y, "ABCD": a, "T": t, "H": h, "G": g}
```

The `Parameters` tab lets the user choose one of the seven sets and how to display it (magnitude, dB, unwrapped phase, real or imaginary part); the display transformation is

Source: `si_analyzer.py` — `_format_values`

```
def _format_values(v: np.ndarray, fmt: str) -> np.ndarray:
    if fmt == "Magnitude (dB)":
        return _db(v)
    if fmt == "Magnitude":
        return np.abs(v)
    if fmt == "Phase (°)":
        return np.degrees(np.unwrap(np.angle(np.where(np.isfinite(v), v, 0.0))))
    if fmt == "Real":
        return np.real(v)
    return np.imag(v)
```

3.6. Mixed-Mode S-Parameters

Differential signalling transmits information as the *difference* of two conductors, with unwanted disturbances appearing as a *common* component. For a pair of ports (p, n) the differential and common-mode waves are defined by

$$a_d = \frac{a_p - a_n}{\sqrt{2}}, \qquad a_c = \frac{a_p + a_n}{\sqrt{2}}, \tag{3.11}$$

and analogously for b [**bockelman1995**]. For two port pairs, with the wave vector ordered $(a_1^+, a_1^-, a_2^+, a_2^-)$, the transformation is the real orthogonal matrix

$$\mathbf{M} = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 & -1 & 0 & 0 \\ 0 & 0 & 1 & -1 \\ 1 & 1 & 0 & 0 \\ 0 & 0 & 1 & 1 \end{bmatrix}, \qquad \mathbf{M}\mathbf{M}^{T} = \mathbf{I}, \tag{3.12}$$

so the mixed-mode scattering matrix is obtained by a similarity transformation,

$$\mathbf{S}_{mm} = \mathbf{M}\,\mathbf{S}\,\mathbf{M}^{T} = \begin{bmatrix} \mathbf{S}_{dd} & \mathbf{S}_{dc} \\ \mathbf{S}_{cd} & \mathbf{S}_{cc} \end{bmatrix}. \tag{3.13}$$

$\mathbf{S}_{dd}$ describes the desired differential transmission, $\mathbf{S}_{cc}$ the common-mode transmission and the off-diagonal blocks the conversion between the modes (for example $S_{cd,21}$: differential excitation producing a common-mode output, the mechanism by which imbalance creates emissions). Because $\mathbf{M}$ is orthogonal and the waves are normalized by $\sqrt{2}$, the mode ports have reference impedances $2Z_0$ (differential) and $Z_0/2$ (common) automatically.

Port ordering conventions differ between instruments, so the pairing is selectable: “1-2 / 3-4” means differential port 1 is formed by ports 1 and 2 and differential port 2 by ports 3 and 4; “1-3 / 2-4” pairs ports 1 with 3 and 2 with 4. The pairing is implemented as a permutation of the rows and columns of $\mathbf{S}$ before the transformation.

Source: `si_analyzer.py` — `PAIRINGS`

```
PAIRINGS = {"1-2 / 3-4": [0, 1, 2, 3], "1-3 / 2-4": [0, 2, 1, 3]}
```

Source: `si_analyzer.py` — `mixed_mode`

```
def mixed_mode(s4: np.ndarray, pairing: str = "1-2 / 3-4") -> dict[str, np.ndarray]:
    """Single-ended 4-port -> mixed-mode 2x2 blocks (Sdd, Sdc, Scd, Scc)."""
    order = PAIRINGS[pairing]
    sp = s4[:, order][:, :, order]
    r = 1.0 / math.sqrt(2.0)
    m = r * np.array([[1, -1, 0, 0], [0, 0, 1, -1], [1, 1, 0, 0], [0, 0, 1, 1]], dtype=float)
    smm = m @ sp @ m.T
    return {"sdd": smm[:, :2, :2], "sdc": smm[:, :2, 2:], "scd": smm[:, 2:, :2], "scc": smm
        [:, 2:, 2:]}
```

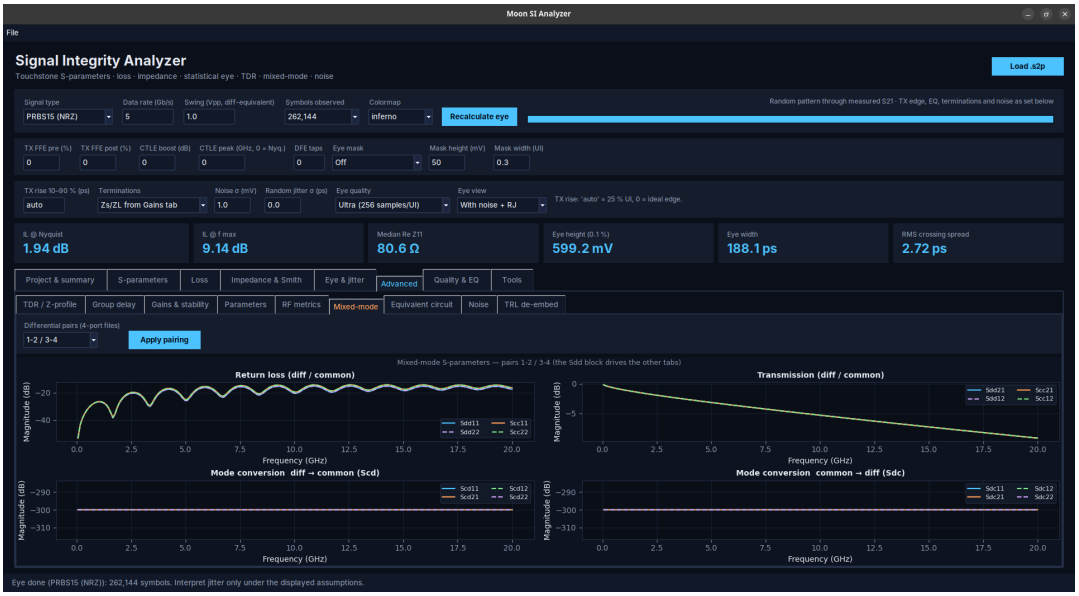


Figure 3.1: Mixed-mode analysis of a four-port differential channel.

3.7. Noise Parameters in the File

Two-port Touchstone files may end with a noise block whose lines contain five numbers: the frequency, the minimum noise figure $NF_{\min}$ in dB, the magnitude and angle (in degrees) of the optimum source reflection coefficient Γ_{opt}, and the equivalent noise resistance normalized to the reference resistance, $r_n = R_n/R_0$. A line with five numbers cannot be confused with a data line of a two-port (nine numbers), which makes the block easy to recognize. The parser also reads the option line for the frequency unit and the reference resistance R_0, so that $R_n = r_n R_0$. The noise-parameter theory is discussed in Section 4.7.

Source: `si_analyzer.py` — `parse_touchstone_noise`

```
def parse_touchstone_noise(path: str | Path) -> dict[str, Any] | None:
    """Read the optional noise block (freq, NFmin dB, |Gopt|, angle, Rn/R0) of a 2-port
        Touchstone file."""
    units = {"HZ": 1.0, "KHZ": 1e3, "MHZ": 1e6, "GHZ": 1e9}
    unit, ref = 1e9, 50.0
    rows: list[list[float]] = []
    with open(path, "r", errors="ignore") as fh:
        for raw in fh:
            line = raw.split("!")[0].strip()
            if not line:
                continue
            if line.startswith("#"):
                tok = line[1:].upper().split()
                for i, t in enumerate(tok):
                    if t in units:
                        unit = units[t]
                    elif t == "R" and i + 1 < len(tok):
```

```
                        try:
                            ref = float(tok[i + 1])
                        except ValueError:
                            pass
                continue
            try:
                vals = [float(t) for t in line.split()]
            except ValueError:
                continue
            if len(vals) == 5:
                rows.append(vals)
    if not rows:
        return None
    a = np.array(sorted(rows))
    return {
        "freq_hz": a[:, 0] * unit,
        "nfmin_db": a[:, 1],
        "gopt": a[:, 2] * np.exp(1j * np.deg2rad(a[:, 3])),
        "rn": a[:, 4] * ref,
        "ref": ref,
    }
```

4

Frequency-Domain Analysis

The frequency-domain tools turn the S-parameters directly into the figures of merit used in channel specifications. This chapter explains each of them: loss and reflection metrics, group delay, passivity and reciprocity, power gain and stability, noise, the Smith chart and its circles, and impedance matching.

4.1. S-Parameter, Loss and Impedance Views

The first three tabs show the channel as measured. The S-parameter tab plots the magnitude in decibels and the *unwrapped* phase of all four parameters:

$$|S_{ij}|_{\mathrm{dB}} = 20\log_{10}|S_{ij}|, \qquad \varphi_{ij} = \operatorname{unwrap}\big(\arg S_{ij}\big). \tag{4.1}$$

A floor of 10^{-15} is applied to the magnitude before the logarithm so that exact zeros plot as $-300\,\mathrm{dB}$ instead of producing an error. The loss tab shows the insertion loss and marks the Nyquist frequency of the selected signal:

$$\mathrm{IL}(f) = -20\log_{10}|S_{21}(f)|, \qquad f_{\mathrm{Nyq}} = \frac{1}{2T} = \frac{R_b}{2\beta_s}, \tag{4.2}$$

where T is the unit interval, R_b the data rate and β_s the number of bits per symbol of the chosen signal type (Chapter 6). The Nyquist frequency therefore depends on the signal: PAM4 carries two bits per symbol and has half the Nyquist frequency of NRZ at the same bit rate. The impedance tab shows the real part and magnitude of the impedance seen at port 1 and port 2, Equation (3.4).

Source: `si_analyzer.py` — `si_analyzer.py`, lines 3405–3420

```
colors = {"S11": ACCENT, "S21": ACCENT2, "S12": GREEN, "S22": PURPLE}
self.ax_s_mag.clear()
self.ax_s_phase.clear()
for row, col, name in ((0, 0, "S11"), (1, 0, "S21"), (0, 1, "S12"), (1, 1, "S22")):
    values = d.s[:, row, col]
    ls = "--" if name in ("S12", "S22") else "-"
    self.ax_s_mag.plot(f_ghz, 20 * np.log10(np.maximum(np.abs(values), 1e-15)), ls, color=
        colors[name], label=name)
    self.ax_s_phase.plot(f_ghz, np.unwrap(np.angle(values)) * 180 / np.pi, ls, color=colors[
        name], label=name)
self.ax_s_mag.set_ylabel("Magnitude (dB)")
self.ax_s_phase.set_ylabel("Unwrapped phase (°)")
self.ax_s_phase.set_xlabel("Frequency (GHz)")
self.ax_s_mag.set_title("Two-port S-parameters")
for ax in (self.ax_s_mag, self.ax_s_phase):
    ax.grid(True)
    ax.legend(ncol=4, loc="best")
self.canvas_s.draw_idle()
```

Source: `si_analyzer.py` — `SIAnalyzerApp._draw_loss`

```
def _draw_loss(self) -> None:
    if self.channel is None:
        return
    d = self.channel
    f_ghz = d.frequency_hz / 1e9
    il = -20 * np.log10(np.maximum(np.abs(d.s[:, 1, 0]), 1e-15))
    self.ax_loss.clear()
    self.ax_loss.fill_between(f_ghz, 0, il, color=ACCENT, alpha=0.12)
    self.ax_loss.plot(f_ghz, il, color=ACCENT, label="Insertion loss = -20 log10 |S21|")
    nyq = self._nyquist_hz()
    if nyq is not None and d.frequency_hz[0] <= nyq <= d.frequency_hz[-1]:
        self.ax_loss.axvline(nyq / 1e9, color=ACCENT2, linestyle="--", linewidth=1.2, label=f
            "Nyquist ({nyq/1e9:.3g} GHz)")
    self.ax_loss.axhline(0, color=MUTED, linewidth=0.7)
    self.ax_loss.set_title("Forward insertion loss")
    self.ax_loss.set_xlabel("Frequency (GHz)")
    self.ax_loss.set_ylabel("Insertion loss (dB)")
    self.ax_loss.grid(True)
    self.ax_loss.legend()
    self.canvas_loss.draw_idle()
```

Source: `si_analyzer.py` — `SIAnalyzerApp._nyquist_hz`

```
def _nyquist_hz(self) -> float | None:
    """Nyquist = symbol rate / 2 (symbol rate = data rate / bits-per-symbol of the chosen
        signal)."""
    try:
        rate = float(self.rate_var.get())
        bps = SIGNALS.get(self.signal_var.get(), SIGNALS[DEFAULT_SIGNAL])["bps"]
        return (rate / bps) * 1e9 / 2.0 if rate > 0 else None
    except ValueError:
        return None
```

4.2. RF Metrics

The RF-metrics tab and the summary report derive the following quantities from the S-parameters.

Return loss, VSWR and mismatch loss. For a reflection coefficient magnitude $|\Gamma|$,

$$\mathrm{RL} = -20\log_{10}|\Gamma|, \qquad \mathrm{VSWR} = \frac{1+|\Gamma|}{1-|\Gamma|}, \qquad \mathrm{ML} = -10\log_{10}\!\left(1-|\Gamma|^2\right). \tag{4.3}$$

The magnitude is limited to 0.9999 before computing VSWR and mismatch loss so that a total reflection yields a large finite number rather than infinity.

-3 dB bandwidth. The bandwidth is defined relative to the insertion loss at the first measured frequency f_1: it is the lowest frequency at which $\mathrm{IL}(f) \geq \mathrm{IL}(f_1) + 3\,\mathrm{dB}$, found by linear interpolation between the two bracketing points. If the loss never grows that much, the report states that the bandwidth is not reached inside the measured band.

Passivity. A passive network cannot create power, which for the scattering matrix means that its largest singular value does not exceed one at any frequency,

$$\sigma_{\max}\!\left(\mathbf{S}(f)\right) \leq 1 \quad \forall f. \tag{4.4}$$

It is evaluated by a batched singular-value decomposition. Violations beyond measurement tolerance indicate errors in the data (a calibration problem, or numerical noise of a field solver) and make time-domain results unreliable [**triverio2007**].

Reciprocity. For networks without non-reciprocal materials $\mathbf{S} = \mathbf{S}^{T}$. The tab plots $|S_{21} - S_{12}|$ in dB.

Source: `si_analyzer.py` — `passivity`

```
def passivity(s: np.ndarray) -> np.ndarray:
    """Largest singular value of S at each frequency (<= 1 for a passive network)."""
    return np.linalg.svd(s, compute_uv=False)[:, 0]
```

Source: `si_analyzer.py` — `rf_metrics_lines`

```
def rf_metrics_lines(f: np.ndarray, s: np.ndarray) -> list[str]:
    s11, s12, s21, s22 = s[:, 0, 0], s[:, 0, 1], s[:, 1, 0], s[:, 1, 1]
    g = f / 1e9
    il = -_db(s21)
    lines = [
        f"Insertion loss: min {il.min():.2f} dB @ {g[il.argmin()]:.4g} GHz, max {il.max():.2f} dB @ {g[il.argmax()]:.4g} GHz",
    ]
    idx = np.flatnonzero(il >= il[0] + 3.0)
    if len(idx):
        i = int(idx[0])
        f3 = f[0] if i == 0 else float(np.interp(il[0] + 3.0, il[i - 1:i + 1], f[i - 1:i + 1]))
        lines.append(f"-3 dB bandwidth (relative to IL at {g[0]:.3g} GHz): {f3/1e9:.4g} GHz")
    else:
        lines.append("-3 dB bandwidth: not reached inside the measured band")
    for name, sp in (("S11", s11), ("S22", s22)):
        mag = np.minimum(np.abs(sp), 0.9999)
        rl = -_db(sp)
        lines.append(
            f"{name}: worst RL {rl.min():.2f} dB @ {g[rl.argmin()]:.4g} GHz, mean RL {rl.mean():.2f} dB, "
            f"max VSWR {np.max((1+mag)/(1-mag)):.2f}, max mismatch loss {abs(np.max(-10*np.log10(1-mag**2))):.3f} dB")
    lines.append(f"Passivity: max singular value of S = {passivity(s).max():.4f} (passive if <= 1)")
    lines.append(f"Reciprocity: max |S21 - S12| = {_db(s21 - s12).max():.1f} dB")
    return lines
```

4.3. Group Delay and Phase Linearity

The group delay is the delay experienced by the envelope of a narrow-band signal,

$$\tau_g(f) = -\frac{d\varphi(\omega)}{d\omega} = -\frac{1}{2\pi}\frac{d\varphi}{df}. \tag{4.5}$$

The program unwraps the phase and differentiates numerically with second-order central differences on the (possibly non-uniform) frequency grid. For a lossless line of length ℓ and effective permittivity $\varepsilon_{r,\mathrm{eff}}$, $\tau_g = \ell\sqrt{\varepsilon_{r,\mathrm{eff}}}/c$, which is one of the verification checks (Chapter 11). A constant group delay means the channel preserves the shape of a wide-band signal; variations of τ_g across the signal band cause dispersion. The phase-linearity plot shows the deviation of the S21 phase from its least-squares straight line,

$$\Delta\varphi(f) = \varphi_{21}(f) - (af + b), \tag{4.6}$$

computed by a first-order polynomial fit. The three panels of the tab show the transmission group delay (S21 and S12), the reflection group delay (S11) and this deviation.

Source: `si_analyzer.py` — `group_delay`

```
def group_delay(f: np.ndarray, s: np.ndarray) -> np.ndarray:
    """Group delay in seconds: -d(phase)/d(omega)."""
    phase = np.unwrap(np.angle(s))
    return -np.gradient(phase, 2.0 * np.pi * np.asarray(f, dtype=float))
```

Source: `si_analyzer.py` — `AdvancedTabs._draw_gd`

```
def _draw_gd(self) -> None:
    if self.channel is None:
        return self._placeholder("gd", "Load a Touchstone file first.")
    d, f = self.channel, self.channel.frequency_hz
    g = f / 1e9
    gd21, gd12 = group_delay(f, d.s[:, 1, 0]) * 1e12, group_delay(f, d.s[:, 0, 1]) * 1e12
    gd11 = group_delay(f, d.s[:, 0, 0]) * 1e12
    phase = np.degrees(np.unwrap(np.angle(d.s[:, 1, 0])))
    dev = phase - np.polyval(np.polyfit(f, phase, 1), f)
    fig = self.fig_gd
    fig.clf()
    ax1, ax2, ax3 = fig.subplots(1, 3)
    ax1.plot(g, gd21, color=ACCENT2, label="S21")
    ax1.plot(g, gd12, color=GREEN, linestyle="--", label="S12")
    ax1.set_title("Transmission group delay")
    ax1.set_ylabel(" g (ps)")
    ax2.plot(g, gd11, color=ACCENT, label="S11")
    ax2.set_title("Reflection group delay")
    ax2.set_ylabel(" g (ps)")
    ax3.plot(g, dev, color=PURPLE, label="S21 phase - linear fit")
    ax3.set_title("Phase linearity")
    ax3.set_ylabel("Deviation (°)")
    for ax in (ax1, ax2, ax3):
        ax.set_xlabel("Frequency (GHz)")
        ax.grid(True)
        ax.legend(loc="best")
    self._finish("gd")
    self._set_section("gd", "Group delay (S21)", [
        f"Mean {np.nanmean(gd21):.2f} ps , min {np.nanmin(gd21):.2f} ps , max {np.nanmax(gd21)
            :.2f} ps , "
        f"variation (p-p) {np.nanmax(gd21) - np.nanmin(gd21):.2f} ps",
        f"Max deviation from linear phase: {np.max(np.abs(dev)):.2f}°",
    ])
```

4.4. Terminations

Gain, noise and the eye with terminations depend on the impedances that the source and load present. They are entered as resistance and reactance, $Z = R + jX$, and converted to a reflection coefficient relative to the port reference impedance,

$$\Gamma_s = \frac{Z_s - Z_{01}}{Z_s + Z_{01}}, \qquad \Gamma_L = \frac{Z_L - Z_{02}}{Z_L + Z_{02}}. \tag{4.7}$$

Source: `si_analyzer.py` — `AdvancedTabs._terminations`

```
def _terminations(self) -> tuple[complex, complex]:
    z1, z2 = self._z0s()
    zs = complex(self._fv(self.rs_var, z1), self._fv(self.xs_var, 0.0))
    zl = complex(self._fv(self.rl_var, z2), self._fv(self.xl_var, 0.0))
    return (zs - z1) / (zs + z1), (zl - z2) / (zl + z2)
```

4.5. Power Gains

With a source reflection coefficient Γ_s and a load reflection coefficient Γ_L the input and output reflection coefficients of the two-port are [**pozar2012**]

$$\Gamma_{in} = S_{11} + \frac{S_{12}S_{21}\Gamma_L}{1 - S_{22}\Gamma_L}, \qquad \Gamma_{out} = S_{22} + \frac{S_{12}S_{21}\Gamma_s}{1 - S_{11}\Gamma_s}. \tag{4.8}$$

The three standard power gains are

$$G_T = \frac{(1-|\Gamma_s|^2)\,|S_{21}|^2\,(1-|\Gamma_L|^2)}{|1-\Gamma_s\Gamma_{in}|^2\,|1-S_{22}\Gamma_L|^2}, \tag{4.9}$$

$$G_A = \frac{|S_{21}|^2\,(1-|\Gamma_s|^2)}{|1-S_{11}\Gamma_s|^2\,(1-|\Gamma_{out}|^2)}, \tag{4.10}$$

$$G_P = \frac{|S_{21}|^2\,(1-|\Gamma_L|^2)}{(1-|\Gamma_{in}|^2)\,|1-S_{22}\Gamma_L|^2}, \tag{4.11}$$

the transducer, available and operating power gain. For matched terminations ($\Gamma_s = \Gamma_L = 0$) the transducer gain reduces to $|S_{21}|^2$, which is another verification check. The maximum stable gain and, for unconditionally stable devices, the maximum available gain are

$$\mathrm{MSG} = \frac{|S_{21}|}{|S_{12}|}, \qquad \mathrm{MAG} = \mathrm{MSG}\left(K - \sqrt{K^2-1}\right) \quad (K>1), \tag{4.12}$$

and Mason's unilateral gain is [**mason1954**]

$$U = \frac{\left|S_{21}/S_{12} - 1\right|^2}{2K\,|S_{21}/S_{12}| - 2\,\mathrm{Re}\!\left(S_{21}/S_{12}\right)}. \tag{4.13}$$

For passive channels all gains are below unity (negative in dB); the quantities are still computed because they quantify the interaction of the channel with mismatched terminations.

Source: `si_analyzer.py` — `power_gains`

```python
def power_gains(s: np.ndarray, gs: complex, gl: complex) -> dict[str, np.ndarray]:
    """Transducer (GT), available (GA) and operating (GP) power gain for given terminations."""
    s11, s12, s21, s22 = s[:, 0, 0], s[:, 0, 1], s[:, 1, 0], s[:, 1, 1]
    with np.errstate(all="ignore"):
        gin = s11 + s12 * s21 * gl / (1.0 - s22 * gl)
        gout = s22 + s12 * s21 * gs / (1.0 - s11 * gs)
        gt = (1 - abs(gs) ** 2) * np.abs(s21) ** 2 * (1 - abs(gl) ** 2) / (
            np.abs(1 - gs * gin) ** 2 * np.abs(1 - s22 * gl) ** 2)
        ga = np.abs(s21) ** 2 * (1 - abs(gs) ** 2) / (np.abs(1 - s11 * gs) ** 2 * (1 - np.abs(gout) ** 2))
        gp = np.abs(s21) ** 2 * (1 - abs(gl) ** 2) / ((1 - np.abs(gin) ** 2) * np.abs(1 - s22 * gl) ** 2)
    return {"GT": gt, "GA": ga, "GP": gp}
```

4.6. Stability Factors

A two-port is *unconditionally stable* if no passive source and load can make it oscillate. Using the determinant $\Delta = S_{11}S_{22} - S_{12}S_{21}$, Rollett's factor is [**rollett1962**]

$$K = \frac{1 - |S_{11}|^2 - |S_{22}|^2 + |\Delta|^2}{2\,|S_{12}S_{21}|}, \tag{4.14}$$

and the device is unconditionally stable if $K > 1$ *and* $|\Delta| < 1$. Edwards and Sinsky showed that a single parameter suffices [**edwards1992**]: the distance from the origin of the Smith chart to the nearest unstable region, normalized,

$$\mu = \frac{1-|S_{11}|^2}{|S_{22}-\Delta S_{11}^{*}| + |S_{12}S_{21}|}, \qquad \mu' = \frac{1-|S_{22}|^2}{|S_{11}-\Delta S_{22}^{*}| + |S_{12}S_{21}|}. \tag{4.15}$$

The condition $\mu > 1$ is equivalent to unconditional stability; μ' is the same test referred to the other port. This equivalence is exploited as a strong consistency check in the verification suite: on thousands of

random matrices the two criteria must always agree. The Gains tab plots K (logarithmic axis, with the limit $K = 1$), $|\Delta|$, μ and μ' together with the gains of Section 4.5, and the summary reports the verdict over the whole band.

Source: `si_analyzer.py` — `stability_metrics`

```python
def stability_metrics(s: np.ndarray) -> dict[str, np.ndarray]:
    s11, s12, s21, s22 = s[:, 0, 0], s[:, 0, 1], s[:, 1, 0], s[:, 1, 1]
    delta = s11 * s22 - s12 * s21
    p = np.abs(s12 * s21)
    with np.errstate(all="ignore"):
        k = (1.0 - np.abs(s11) ** 2 - np.abs(s22) ** 2 + np.abs(delta) ** 2) / (2.0 * p)
        mu = (1.0 - np.abs(s11) ** 2) / (np.abs(s22 - delta * np.conj(s11)) + p)
        mu_p = (1.0 - np.abs(s22) ** 2) / (np.abs(s11 - delta * np.conj(s22)) + p)
        msg = np.abs(s21) / np.abs(s12)
        mag = np.where(k > 1.0, msg * (k - np.sqrt(np.maximum(k * k - 1.0, 0.0))), np.nan)
        ratio = s21 / s12
        u = np.abs(ratio - 1.0) ** 2 / (2.0 * k * np.abs(ratio) - 2.0 * np.real(ratio))
    return {"K": k, "delta": delta, "mu": mu, "mu_p": mu_p, "MSG": msg, "MAG": mag, "U": u}
```

Source: `si_analyzer.py` — `AdvancedTabs._draw_gains`

```python
def _draw_gains(self) -> None:
    if self.channel is None:
        return self._placeholder("gains", "Load␣a␣Touchstone␣file␣first.")
    d, f = self.channel, self.channel.frequency_hz
    g = f / 1e9
    gs, gl = self._terminations()
    gains = power_gains(d.s, gs, gl)
    st = stability_metrics(d.s)
    fig = self.fig_gains
    fig.clf()
    ax1, ax2, ax3, ax4 = fig.subplots(2, 2).ravel()
    for key, col, ls in (("GT", ACCENT, "-"), ("GA", GREEN, "-"), ("GP", PURPLE, "-")):
        ax1.plot(g, _pdb(gains[key]), ls, color=col, label=f"{key}␣Γ(s={abs(gs):.2f},␣ΓL={abs
            (gl):.2f})" if key == "GT" else key)
    ax1.plot(g, _pdb(st["MSG"]), "--", color=MUTED, label="MSG")
    ax1.plot(g, _pdb(st["MAG"]), "--", color=ACCENT2, label="MAG")
    ax1.plot(g, _pdb(st["U"]), ":", color=RED, label="Mason␣U")
    ax1.set_title("Power␣gains")
    ax1.set_ylabel("Gain␣(dB)")
    ax2.semilogy(g, np.where(np.isfinite(st["K"]) & (st["K"] > 0), st["K"], np.nan), color=
        ACCENT, label="Rollett␣K")
    ax2.axhline(1.0, color=RED, linestyle="--", linewidth=1, label="K␣=␣1")
    ax2.set_title("K-factor")
    ax3.plot(g, np.abs(st["delta"]), color=ACCENT2, label="Δ||")
    ax3.axhline(1.0, color=RED, linestyle="--", linewidth=1, label="Δ||␣=␣1")
    ax3.set_title("Determinant␣Δ||")
    ax4.plot(g, st["mu"], color=GREEN, label="␣(load)")
    ax4.plot(g, st["mu_p"], color=PURPLE, linestyle="--", label="␣(source)")
    ax4.axhline(1.0, color=RED, linestyle="--", linewidth=1, label="␣=␣1")
    ax4.set_title("-EdwardsSinsky␣␣/␣")
    for ax in (ax1, ax2, ax3, ax4):
        ax.set_xlabel("Frequency␣(GHz)")
        ax.grid(True)
        ax.legend(loc="best", fontsize=8)
    self._finish("gains")

    k, delta = st["K"], st["delta"]
    ok_k = bool(np.all((k > 1) & (np.abs(delta) < 1)))
    ok_mu = bool(np.all(st["mu"] > 1))
    self._set_section("stab", "Stability␣&␣power␣gain", [
        f"Terminations:␣Γs␣=␣{abs(gs):.3f }{np.degrees(np.angle(gs)):.0f}°,␣ΓL␣=␣{abs(gl):.3f
            }{np.degrees(np.angle(gl)):.0f}°",
        f"Unconditionally␣stable␣(K␣>␣1␣and␣Δ||␣<␣1)␣over␣the␣band:␣{'yes'␣if␣ok_k␣else␣'no
            '};␣␣>␣1␣everywhere:␣{'yes'␣if␣ok_mu␣else␣'no'}",
        f"K␣min␣{np.nanmin(k):.3g},␣Δ||␣max␣{np.nanmax(np.abs(delta)):.3g},␣␣min␣{np.nanmin(
            st['mu']):.3g},␣␣min␣{np.nanmin(st['mu_p']):.3g}",
        f"At␣{g[-1]:.4g}␣GHz:␣GT␣{_pdb(gains['GT'])[-1]:.2f}␣dB,␣GA␣{_pdb(gains['GA'])[-1]:.2
            f}␣dB,␣GP␣{_pdb(gains['GP'])[-1]:.2f}␣dB",
```

```
    ])
```

4.7. Noise Parameters and Noise Figure

A noisy linear two-port is characterized by four noise parameters: $F_{\min}$, R_n and the complex Γ_{opt} [**rothe1956**, **haus1960**]. The noise factor presented to a source with reflection coefficient Γ_s is

$$F = F_{\min} + \frac{4R_n}{Z_0}\,\frac{|\Gamma_s - \Gamma_{opt}|^2}{\left(1 - |\Gamma_s|^2\right)\,|1 + \Gamma_{opt}|^2}, \qquad NF = 10\log_{10} F. \tag{4.16}$$

When the file contains a noise block (Section 3.7), the Noise tab plots $NF_{\min}$, R_n, $|\Gamma_{opt}|$, $\angle\Gamma_{opt}$ and the noise figure for the source chosen in the Gains tab. When no noise block exists, a *passive* channel at the reference temperature $T_0 = 290\,\mathrm{K}$ has the noise factor equal to the reciprocal of its available gain,

$$F = \frac{1}{G_A}, \qquad NF = -10\log_{10} G_A, \tag{4.17}$$

which the tab shows as an estimate together with the maximum singular value of $\mathbf{S}$, because the formula is only valid for passive networks.

Source: `si_analyzer.py` — `noise_figure_db`

```
def noise_figure_db(noise: dict[str, Any], gs: complex) -> np.ndarray:
    fmin = 10.0 ** (noise["nfmin_db"] / 10.0)
    gopt = noise["gopt"]
    with np.errstate(all="ignore"):
        f = fmin + 4.0 * (noise["rn"] / noise["ref"]) * np.abs(gs - gopt) ** 2 / (
            (1.0 - abs(gs) ** 2) * np.abs(1.0 + gopt) ** 2)
    return 10.0 * np.log10(f)
```

Source: `si_analyzer.py` — `AdvancedTabs._draw_noise`

```
def _draw_noise(self) -> None:
    d = self.channel
    if d is None:
        return self._placeholder("noise", "Load␣a␣Touchstone␣file␣first.")
    gs, _ = self._terminations()
    fig = self.fig_noise
    fig.clf()
    nz = d.noise
    if nz is None:
        ax = fig.add_subplot(111)
        nf = -_pdb(power_gains(d.s, gs, 0j)["GA"])
        g = d.frequency_hz / 1e9
        ax.plot(g, nf, color=ACCENT2, label="-10␣log10(GA)␣␣(passive,␣290␣K)")
        ax.set_title("No␣noise␣block␣in␣this␣file␣-␣passive-channel␣noise-figure␣estimate")
        ax.set_xlabel("Frequency␣(GHz)")
        ax.set_ylabel("Noise␣figure␣(dB)")
        ax.grid(True)
        ax.legend(loc="best")
        self._finish("noise")
        sig = float(passivity(d.s).max())
        self._set_section("noise", "Noise", [
            "No␣noise-parameter␣block␣found␣in␣the␣Touchstone␣file.",
            f"Passive␣estimate␣NF␣=␣-10␣log10(GA)␣at␣290␣K:␣{np.nanmin(nf):.2f}␣to␣{np.nanmax
                (nf):.2f}␣dB␣"
            f"(valid␣only␣for␣a␣passive␣network;␣max␣(S)␣=␣{sig:.3f}).",
        ])
        return
    gnf = nz["freq_hz"] / 1e9
    nf_gs = noise_figure_db(nz, gs)
    ax1, ax2, ax3, ax4 = fig.subplots(2, 2).ravel()
    ax1.plot(gnf, nz["nfmin_db"], "o-", color=ACCENT, label="NFmin")
    ax1.plot(gnf, nf_gs, "s--", color=ACCENT2, label=f"NF␣@␣source␣Γs␣=␣{abs(gs):.2f}")
```

```
    ax1.set_title("Noise figure")
    ax1.set_ylabel("dB")
    ax2.plot(gnf, nz["rn"], "o-", color=GREEN, label="Rn")
    ax2.set_title("Equivalent noise resistance")
    ax2.set_ylabel("Ω")
    ax3.plot(gnf, np.abs(nz["gopt"]), "o-", color=PURPLE, label="Γ|opt|")
    ax3.set_title("Optimum source reflection magnitude")
    ax4.plot(gnf, np.degrees(np.angle(nz["gopt"])), "o-", color=RED, label="Γopt")
    ax4.set_title("Optimum source reflection angle")
    ax4.set_ylabel("deg")
    for ax in (ax1, ax2, ax3, ax4):
        ax.set_xlabel("Frequency (GHz)")
        ax.grid(True)
        ax.legend(loc="best")
    self._finish("noise")
    self._set_section("noise", "Noise parameters", [
        f"{len(gnf)} noise points, {gnf[0]:.4g}-{gnf[-1]:.4g} GHz",
        f"NFmin {nz['nfmin_db'].min():.2f}-{nz['nfmin_db'].max():.2f} dB; NF at the chosen
            source {np.nanmin(nf_gs):.2f}-{np.nanmax(nf_gs):.2f} dB",
        f"Rn {nz['rn'].min():.2f}-{nz['rn'].max():.2f} Ω",
    ])
```

4.8. The Smith Chart

The Smith chart is the unit disc of the complex reflection-coefficient plane with a grid of constant normalized resistance and reactance. With $z = r + jx = Z/Z_0$, the lines of constant resistance r are circles with centre $r/(1+r)$ and radius $1/(1+r)$ on the real axis, and the lines of constant reactance x are circle arcs with centre $(1, 1/x)$ and radius $1/|x|$, clipped to the disc.

Frequency as colour. The S11 trace of a channel is a curve in this plane parameterized by frequency. To make the direction and rate of rotation visible, the trace is drawn as a gradient from blue at the lowest to red at the highest frequency: the curve is split into segments between consecutive points, each segment receives the colour of the mean frequency of its end points, and a colour bar shows the frequency scale. In Matplotlib this is a `LineCollection` with a linear colour map between the two end colours.

Source: `si_analyzer.py` — `si_analyzer.py`, lines 59–59

```
FREQ_CMAP = LinearSegmentedColormap.from_list("freq_blue_red", ["#2f7bff", "#ff3b3b"])
```

Source: `si_analyzer.py` — `SIAnalyzerApp._draw_smith`

```
def _draw_smith(self, ax: Any, gamma: np.ndarray, f_ghz: np.ndarray) -> None:
    # Drop the previous colorbar first, otherwise a new one is stacked on every reload.
    if self._smith_cbar is not None:
        try:
            self._smith_cbar.remove()
        except Exception:
            pass
        self._smith_cbar = None
    ax.clear()

    theta = np.linspace(0, 2 * np.pi, 720)
    for r in (0.2, 0.5, 1.0, 2.0, 5.0):
        c, rad = r / (1 + r), 1 / (1 + r)
        ax.plot(c + rad * np.cos(theta), rad * np.sin(theta), color=GRID, linewidth=0.9)
    for x in (0.2, 0.5, 1.0, 2.0, 5.0):
        for sgn in (1, -1):
            px = 1 + (1 / x) * np.cos(theta)
            py = sgn / x + (1 / x) * np.sin(theta)
            inside = px ** 2 + py ** 2 <= 1.0005
            ax.plot(np.where(inside, px, np.nan), np.where(inside, py, np.nan), color=GRID,
                linewidth=0.9)
    ax.plot(np.cos(theta), np.sin(theta), color=MUTED, linewidth=1.3)
    ax.axhline(0, color=GRID, linewidth=0.9)

```

```
    # S11 trace as a gradient line: blue (lowest f) -> red (highest f).
    pts = np.column_stack([np.real(gamma), np.imag(gamma)]).reshape(-1, 1, 2)
    segments = np.concatenate([pts[:-1], pts[1:]], axis=1)
    lc = LineCollection(segments, cmap=FREQ_CMAP, norm=Normalize(float(f_ghz[0]), float(f_ghz
        [-1])), linewidth=2.4)
    lc.set_array(0.5 * (f_ghz[:-1] + f_ghz[1:]))
    ax.add_collection(lc)

    self._smith_cbar = ax.figure.colorbar(lc, ax=ax, fraction=0.046, pad=0.03)
    self._smith_cbar.set_label("Frequency␣(GHz)")
    self._smith_cbar.outline.set_edgecolor(BORDER)

    ax.set_aspect("equal", adjustable="box")
    ax.set_xlim(-1.05, 1.05)
    ax.set_ylim(-1.05, 1.05)
    ax.set_title("S11␣on␣the␣Smith␣chart")
    ax.set_xlabel("RealΓ()")
    ax.set_ylabel("ImagΓ()")
```

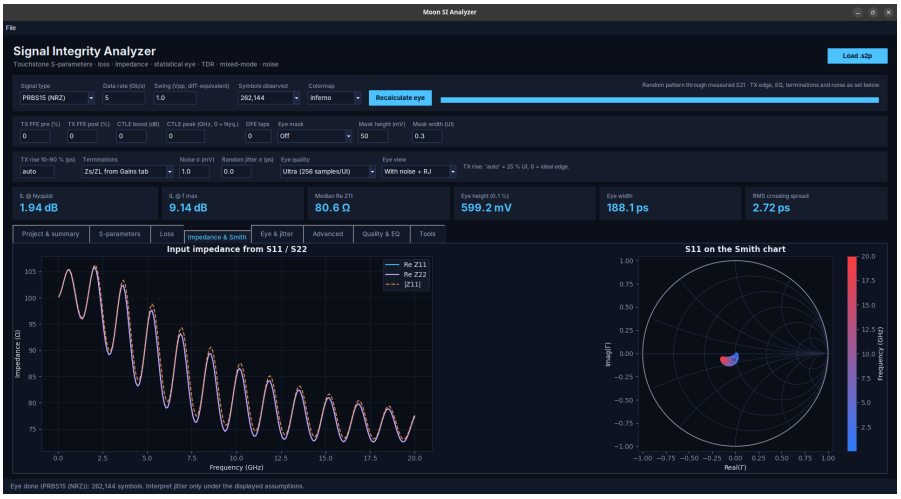


Figure 4.1: Smith chart with the S11 trace coloured by frequency.

4.9. Stability, Gain and Noise Circles

At one frequency the stability, gain and noise conditions can be drawn as circles in the source and load reflection-coefficient planes. For the source plane, the *source stability circle* is the locus where $|\Gamma_{out}| = 1$,

$$C_s = \frac{\left(S_{11} - \Delta S_{22}^{*}\right)^{*}}{|S_{11}|^2 - |\Delta|^2}, \qquad r_s = \frac{|S_{12}S_{21}|}{\left||S_{11}|^2 - |\Delta|^2\right|}, \tag{4.18}$$

and the *load stability circle* follows by exchanging the roles of the ports, $C_L = (S_{22} - \Delta S_{11}^{*})^{*}/(|S_{22}|^2 - |\Delta|^2)$ and $r_L = |S_{12}S_{21}|/\left||S_{22}|^2 - |\Delta|^2\right|$ **[pozar2012]**. Constant *available-gain circles* for a gain G_A with normalized gain $g_a = G_A/|S_{21}|^2$ have

$$C_a = \frac{g_a\left(S_{11} - \Delta S_{22}^{*}\right)^{*}}{1 + g_a(|S_{11}|^2 - |\Delta|^2)}, \qquad r_a = \frac{\sqrt{1 - 2K|S_{12}S_{21}|\,g_a + |S_{12}S_{21}|^2 g_a^2}}{\left|1 + g_a(|S_{11}|^2 - |\Delta|^2)\right|}, \tag{4.19}$$

and a circle exists only if the radicand is positive. *Constant noise-figure circles* follow from Equation (4.16). With $N = \dfrac{F - F_{\min}}{4R_n/Z_0}\,|1 + \Gamma_{opt}|^2$,

$$C_F = \frac{\Gamma_{opt}}{1 + N}, \qquad r_F = \frac{\sqrt{N^2 + N\left(1 - |\Gamma_{opt}|^2\right)}}{1 + N}. \tag{4.20}$$

The Smith+ tab draws the source-plane stability, gain and noise circles and the load-plane stability circle, with the conjugate reflection coefficients S_{11}^{*} and S_{22}^{*} marked.

Source: `si_analyzer.py` — `stability_circles`

```
def stability_circles(s: np.ndarray) -> tuple[tuple[complex, float], tuple[complex, float]]:
    """Source- and load-plane stability circles (centre, radius) for one 2x2 S matrix."""
    s11, s12, s21, s22 = s[0, 0], s[0, 1], s[1, 0], s[1, 1]
    d = s11 * s22 - s12 * s21
    with np.errstate(all="ignore"):
        cs = np.conj(s11 - d * np.conj(s22)) / (abs(s11) ** 2 - abs(d) ** 2)
        rs = abs(s12 * s21) / abs(abs(s11) ** 2 - abs(d) ** 2)
        cl = np.conj(s22 - d * np.conj(s11)) / (abs(s22) ** 2 - abs(d) ** 2)
        rl = abs(s12 * s21) / abs(abs(s22) ** 2 - abs(d) ** 2)
    return (complex(cs), float(rs)), (complex(cl), float(rl))
```

Source: `si_analyzer.py` — `available_gain_circle`

```
def available_gain_circle(s: np.ndarray, ga_lin: float) -> tuple[complex, float] | None:
    s11, s12, s21, s22 = s[0, 0], s[0, 1], s[1, 0], s[1, 1]
    d = s11 * s22 - s12 * s21
    k = float(stability_metrics(s[None])["K"][0])
    g = ga_lin / abs(s21) ** 2
    den = 1.0 + g * (abs(s11) ** 2 - abs(d) ** 2)
    rad2 = 1.0 - 2.0 * k * abs(s12 * s21) * g + abs(s12 * s21) ** 2 * g * g
    if not np.isfinite(rad2) or rad2 <= 0 or abs(den) < 1e-12:
        return None
    return complex(g * np.conj(s11 - d * np.conj(s22)) / den), float(math.sqrt(rad2) / abs(
        den))
```

Source: `si_analyzer.py` — `noise_circle`

```
def noise_circle(fmin_lin: float, gopt: complex, rn_norm: float, f_lin: float) -> tuple[
    complex, float] | None:
    n = (f_lin - fmin_lin) / (4.0 * rn_norm) * abs(1.0 + gopt) ** 2
    if not np.isfinite(n) or n < 0:
        return None
    return complex(gopt / (1.0 + n)), float(math.sqrt(max(n * n + n * (1.0 - abs(gopt) ** 2),
         0.0)) / (1.0 + n))
```

4.10. Impedance Matching with L-Sections

To match a load $Z_L = R + jX$ to a source of resistance R_0 with two reactive elements there are two topologies. In the first, a shunt susceptance B_p is placed across the load and a series reactance X_s is added toward the source. Writing the load admittance as $Y_L = G + jB$, the shunt element makes the total susceptance $B_t = B + B_p$ and the real part of the resulting impedance is $G/(G^2 + B_t^2)$. Setting it equal to R_0 gives

$$B_t = \pm\sqrt{\frac{G}{R_0} - G^2}, \qquad B_p = B_t - B, \qquad X_s = \frac{B_t R_0}{G}, \tag{4.21}$$

which is possible only if $G \le 1/R_0$. In the second topology a series reactance is added at the load first; with the total reactance $X_t = X + X_s$ the real part of the admittance is $R/(R^2 + X_t^2) = 1/R_0$, so

$$X_t = \pm\sqrt{R\,R_0 - R^2}, \qquad X_s = X_t - X, \qquad B_p = \frac{X_t}{R\,R_0}, \tag{4.22}$$

possible only if $R \le R_0$. Each topology therefore has up to two solutions (the two signs). The program evaluates the impedance seen at port 1 at the selected frequency, solves both topologies and prints the component values, converting a positive reactance to an inductance $L = X/\omega$, a negative reactance to a capacitance $C = -1/(\omega X)$, and likewise for susceptances ($C = B/\omega$ for $B > 0$, $L = -1/(\omega B)$ for $B < 0$). The verification suite checks that each solution transforms the load to R_0 exactly.

Source: `si_analyzer.py` — `l_match`

```
def l_match(z_load: complex, r0: float, w: float) -> list[str]:
    """Lumped L-section solutions matching z_load to r0 at angular frequency w."""
    sols: list[str] = []
    g, b = (1.0 / z_load).real, (1.0 / z_load).imag
    if 0 < g <= 1.0 / r0:
        for sgn in (1, -1):
            bt = sgn * math.sqrt(max(g / r0 - g * g, 0.0))
            sols.append(f"shunt {_b_text(bt - b, w)} across the load, then series {_x_text(bt * r0 / g, w)} toward the source")
    r, x = z_load.real, z_load.imag
    if 0 < r <= r0:
        for sgn in (1, -1):
            xt = sgn * math.sqrt(max(r * r0 - r * r, 0.0))
            sols.append(f"series {_x_text(xt - x, w)} at the load, then shunt {_b_text(xt / (r * r0), w)} toward the source")
    return sols
```

Source: `si_analyzer.py` — `ExtraTabs._draw_sm`

```
def _draw_sm(self) -> None:
    if self.channel is None:
        return self._placeholder("sm", "Load a Touchstone file first.")
    d, f = self.channel, self.channel.frequency_hz
    idx = int(np.argmin(np.abs(f - self._fv(self.sm_freq_var, 5.0) * 1e9)))
    f0, s = float(f[idx]), d.s[idx]
    z1 = self._z0s()[0]
    fig = self.fig_sm
    fig.clf()
    axs, axl, axt = fig.subplots(1, 3)
    self._smith_grid(axs, f"Source plane Γs @ {f0/1e9:.4g} GHz")
    self._smith_grid(axl, "Load plane ΓL")
    (cs, rs), (cl, rl) = stability_circles(s)
    for ax, c, r, lab in ((axs, cs, rs, "Source stability circle"), (axl, cl, rl, "Load stability circle")):
        if np.isfinite(r):
            self._circle(ax, c, r, color=RED, linewidth=1.6, label=lab)
    st = stability_metrics(s[None])
    lines = [f"At {f0/1e9:.4g} GHz: K = {float(st['K'][0]):.3g}, Δ|| = {abs(complex(st['delta'][0])):.3g}, μ = {float(st['mu'][0]):.3g}"]
    for k, txt in enumerate(self.sm_ga_var.get().replace(";", ",").split(",")):
        try:
            circ = available_gain_circle(s, 10.0 ** (float(txt) / 10.0))
        except ValueError:
            continue
        if circ is not None:
            self._circle(axs, circ[0], circ[1], color=GREEN, linewidth=1.1, label="Available-gain circles" if k == 0 else None)
    nz = d.noise
    if nz is not None and len(nz["freq_hz"]):
        fn = nz["freq_hz"]
        fmin = float(np.interp(f0, fn, nz["nfmin_db"]))
        gopt = complex(np.interp(f0, fn, nz["gopt"].real), np.interp(f0, fn, nz["gopt"].imag))
        rn = float(np.interp(f0, fn, nz["rn"])) / nz["ref"]
        axs.plot(gopt.real, gopt.imag, "*", color=ACCENT2, markersize=9, label="Γopt")
        for k, txt in enumerate(self.sm_nf_var.get().replace(";", ",").split(",")):
            try:
                nf = fmin + float(txt)
            except ValueError:
                continue
            circ = noise_circle(10.0 ** (fmin / 10.0), gopt, max(rn, 1e-9), 10.0 ** (nf / 10.0))
            if circ is not None:
                self._circle(axs, circ[0], circ[1], color=ACCENT2, linewidth=1.1, linestyle="--", label="Noise circles" if k == 0 else None)
        lines.append(f"NFmin {fmin:.2f} dB at this frequency, Γopt = {abs(gopt):.2f }{np.degrees(np.angle(gopt)):.0f}°")
    g11, g22 = s[0, 0], s[1, 1]
```

```
    axs.plot(np.conj(g11).real, np.conj(g11).imag, "o", color=ACCENT, markersize=5, label="
        S11*")
    axl.plot(np.conj(g22).real, np.conj(g22).imag, "o", color=ACCENT, markersize=5, label="
        S22*")
    for ax in (axs, axl):
        ax.legend(loc="lower left", fontsize=7)
    axt.axis("off")
    w = 2 * np.pi * f0
    zin = z1 * (1 + g11) / (1 - g11) if abs(1 - g11) > 1e-9 else complex("nan")
    lines.append(f"Zin (port 1) = {zin.real:.2f} {zin.imag:+.2f}j Ω; L-match to {z1:g} Ω:")
    sols = l_match(zin, z1, w) if np.isfinite(zin) else []
    lines += [f"  • {s_}" for s_ in sols] or ["  (no L-section solution for this load; use a
        different topology)"]
    axt.text(0.0, 1.0, "\n".join(lines), va="top", ha="left", color=FG, fontsize=8, transform
        =axt.transAxes, wrap=True)
    self._finish("sm")
    self._set_section("sm", "Smith + / matching", lines)
```

5

Time-Domain Analysis

A frequency-domain measurement contains all the information about where energy is reflected and how a pulse is delayed and distorted, but it is far easier to read in the time domain: a reflection from a via, a connector or a width change appears as a feature at the time (and therefore distance) at which it occurs. This chapter explains how the program computes time-domain responses from band-limited S-parameters, how it converts them to impedance profiles, how it finds discontinuities, and how it gates, extrapolates and resamples data.

5.1. From Frequency Data to a Time Response

5.1.1. The low-pass transform

A measured response $H(f)$ (an S-parameter) is sampled at N_f frequencies between f_1 and $f_{\max}$. Its impulse response is the inverse Fourier transform,

$$h(t) = \int_{-\infty}^{\infty} H(f)\, e^{j2\pi f t}\, df, \tag{5.1}$$

and the step response is its running integral,

$$s(t) = \int_{-\infty}^{t} h(\tau)\, d\tau. \tag{5.2}$$

Since $h(t)$ is real, $H(-f) = H^*(f)$ and only non-negative frequencies are needed. The program uses the *low-pass* transform, which treats the data as the lower part of the spectrum of a real signal; this requires the response at DC, which is generally not measured (Section 5.4).

5.1.2. Grid construction

The measured data are interpolated onto a uniform grid of $M = 8192$ points extending from DC to $f_{ext} = 4f_{\max}$. The extension is filled with zeros (after the windowing described below); it does not add information but makes the transform *interpolate* the time response with a finer sampling interval

$$\Delta t = \frac{1}{2 f_{ext}}. \tag{5.3}$$

The inverse real FFT then returns $N = 2(M-1)$ samples. With the discrete transform, the sample $x[n]$ equals $h(t_n)\Delta t$, so that the step response is simply the cumulative sum of the samples,

$$s[n] = \sum_{k \le n} x[k]. \tag{5.4}$$

A response $H(f) \equiv 1$ returns $x[n] = \delta[n]$ and therefore a unit step, which fixes the normalization. Only the first half of the time axis is returned, to avoid showing the circularly wrapped (aliased) second half.

5.1.3. Interpolation and DC handling

The grid starts at zero frequency but the measurement does not. The program extrapolates below the first point in a way that keeps the signal real and avoids a discontinuity: the real part is held constant (it is an even function of f), while the imaginary part (odd in f) is ramped linearly to zero at DC. Above the last measured point the response is zero. The first version of the program held the imaginary part constant, which created a step of the size of $\operatorname{Im} H(f_1)$ at DC; for a delay line this produces a slowly decaying error in the time domain. The verification campaign detected the effect (an error of $9\,\%$ in the settled value of a lossless-line TDT), and the correction reduced it to $1.4\,\%$ (Chapter 11).

Source: `si_analyzer.py` — `_interp_response`

```
def _interp_response(fg: np.ndarray, f: np.ndarray, h: np.ndarray) -> np.ndarray:
    """Interpolate a measured complex response onto the grid `fg` (starting at DC).

    The real part is held constant below the first measured point (even in f), the imaginary part is
    ramped linearly to zero at DC (odd in f, so the extrapolated signal stays real), and the response is
    zero above the last point. Avoids the step discontinuity a plain hold would create, which shows up
    as a slowly decaying error in the time domain when the first frequency is not DC.
    """
    f = np.asarray(f, dtype=float)
    h = np.asarray(h, dtype=complex)
    if f[0] > 0:
        f = np.concatenate([[0.0], f])
        re = np.concatenate([[h.real[0]], h.real])
        im = np.concatenate([[0.0], h.imag])
    else:
        re, im = h.real, h.imag
    return np.interp(fg, f, re, right=0.0) + 1j * np.interp(fg, f, im, right=0.0)
```

5.1.4. Windowing and edge-rate limitation

Truncating the spectrum abruptly at $f_{\max}$ convolves the time response with a sinc function and produces ringing (Gibbs phenomenon). A taper reduces it [**harris1978window**, **oppenheim2010**]. The program offers three windows applied to the measured band $[0, f_{\max}]$ (only the descending half is used):

$$w_{\mathrm{Hann}}(f) = \tfrac{1}{2}\bigl(1 + \cos(\pi f / f_{\max})\bigr), \tag{5.5}$$

$$w_{\mathrm{Kaiser}}(f) = \text{descending half of a Kaiser window with } \beta = 6, \tag{5.6}$$

$$w_{\mathrm{rect}}(f) = 1. \tag{5.7}$$

In addition, a TDR step produced by a real instrument has a finite rise time. The program shapes the step with a Gaussian edge of 10–90 % rise time t_r. A Gaussian pulse with standard deviation σ_t has the spectrum $\exp(-2\pi^2 f^2 \sigma_t^2)$, and the 10–90 % rise time of its integral (a Gaussian step) is $t_r = 2\sigma_t\,\Phi^{-1}(0.9) \approx 2.563\,\sigma_t$, so the filter is

$$G(f) = \exp\!\left[-2\pi^2 f^2 \sigma_t^2\right], \qquad \sigma_t = \frac{t_r}{2.563}. \tag{5.8}$$

The product $H(f)\,w(f)\,G(f)$ is the spectrum that is transformed back. The spatial resolution of the result is of the order of half the rise time multiplied by the propagation velocity, and it is limited by $f_{\max}$ as well: a channel measured to $20\,\mathrm{GHz}$ cannot resolve features faster than about $17\,\mathrm{ps}$.

Source: `si_analyzer.py` — `time_response`

```python
def time_response(f: np.ndarray, h: np.ndarray, rise_s: float, window: str = "Hann",
                  n_freq: int = 8192, ext: float = 4.0) -> tuple[np.ndarray, np.ndarray, np.ndarray]:
    """Step␣and␣impulse␣response␣of␣a␣measured␣transfer/reflection␣function.

␣␣␣␣DC␣is␣extrapolated␣by␣holding␣the␣first␣point;␣data␣above␣the␣last␣point␣is␣zero
␣␣␣␣(the␣grid␣is␣extended␣`ext`x␣to␣interpolate␣in␣time).␣A␣Gaussian␣edge␣with␣the␣given
␣␣␣␣10-90␣%␣rise␣time␣shapes␣the␣step.␣Returns␣(t,␣step,␣impulse)␣over␣the␣unaliased␣half.
␣␣␣␣"""
    f = np.asarray(f, dtype=float)
    h = np.asarray(h, dtype=complex)
    fmax = float(f[-1])
    f_ext = ext * fmax
    fg = np.linspace(0.0, f_ext, n_freq)
    hg = _interp_response(fg, f, h)
    k = int(np.searchsorted(fg, fmax, side="right"))
    w = np.zeros(n_freq)
    if window.startswith("Hann"):
        w[:k] = 0.5 * (1.0 + np.cos(np.pi * fg[:k] / fmax))
    elif window.startswith("Kaiser"):
        w[:k] = np.kaiser(2 * k, 6.0)[k:]
    else:
        w[:k] = 1.0
    if rise_s > 0:
        sigma = rise_s / 2.563  # Gaussian edge: 10-90 % rise = 2.563 sigma
        w = w * np.exp(-2.0 * (np.pi * fg * sigma) ** 2)
    x = np.fft.irfft(hg * w, n=2 * (n_freq - 1))
    dt = 1.0 / (2.0 * f_ext)
    half = len(x) // 2
    t = np.arange(len(x)) * dt
    return t[:half], np.cumsum(x)[:half], (x / dt)[:half]
```

5.2. Impedance Profile and Distance Axis

In a TDR measurement a step of unit amplitude launched into a line of impedance Z_0 returns a reflection step $\rho(t)$. The impedance seen at the corresponding position follows from the reflection coefficient relation (3.4) applied to ρ,

$$Z(t) = Z_0 \, \frac{1 + \rho(t)}{1 - \rho(t)}. \tag{5.9}$$

To keep the expression finite the program limits ρ to the interval $[-0.98, 0.98]$. The TDR time is a round-trip time, so the distance to the feature is

$$d = \frac{v\,t}{2}, \qquad v = \frac{c}{\sqrt{\varepsilon_{r,\mathrm{eff}}}}, \tag{5.10}$$

with the effective permittivity entered by the user (default 4.0). Because the profile is computed from the reflection step it is *not* the true impedance of a lossy line after the first discontinuity: later features are attenuated and multiple reflections accumulate. It is an excellent *localization* tool, and its absolute values are most reliable for the first discontinuities [**johnson1993**].

The four panels of the tab show the TDR step (both ports), the impedance profile with a secondary distance axis, the TDT step (S21, S12) and the impulse response of S21. The visible window is chosen automatically as the larger of $6\bar{\tau}_g + 8t_r$ and $20/f_{\max}$, where $\bar{\tau}_g$ is the median group delay; the user can override it.

Source: `si_analyzer.py` — `impedance_from_step`

```python
def impedance_from_step(rho: np.ndarray, z0: float) -> np.ndarray:
    rho = np.clip(np.real(rho), -0.98, 0.98)
    return z0 * (1.0 + rho) / (1.0 - rho)
```

Source: `si_analyzer.py` — `AdvancedTabs._draw_tdr`

```
def _draw_tdr(self) -> None:
    if self.channel is None:
        return self._placeholder("tdr", "Load a Touchstone file first.")
    d, f = self.channel, self.channel.frequency_hz
    tr = max(self._fv(self.tr_var, 35.0), 0.0) * 1e-12
    er = max(self._fv(self.er_var, 4.0), 1.0)
    v = SPEED_OF_LIGHT / math.sqrt(er)
    z1, z2 = self._z0s()
    win = self.win_var.get()
    resp = {name: time_response(f, d.s[:, i, j], tr, win)
            for name, (i, j) in {"S11": (0, 0), "S22": (1, 1), "S21": (1, 0), "S12": (0, 1)}.
                items()}
    t = resp["S11"][0]
    tau = float(np.nanmedian(np.abs(group_delay(f, d.s[:, 1, 0]))))
    span = self._fv(self.span_var, 0.0) * 1e-9
    if span <= 0:
        span = max(6.0 * (tau if np.isfinite(tau) else 0.0) + 8.0 * tr, 20.0 / f[-1])
    tn = t * 1e9

    fig = self.fig_tdr
    fig.clf()
    ax1, ax2, ax3, ax4 = fig.subplots(2, 2).ravel()
    ax1.plot(tn, resp["S11"][1], color=ACCENT, label="Port 1 (S11)")
    ax1.plot(tn, resp["S22"][1], color=PURPLE, linestyle="--", label="Port 2 (S22)")
    ax1.set_title("TDR - reflection step response")
    ax1.set_ylabel("Reflection ")
    z_a, z_b = impedance_from_step(resp["S11"][1], z1), impedance_from_step(resp["S22"][1],
        z2)
    ax2.plot(tn, z_a, color=ACCENT, label="Port 1 profile")
    ax2.plot(tn, z_b, color=PURPLE, linestyle="--", label="Port 2 profile")
    ax2.axhline(z1, color=MUTED, linewidth=0.8, linestyle=":")
    ax2.set_title("Impedance profile Z(t)")
    ax2.set_ylabel("Impedance Ω()")
    to_mm = lambda x: x * 1e-9 * v / 2.0 * 1e3  # noqa: E731  (round-trip distance)
    top = ax2.secondary_xaxis("top", functions=(to_mm, lambda x: x / (1e-9 * v / 2.0 * 1e3)))
    top.set_xlabel(f"Distance (mm),  r,eff = {er:g}", color=MUTED)
    top.tick_params(colors=MUTED)
    ax3.plot(tn, resp["S21"][1], color=ACCENT2, label="S21 step")
    ax3.plot(tn, resp["S12"][1], color=GREEN, linestyle="--", label="S12 step")
    ax3.set_title("TDT - transmission step response")
    ax3.set_ylabel("Amplitude")
    ax4.plot(tn, resp["S21"][2] * 1e-9, color=ACCENT2, label="S21 impulse")
    ax4.set_title("Impulse response of S21")
    ax4.set_ylabel("h(t)  (1/ns)")
    for ax in (ax1, ax2, ax3, ax4):
        ax.set_xlim(0, span * 1e9)
        ax.set_xlabel("Time  (ns)")
        ax.grid(True)
        ax.legend(loc="best")
    self._fit_y(ax1, t, [resp["S11"][1], resp["S22"][1]], span)
    self._fit_y(ax2, t, [z_a, z_b], span)
    self._fit_y(ax3, t, [resp["S21"][1], resp["S12"][1]], span)
    self._fit_y(ax4, t, [resp["S21"][2] * 1e-9], span)
    self._finish("tdr")

    m = t <= span
    self._set_section("tdr", "Impedance profile  (TDR)", [
        f"Rise time {tr*1e12:g} ps,  window {win},  view {span*1e9:.3g} ns",
        f"Port 1:  Z  min {np.nanmin(z_a[m]):.1f} Ω,  max {np.nanmax(z_a[m]):.1f} Ω  (reference {
            z1:g} Ω)",
        f"Port 2:  Z  min {np.nanmin(z_b[m]):.1f} Ω,  max {np.nanmax(z_b[m]):.1f} Ω  (reference {
            z2:g} Ω)",
        self._disc_line("Port 1 discontinuities", t, z_a, z1, span, v),
        self._disc_line("Port 2 discontinuities", t, z_b, z2, span, v),
    ])
```

5.3. Locating Discontinuities

Reading an impedance profile by eye is subjective. The program therefore lists the most significant excursions. For the profile $Z(t)$ inside the viewed window, the deviation $|Z(t) - Z_0|$ is searched for peaks that exceed a threshold

$$\theta = \max\bigl(2\,\Omega,\; 0.03\,Z_0\bigr), \tag{5.11}$$

with a minimum prominence of $\theta/2$ so that ripple on a plateau is not reported as many features. The six strongest peaks are returned in time order and printed with their time, their distance (Equation (5.10)) and their impedance. A verification check inserts a $70\,\Omega$ section into a $50\,\Omega$ line and requires the detected position to be within $60\,\mathrm{ps}$ of the centre of the section.

Source: `si_analyzer.py` — `find_discontinuities`

```
def find_discontinuities(t: np.ndarray, z: np.ndarray, z0: float, span: float, max_n: int =
    6) -> list[tuple[float, float]]:
    """Strongest impedance excursions (time, Z) inside the viewed window, in time order."""
    from scipy.signal import find_peaks
    m = t <= span
    dev = np.abs(z[m] - z0)
    thr = max(2.0, 0.03 * z0)
    pk, _ = find_peaks(dev, height=thr, prominence=thr / 2.0)
    pk = sorted(pk, key=lambda i: -dev[i])[:max_n]
    return [(float(t[m][i]), float(z[m][i])) for i in sorted(pk)]
```

Source: `si_analyzer.py` — `UltimateTabs._disc_line`

```
@staticmethod
def _disc_line(label: str, t: np.ndarray, z: np.ndarray, z0: float, span: float, v: float) ->
     str:
    found = find_discontinuities(t, z, z0, span)
    if not found:
        return f"{label}: no excursion larger than {max(2.0, 0.03 * z0):.1f} Ω from {z0:g} Ω"
    items = "; ".join(f"{tt*1e9:.3g} ns ({tt*v/2*1e3:.3g} mm): {zz:.1f} Ω" for tt, zz in
        found)
    return f"{label}: {items}"
```

5.4. Extrapolation to DC and Resampling

Some analyses are sensitive to the missing DC point. The Conditioning tool can prepend it explicitly. The real part of each of the four S-parameters is fitted over the first $k = 6$ points as an even polynomial in frequency,

$$\mathrm{Re}\,S_{ij}(f) \approx a_0 + a_1 f^2, \qquad S_{ij}(0) = a_0, \tag{5.12}$$

the imaginary part at DC is zero, the matrix is symmetrized ($\mathbf{S} \leftarrow (\mathbf{S} + \mathbf{S}^{T})/2$ as a DC network must be reciprocal), and its singular values are limited to 0.9999 so that the extrapolated point is passive.

Source: `si_analyzer.py` — `extrapolate_dc`

```
def extrapolate_dc(f: np.ndarray, s: np.ndarray, k: int = 6) -> tuple[np.ndarray, np.ndarray
    ]:
    """Prepend a DC point: real part extrapolated in f^2 from the first k points, imaginary
        part = 0."""
    if f[0] <= 0:
        return f, s
    k = min(k, len(f))
    dc = np.zeros((2, 2), dtype=complex)
    for i in range(2):
        for j in range(2):
            dc[i, j] = np.polyfit(f[:k] ** 2, s[:k, i, j].real, 1)[1]
    dc = 0.5 * (dc + dc.T)
```

```
    u, sv, vh = np.linalg.svd(dc)
    dc = (u * np.minimum(sv, 0.9999)) @ vh
    return np.concatenate([[0.0], f]), np.concatenate([dc[None], s], axis=0)
```

Data from different sources often use non-uniform frequency steps. The resampling tool interpolates the real and imaginary parts separately with a cubic spline onto a uniform grid of the requested number of points, which is the form preferred by transforms and many field-solver imports:

Source: `si_analyzer.py` — `resample_uniform`

```
def resample_uniform(f: np.ndarray, s: np.ndarray, n: int) -> tuple[np.ndarray, np.ndarray]:
    from scipy.interpolate import interp1d
    fn = np.linspace(float(f[0]), float(f[-1]), int(n))
    kind = "cubic" if len(f) >= 4 else "linear"
    out = np.empty((len(fn), 2, 2), dtype=complex)
    for i in range(2):
        for j in range(2):
            out[:, i, j] = (interp1d(f, s[:, i, j].real, kind=kind)(fn) + 1j * interp1d(f, s
                [:, i, j].imag, kind=kind)(fn))
    return fn, out
```

5.5. Time-Domain Gating

Gating isolates an event in time. For example, keeping only the window around a connector removes the reflections of everything before and after it from the measurement, and removing a fixture reflection leaves the DUT. The procedure is: transform the response to the time domain (rectangular window, no rise-time filter), multiply by a gate, and transform back.

$$x_g(t) = x(t)\,g(t), \qquad H_g(f) = \mathcal{F}\{x_g\}. \tag{5.13}$$

The gate between the start t_1 and the stop t_2 has raised-cosine edges of width e to avoid ringing:

$$g(t) = r\!\left(\frac{t-(t_1-e)}{e}\right) r\!\left(\frac{(t_2+e)-t}{e}\right), \qquad r(u) = \tfrac{1}{2} - \tfrac{1}{2}\cos\!\big(\pi\ \mathrm{clip}(u,0,1)\big). \tag{5.14}$$

The *keep* mode uses $g(t)$ and the *remove* mode uses $1-g(t)$. Because the two are complementary, the responses of both modes add up to the original response, which the verification suite checks. The gated response can be inspected and then, optionally, applied to the active channel (with undo).

Source: `si_analyzer.py` — `gate_response`

```
def gate_response(f: np.ndarray, h: np.ndarray, t1: float, t2: float, edge: float, keep: bool
    = True,
                  n_freq: int = 8192, ext: float = 4.0) -> tuple[np.ndarray, np.ndarray, np.
                      ndarray, np.ndarray]:
    """Time-gate a response: keep (or remove) the window [t1, t2] with raised-cosine edges.

    Returns (gated response at f, t, impulse, gate) with t sorted from negative to positive
    time.
    """
    f = np.asarray(f, dtype=float)
    h = np.asarray(h, dtype=complex)
    fg = np.linspace(0.0, ext * float(f[-1]), n_freq)
    hg = _interp_response(fg, f, h)
    x = np.fft.irfft(hg, n=2 * (n_freq - 1))
    n = len(x)
    dt = 1.0 / (2.0 * ext * float(f[-1]))
    idx = np.arange(n)
    ts = np.where(idx <= n // 2, idx, idx - n) * dt
    e = max(edge, dt)

    def rc(u: np.ndarray) -> np.ndarray:
        return 0.5 - 0.5 * np.cos(np.pi * np.clip(u, 0.0, 1.0))

```

```
    g = rc((ts - (t1 - e)) / e) * rc(((t2 + e) - ts) / e)
    xg = x * g if keep else x * (1.0 - g)
    hh = np.fft.rfft(xg)
    out = np.interp(f, fg, hh.real) + 1j * np.interp(f, fg, hh.imag)
    order = np.argsort(ts)
    return out, ts[order], (x / dt)[order], g[order]
```

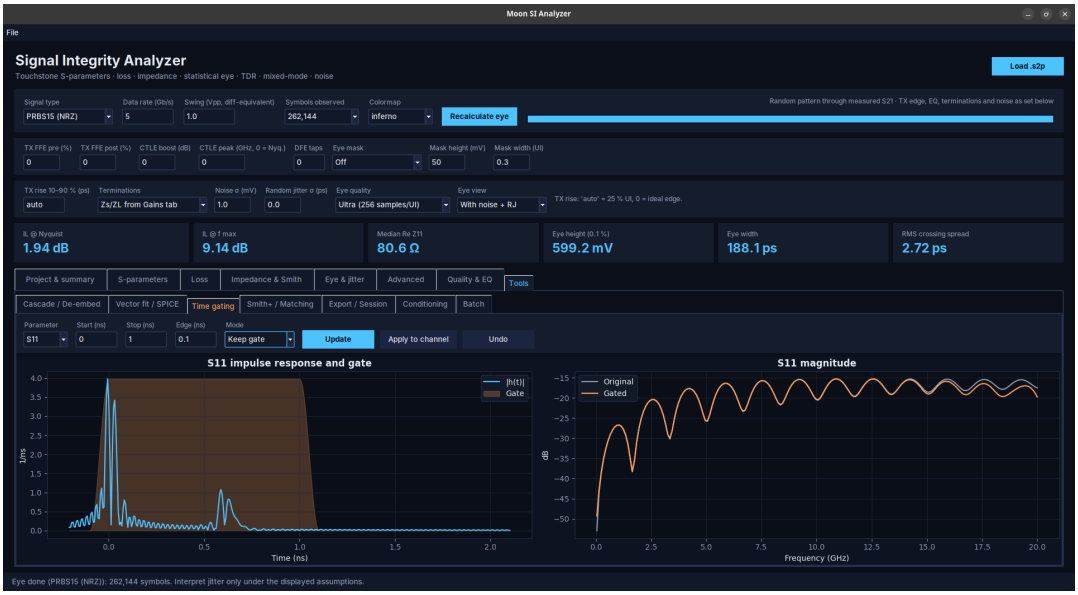


Figure 5.1: Time-domain gating of a reflection response.

5.6. Causality Check

A physically realizable channel is causal: nothing arrives before it is sent. Measured or simulated data that violate this (band-limited extrapolation, interpolation errors, wrongly resolved phase) make time-domain results meaningless [**triverio2007**]. The program provides a *heuristic* check. It computes the two-sided impulse response of S21 (Hann window), estimates the propagation delay as the median group delay $\bar{\tau}_g$, and reports the fraction of energy that arrives before $0.5\,\bar{\tau}_g$,

$$E_{pre} = \frac{\sum_{t<0.5\bar{\tau}_g} h^2(t)}{\sum_t h^2(t)} \times 100\,\%. \tag{5.15}$$

A value below $1\,\%$ is reported as “looks causal”. The test cannot prove causality and can be affected by truncation ringing; it is a screening tool, not a certificate (Chapter 12).

Source: `si_analyzer.py` — `impulse_full`

```
def impulse_full(f: np.ndarray, h: np.ndarray, n_freq: int = 8192, ext: float = 4.0) -> tuple
    [np.ndarray, np.ndarray]:
    """Two-sided impulse response (t from -T/2 to +T/2) of a measured response, Hann-windowed
        ."""
    f = np.asarray(f, dtype=float)
    h = np.asarray(h, dtype=complex)
    fmax = float(f[-1])
    fg = np.linspace(0.0, ext * fmax, n_freq)
    hg = _interp_response(fg, f, h)
    k = int(np.searchsorted(fg, fmax, side="right"))
    w = np.zeros(n_freq)
    w[:k] = 0.5 * (1.0 + np.cos(np.pi * fg[:k] / fmax))
    x = np.fft.irfft(hg * w, n=2 * (n_freq - 1))
    dt = 1.0 / (2.0 * ext * fmax)
    t = (np.arange(len(x)) - len(x) // 2) * dt
    return t, np.fft.fftshift(x) / dt
```

6

The Eye-Diagram Engine

The eye diagram is the most intuitive way to judge a serial link: if the received waveform, folded modulo the symbol period, leaves an open region around the sampling instant, a receiver can decide correctly. This chapter documents how the program synthesizes a data stream, filters it with the measured channel, optionally applies transmitter and receiver equalization, and converts the received waveform into a two-dimensional intensity picture and into statistical measurements.

6.1. Overview of the Algorithm

The engine implements the chain of Figure 6.1. For each block of symbols it (1) generates the symbols of the selected signal type, (2) builds the transmitted waveform with oversampling and the optional TX FFE, (3) multiplies the spectrum by the channel (S21 or the terminated response) combined with the TX edge and the RX CTLE, (4) optionally subtracts decision-feedback taps, (5) detects threshold crossings to align the eye and measure jitter, and (6) accumulates the folded waveform in two histograms. After the last block it derives the eye height and width and returns the density arrays and the pulse response.

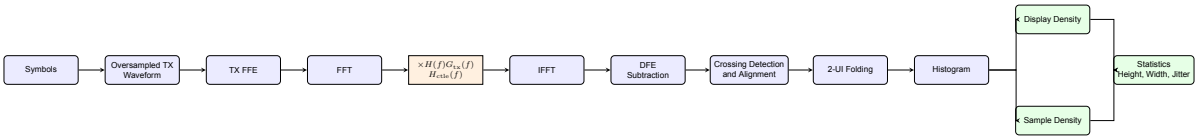


Figure 6.1: Processing chain of the eye engine for one block of symbols.

6.2. Signal Types

The program supports twelve signal types. Each is defined by the generator of its symbol levels in $[-1, 1]$ and by the number of information bits carried per symbol β_s. The unit interval follows from the data rate R_b:

$$T = \frac{\beta_s}{R_b}, \qquad f_{\mathrm{baud}} = \frac{1}{T}, \qquad f_{\mathrm{Nyq}} = \frac{1}{2T}. \tag{6.1}$$

For NRZ ($\beta_s = 1$) at $10\,\mathrm{Gb/s}$, $T = 100\,\mathrm{ps}$.

For an L-level PAM signal the levels are

$$\ell_k = -1 + \frac{2k}{L-1}, \qquad k = 0, \ldots, L-1, \qquad \beta_s = \log_2 L. \tag{6.2}$$

The specification dictionary contains, for each type, the kind, the bits per symbol and (for PAM) the number of levels.

Table 6.1: Signal types supported by the eye engine.

Signal	β_s	Definition of the symbol sequence
NRZ (PAM2)	1	Random levels ± 1
PAM3	$\log_2 3$	Random levels $\{-1, 0, 1\}$
PAM4	2	Random levels $\{-1, -\frac{1}{3}, \frac{1}{3}, 1\}$
PAM8	3	Eight equally spaced levels
RZ (50 % duty)	1	± 1 during the first half of the UI, zero in the second half
Manchester	0.5	Each bit is a pair of half-bit symbols $(b, -b)$
MLT-3	1	Cycle $0, +1, 0, -1$ advancing on every “1” bit, holding on “0”
Duobinary	1	$a_k = \frac{1}{2}(b_k + b_{k-1})$ with $b_k = \pm 1$, giving levels $\{-1, 0, 1\}$
Clock (1010…)	1	Alternating ± 1
PRBS7, PRBS9, PRBS15	1	Maximal-length LFSR sequences, NRZ

Source: `si_analyzer.py` — `_pam`

```
def _pam(levels: int) -> dict[str, Any]:
    lv = np.linspace(-1.0, 1.0, levels)
    return {"kind": "pam", "levels": levels, "bps": math.log2(levels), "thr": list(0.5 * (lv
        [:-1] + lv[1:]))}
```

Source: `si_analyzer.py` — `SIGNALS`

```
SIGNALS: dict[str, dict[str, Any]] = {
    "NRZ (PAM2)": _pam(2),
    "PAM3": _pam(3),
    "PAM4": _pam(4),
    "PAM8": _pam(8),
    "RZ (50 % duty)": {"kind": "rz", "bps": 1.0, "thr": [-0.5, 0.5]},
    "Manchester": {"kind": "manchester", "bps": 0.5, "thr": [0.0]},
    "MLT-3": {"kind": "mlt3", "bps": 1.0, "thr": [-0.5, 0.5]},
    "Duobinary": {"kind": "duobinary", "bps": 1.0, "thr": [-0.5, 0.5]},
    "Clock (1010…)": {"kind": "clock", "bps": 1.0, "thr": [0.0]},
    "PRBS7 (NRZ)": {"kind": "prbs", "order": 7, "bps": 1.0, "thr": [0.0]},
    "PRBS9 (NRZ)": {"kind": "prbs", "order": 9, "bps": 1.0, "thr": [0.0]},
    "PRBS15 (NRZ)": {"kind": "prbs", "order": 15, "bps": 1.0, "thr": [0.0]},
}
```

6.2.1. Pseudo-random binary sequences

A PRBS of order n is produced by a linear-feedback shift register with a primitive feedback polynomial, giving a sequence of period $2^n - 1$ in which every non-zero n-bit pattern occurs exactly once [**golomb1967**]. The program uses the polynomials $x^7 + x^6 + 1$, $x^9 + x^5 + 1$ and $x^{15} + x^{14} + 1$. The register is initialized with all ones, and with state s the new bit is

$$b_{new} = s_{t_1} \oplus s_{t_2}, \qquad s \leftarrow \left((s \ll 1) \,|\, b_{new}\right) \,\&\, (2^n - 1). \tag{6.3}$$

One full period is computed once and cached; every block starts at a random offset of the period, so the same pattern content is observed at different phases. Because the period of PRBS15 (32767 bits) is shorter than the default block length, each block contains the full pattern. The verification suite checks the period of PRBS7 and PRBS15 and the ones-zeros balance of PRBS7.

Source: `si_analyzer.py` — `_prbs_sequence`

```
@lru_cache(maxsize=None)
def _prbs_sequence(order: int) -> np.ndarray:
    """One full period of a maximal-length LFSR sequence (x^n + x^k + 1)."""
    t1, t2 = _PRBS_TAPS[order]
    length = (1 << order) - 1
```

```
    mask = length
    state = mask
    out = np.empty(length, dtype=np.int8)
    for i in range(length):
        new = ((state >> (t1 - 1)) ^ (state >> (t2 - 1))) & 1
        out[i] = new
        state = ((state << 1) | new) & mask
    return out
```

Source: `si_analyzer.py` — `_generate_symbols`

```
def _generate_symbols(spec: dict[str, Any], rng: np.random.Generator, n: int) -> np.ndarray:
    """Return n symbol levels in [-1, 1] for the selected signal type."""
    kind = spec["kind"]
    if kind == "pam":
        lv = np.linspace(-1.0, 1.0, spec["levels"])
        return lv[rng.integers(0, spec["levels"], size=n)]
    if kind == "rz":
        return 2.0 * rng.integers(0, 2, size=n) - 1.0
    if kind == "manchester":
        b = 2.0 * rng.integers(0, 2, size=n // 2) - 1.0
        return np.column_stack([b, -b]).ravel()
    if kind == "mlt3":
        bits = rng.integers(0, 2, size=n)
        return np.array([0.0, 1.0, 0.0, -1.0])[np.cumsum(bits) % 4]
    if kind == "duobinary":
        b = 2.0 * rng.integers(0, 2, size=n) - 1.0
        return 0.5 * (b + np.roll(b, 1))
    if kind == "clock":
        return np.where(np.arange(n) % 2 == 0, 1.0, -1.0)
    if kind == "prbs":
        seq = _prbs_sequence(spec["order"])
        start = int(rng.integers(len(seq)))
        return 2.0 * seq[(start + np.arange(n)) % len(seq)] - 1.0
    raise ValueError(f"Unknown signal kind: {kind}")
```

6.3. Filtering with the Measured Channel

Let a block contain N_b symbols, each represented by O samples (the oversampling factor). The sampling rate is $f_s = O/T$ and the block has $n = N_b O$ samples. The transmitted waveform is the amplitude-scaled symbol sequence held for one UI,

$$x[m] = A\, a_{\lfloor m/O \rfloor}, \qquad A = \frac{V_{pp}}{2}. \tag{6.4}$$

The received waveform is the circular convolution of x with the channel, computed in the frequency domain,

$$y = \mathcal{F}^{-1}\{\mathcal{F}\{x\} \cdot H_{tot}(f_k)\}, \qquad f_k = \frac{k\, f_s}{n}. \tag{6.5}$$

Circular convolution is exact for a periodic sequence; each block is therefore treated as one period of a periodic data stream, which is a standard technique for statistical eye generation. Edge effects at the block boundaries are immaterial because blocks are statistically independent and the channel memory is short compared with the block length.

6.3.1. Interpolation of the channel and band limitation

The channel $H(f)$ is interpolated onto the FFT grid with the DC treatment of Section 5.1. Beyond the highest measured frequency $f_{\max}$ the response is zero, preceded by a raised-cosine taper over the top $10\,\%$ of the measured band to avoid a hard edge:

$$t(f) = \begin{cases} 1, & f \le 0.9 f_{\max}, \\ \frac{1}{2}\Big[1 + \cos\Big(\pi \, \dfrac{f - 0.9 f_{\max}}{0.1 f_{\max}}\Big)\Big], & 0.9 f_{\max} < f < f_{\max}, \\ 0, & f \ge f_{\max}. \end{cases} \tag{6.6}$$

6.3.2. Finite transmitter edge rate

A real driver does not produce an infinitely fast edge. The engine multiplies the channel by the Gaussian edge filter of Equation (5.8). The default value, labelled `auto`, is $25\,\%$ of the UI (10–90 %) so that a result is obtained that is representative without user input; the user can enter an explicit value in picoseconds or 0 for an ideal edge. Two consequences must be understood. First, if the edge is specified as a fraction of the UI, the eye *looks identical at all data rates* on a UI-normalized axis when the channel is near ideal, because both the UI and the edge scale together. Second, the real transmitter edge is a property of the driver and not of the data rate; entering a fixed value in picoseconds shows the correct behaviour, in which a higher data rate leaves a smaller fraction of the UI for the edge and closes the eye from the sides. With an ideal edge, band limitation of the channel at $f_{\max}$ produces Gibbs ringing, which is an artefact of the truncated measurement and not of the channel itself.

6.4. Equalization

6.4.1. Transmitter feed-forward equalizer (FFE)

A three-tap FFE with a pre-cursor and a post-cursor tap, normalized so that the sum of absolute tap magnitudes is one (peak amplitude preserved), modifies the transmitted waveform:

$$y_{ffe}[m] = c_0\, x[m] + c_{-1}\, x[m + O] + c_{+1}\, x[m - O], \qquad c_{-1} = -p,\; c_{+1} = -q,\; c_0 = 1 - |p| - |q|, \tag{6.7}$$

where p and q are the pre- and post-tap weights (percent in the interface, limited to $45\,\%$ each). Shifting by O samples shifts by one symbol; the shift is circular, consistent with the periodic block. The effect is a high-pass emphasis of the transitions, de-emphasizing repeated symbols.

Source: `si_analyzer.py` — `apply_ffe`

```
def apply_ffe(wave: np.ndarray, os_: int, pre: float, post: float) -> np.ndarray:
    """3-tap␣TX␣FFE␣[-pre,␣1-|pre|-|post|,␣-post]␣applied␣on␣a␣symbol-rate␣waveform␣(circular
        )."""
    if pre == 0.0 and post == 0.0:
        return wave
    return (1.0 - abs(pre) - abs(post)) * wave - pre * np.roll(wave, -os_) - post * np.roll(
        wave, os_)
```

6.4.2. Receiver continuous-time linear equalizer (CTLE)

The CTLE is modelled by a transfer function with one zero and two poles,

$$H_{ctle}(f) = \frac{1 + jf/f_z}{(1 + jf/f_{p1})(1 + jf/f_{p2})} \Bigg/ \left| \frac{1 + jf_p/f_z}{(1 + jf_p/f_{p1})(1 + jf_p/f_{p2})} \right|, \tag{6.8}$$

normalized to unity gain at the peaking frequency f_p. With the boost B (in dB) and $G = 10^{B/20}$, the corner frequencies are chosen as

$$f_z = \frac{f_p}{\sqrt{G}}, \qquad f_{p1} = f_p \sqrt{G}, \qquad f_{p2} = 2.5\, f_{p1}. \tag{6.9}$$

The zero–pole pair provides a rise of about B dB between f_z and f_{p1} (attenuating low frequencies relative to the peak), and the second pole limits the high-frequency noise gain. By default the peak is at the Nyquist frequency of the signal. A boost of zero disables the CTLE.

Source: `si_analyzer.py` — `ctle_response`

```
def ctle_response(freq: np.ndarray, boost_db: float, peak_hz: float) -> np.ndarray:
    """One-zero / two-pole CTLE normalised to unity gain at its peak frequency (low
        frequencies are attenuated)."""
    if boost_db <= 0 or peak_hz <= 0:
        return np.ones(len(freq), dtype=complex)
    g = 10.0 ** (boost_db / 20.0)
    fz, fp1 = peak_hz / math.sqrt(g), peak_hz * math.sqrt(g)
    fp2 = 2.5 * fp1

    def raw(x: Any) -> Any:
        return (1.0 + 1j * x / fz) / ((1.0 + 1j * x / fp1) * (1.0 + 1j * x / fp2))

    return raw(freq) / np.abs(raw(np.asarray(peak_hz, dtype=float)))
```

6.4.3. Pulse response and cursors

The linear response to a single isolated symbol (a "pulse" of amplitude A and one UI duration) through the FFE, the channel and the CTLE is the *pulse response* $p(t)$ [**sanders2004**]. The sample of largest magnitude defines the main cursor at t_0; the samples at integer UI distances from it are the cursors

$$h_k = p(t_0 + kT), \qquad k \in \mathbb{Z}. \tag{6.10}$$

By superposition, the received sample at the decision instant for a symbol sequence a_n is $y[n] = \sum_k h_k\, a_{n-k}$, so that the cursors with $k \neq 0$ are the inter-symbol interference. The engine computes the pulse response with the same operations as the data path (including the RZ duty and the FFE), rolls it by eight UI so that precursors appear at positive indices, and finds its peak. The pulse response is plotted in the Pulse/Bathtub tab and used by the DFE and by the COM-like budget.

Source: `si_analyzer.py` — `_pulse_response`

```
def _pulse_response(h_i: np.ndarray, n: int, os_: int, amp: float, kind: str, ffe: tuple[
    float, float],
                    pre_ui: int = 8) -> tuple[np.ndarray, int]:
    """Single-symbol response of TX FFE + channel + CTLE. Returns (response rolled by pre_ui
        UI, peak index)."""
    pulse = np.zeros(n)
    pulse[:os_] = amp
    if kind == "rz":
        pulse[os_ // 2:os_] = 0.0
    pulse = apply_ffe(pulse, os_, *ffe)
    p = np.roll(np.fft.irfft(np.fft.rfft(pulse) * h_i, n=n), pre_ui * os_)
    return p, int(np.argmax(np.abs(p)))
```

6.4.4. Decision-feedback equalizer (DFE)

A DFE removes the post-cursor interference caused by previously decided symbols [**proakis2008**]. With ideal decisions, the feedback signal during the UI of symbol n is $\sum_{k=1}^{N} h_k\, a_{n-k}$, and the equalized waveform is

$$\tilde{y}(t) = y(t) - \sum_{k=1}^{N} h_k\, a_{n-k}, \qquad t \in \left[t_0 + nT - \tfrac{T}{2},\; t_0 + nT + \tfrac{T}{2}\right). \tag{6.11}$$

The taps are the first N post-cursors of the equalized pulse response ($N \leq 24$). The correction waveform is built at symbol rate, held for one UI, and aligned to the decision UI by a circular shift of $d - O/2$ samples, where d is the position of the main cursor inside the block. Because decisions are assumed correct, the model does not include error propagation.

6.5. Terminated Channel

By default the eye uses S_{21}, which assumes the channel is driven from and terminated in the reference impedance. With real source and load impedances reflections occur at both ends and the signal

bounces. Let Γ_s and Γ_L be the source and load reflection coefficients. With the source wave b_s, the network equations $b_1 = S_{11}a_1 + S_{12}a_2$, $b_2 = S_{21}a_1 + S_{22}a_2$, $a_1 = b_s + \Gamma_s b_1$ and $a_2 = \Gamma_L b_2$ lead to

$$b_2 = \frac{S_{21}\, b_s}{(1 - S_{22}\Gamma_L)(1 - \Gamma_s\Gamma_{in})}, \tag{6.12}$$

and the load voltage is $V_L = \sqrt{Z_0}\,(1+\Gamma_L)\,b_2$. Normalizing so that matched terminations ($\Gamma_s = \Gamma_L = 0$) return exactly S_{21}, and accounting for the source-wave scaling $b_s \propto (1 - \Gamma_s)$, the effective transfer function is

$$H_{term}(f) = \frac{S_{21}\,(1-\Gamma_s)(1+\Gamma_L)}{(1 - S_{22}\Gamma_L)\,(1 - \Gamma_s\Gamma_{in})}. \tag{6.13}$$

The result of this expression was compared with a direct circuit solution (source impedance, two-port, load impedance) at one frequency and agrees to numerical precision (Chapter 11). The user selects the terminations in the second control card; the source and load impedances are the ones entered in the Gains tab.

Source: `si_analyzer.py` — `terminated_response`

```
def terminated_response(s: np.ndarray, gs: complex, gl: complex) -> np.ndarray:
    """Voltage␣transfer␣to␣the␣load␣with␣arbitrary␣source/load␣reflection␣coefficients.

␣␣␣␣Normalised␣so␣that␣gs␣=␣gl␣=␣0␣returns␣exactly␣S21␣(matched␣source␣and␣load).␣Source␣and␣load
␣␣␣␣reflections␣(multiple␣bounces)␣are␣included,␣which␣is␣what␣makes␣a␣mismatched␣eye␣ring.
␣␣␣␣"""
    s11, s12, s21, s22 = s[:, 0, 0], s[:, 0, 1], s[:, 1, 0], s[:, 1, 1]
    with np.errstate(all="ignore"):
        gin = s11 + s12 * s21 * gl / (1.0 - s22 * gl)
        return s21 * (1.0 - gs) * (1.0 + gl) / ((1.0 - s22 * gl) * (1.0 - gs * gin))
```

6.6. Threshold Detection, Alignment and Jitter

6.6.1. Decision thresholds from the data

Multi-level eyes need one threshold between each pair of adjacent levels. Rather than assuming ideal positions (which would be wrong for a lossy channel whose levels are attenuated and distorted), the engine measures them: in the first block it samples the received waveform at the main-cursor instant of every symbol, computes the mean sampled voltage for each transmitted level ℓ_i, and places the thresholds at the midpoints,

$$T_i = \tfrac{1}{2}\big(\bar{v}_i + \bar{v}_{i+1}\big), \qquad \bar{v}_i = \mathrm{mean}\{y[n] : a_n = \ell_i\}. \tag{6.14}$$

This works for every signal type, including three-level and RZ signals. The *centre* threshold (the middle one for an odd number, otherwise the upper of the two middle ones) is used for the crossing statistics.

6.6.2. Zero-crossing detection

The instants at which the waveform crosses the threshold are found by sign changes between consecutive samples and refined by linear interpolation. For samples y_i and y_{i+1} (already shifted by the threshold) with a sign change, the crossing position in samples is

$$\xi = i + \frac{-y_i}{y_{i+1} - y_i}. \tag{6.15}$$

The position modulo the UI, $\phi = \xi \bmod O$, measures where in the UI the transition occurs.

6.6.3. Alignment of the eye

Crossing phases are angular quantities: ϕ near 0 and near O are the same position. The mean phase is therefore computed as a circular mean,

$$\bar{\phi} = \frac{O}{2\pi}\left[\operatorname{atan2}\left(\overline{\sin\theta}, \overline{\cos\theta}\right) \bmod 2\pi\right], \qquad \theta = \frac{2\pi\phi}{O}. \tag{6.16}$$

The waveform is rolled by the rounded value of $\bar{\phi}$ so that crossings fall at 0, 1 and 2 UI and the eye centre at 0.5 and 1.5 UI. The alignment is determined from the first block and reused for the others.

6.6.4. Jitter metrics

The deviation of each crossing from the mean phase, wrapped to $\pm O/2$, gives the distribution of the crossing time. The program reports its standard deviation (RMS crossing spread) and the span between the $0.1\,\%$ and $99.9\,\%$ percentiles (peak-to-peak spread Δ_{pp}), and defines the eye width as the UI minus the peak-to-peak spread,

$$W_{eye} = T - \Delta_{pp}. \tag{6.17}$$

This is deterministic, pattern-dependent timing spread (data-dependent jitter). Random jitter, clock recovery and receiver aperture are not included in these numbers (see Section 7.1 for the statistical treatment of random jitter).

Source: `si_analyzer.py` — `_zero_crossings`

```python
def _zero_crossings(rx: np.ndarray, oversample: int) -> np.ndarray:
    """Return crossing positions (in samples, modulo one UI) of the zero threshold."""
    y0, y1 = rx[:-1], rx[1:]
    idx = np.flatnonzero(((y0 < 0.0) & (y1 >= 0.0)) | ((y0 > 0.0) & (y1 <= 0.0)))
    if len(idx) == 0:
        return np.empty(0)
    denom = rx[idx + 1] - rx[idx]
    with np.errstate(divide="ignore", invalid="ignore"):
        frac = np.where(np.abs(denom) > 1e-15, -rx[idx] / denom, 0.0)
    return np.mod(idx + frac, oversample)
```

6.7. Folding and the Two Density Arrays

6.7.1. Folding

After alignment, the received block is arranged as a matrix with one row per symbol and O columns, and each row is extended with the following row to cover *two* UI. The folded waveform of every symbol is then a trace over $2O$ time columns. The voltage range is divided into $B = 400$ bins covering $\pm v_{\max}$ with $v_{\max} = 1.1 \max|y|$ (taken from the first block, with a lower limit of $5\,\%$ of the swing).

6.7.2. Sample histogram

The first histogram counts how many samples fall in each (column, bin) cell. This is the *statistical* density used for all measurements (eye height, bathtub, mask hits). It is an integer array, free of drawing artefacts.

6.7.3. Display density with trace interpolation

When the edges are steeper than the sampling grid, consecutive samples of a transition lie far apart in voltage, and a histogram of samples alone shows isolated dots instead of continuous curves. An oscilloscope in persistence mode draws the line between successive points. The program does the same in a second, display-only histogram: for each column j and each trace, with bin indices b_j and b_{j+1} of two consecutive samples, the interval $[\min, \max]$ of the two bins is filled with a weight inversely proportional to its length,

$$w = \frac{1}{\max(b_j, b_{j+1}) - \min(b_j, b_{j+1}) + 1}, \tag{6.18}$$

so that every trace column contributes a total weight of one irrespective of the steepness. Adding a weight to every bin of an interval is implemented efficiently with a difference array: $+w$ at the lower bin and $-w$ one bin above the upper bin, followed by a cumulative sum along the voltage axis. This costs the same order of operations as the plain histogram.

6.7.4. Constant memory

The number of symbols per block is $N_b = \max\bigl(2048,\, 2^{20}/O\bigr)$ (limited by the requested total), so that $n = N_b O \approx 2^{20}$ samples at any oversampling factor, and the histogram has fixed size $2O \times B$. The total number of observed symbols $N_{tot} = N_{blk} N_b$ can then be as large as desired without growing memory; the time grows linearly with N_{blk}. The quality options (64, 128 or 256 samples per UI) trade resolution against time.

6.8. Statistical Eye Height

For every sub-eye (one per threshold T_i) and every time column, the cluster above the threshold and the cluster below it are separated by the neighbouring thresholds. The eye opening is the distance between the lower $0.1\,\%$ tail of the upper cluster and the upper $0.1\,\%$ tail of the lower cluster,

$$H_i(t) = q^{\uparrow}_{0.001}(t) - q^{\downarrow}_{0.999}(t). \tag{6.19}$$

The eye height of the signal is $\min_i H_i$ at the column with the largest value inside the first UI (the best sampling phase); for multi-level signals the opening of every sub-eye is also reported. The tail probability of $0.1\,\%$ is a robustness choice: it excludes isolated outliers but remains deterministic. For the bit-error-ratio view the full distribution and noise are used instead (Chapter 7).

Source: `si_analyzer.py` — `_eye_openings`

```
def _eye_openings(dens: np.ndarray, centers: np.ndarray, thresholds: np.ndarray) -> np.
    ndarray:
    """Per-sub-eye, per-column opening (0.1 % tails): array of shape (n_eyes, columns)."""
    edges = np.concatenate([[-np.inf], np.sort(thresholds), [np.inf]])
    rows = []
    for k in range(1, len(edges) - 1):
        lo = (centers > edges[k - 1]) & (centers <= edges[k])
        up = (centers > edges[k]) & (centers <= edges[k + 1])
        if not lo.any() or not up.any():
            continue
        d_lo, d_up = dens[:, lo], dens[:, up]
        cum_up = np.cumsum(d_up, axis=1) / np.maximum(d_up.sum(axis=1, keepdims=True), 1.0)
        cum_lo = np.cumsum(d_lo[:, ::-1], axis=1) / np.maximum(d_lo.sum(axis=1, keepdims=True
            ), 1.0)
        up_edge = centers[up][(cum_up >= 1e-3).argmax(axis=1)]
        lo_edge = centers[lo][::-1][(cum_lo >= 1e-3).argmax(axis=1)]
        rows.append(up_edge - lo_edge)
    return np.array(rows)
```

6.9. The Complete Engine

The listing below is the complete function. Its parameters are the channel frequency axis and response, the data rate, the swing, the total number of symbols, the oversampling factor and optional equalization parameters; the result is a dictionary with the density arrays, thresholds, eye measurements and pulse response.

Source: `si_analyzer.py` — `make_eye_and_jitter`

```
def make_eye_and_jitter(
    frequency_hz: np.ndarray,
    s21: np.ndarray,
    bit_rate_gbps: float,
    swing_vpp: float,
    total_bits: int = 262_144,
    oversample: int = 64,
```

```
    block_bits: int | None = None,
    v_bins: int = 400,
    seed: int = 17,
    signal: str = DEFAULT_SIGNAL,
    progress: Callable[[float], None] | None = None,
    ffe_pre: float = 0.0,
    ffe_post: float = 0.0,
    ctle_boost_db: float = 0.0,
    ctle_peak_ghz: float = 0.0,
    dfe_taps: int = 0,
    tx_rise_ps: float | None = None,
) -> dict[str, Any]:
    """Filter␣random␣symbol␣blocks␣with␣measured␣S21␣and␣accumulate␣a␣2-UI␣eye␣density.

␣␣␣␣Optional␣equalisation:␣TX␣FFE␣(pre/post␣fractions),␣RX␣CTLE␣(boost␣dB␣at␣peak␣frequency)␣and␣an
␣␣␣␣ideal-decision␣DFE␣(post-cursor␣taps␣taken␣from␣the␣equalised␣pulse␣response).
␣␣␣␣Each␣block␣is␣an␣independent␣random␣sequence␣(periodic␣within␣the␣block),␣so␣memory␣stays␣constant
␣␣␣␣while␣the␣number␣of␣observed␣symbols␣scales␣to␣millions.␣Below␣the␣first␣S21␣frequency␣the␣first␣value
␣␣␣␣is␣held;␣above␣the␣last␣point␣S21␣is␣zero␣with␣a␣raised-cosine␣taper␣over␣the␣top␣10␣%.
␣␣␣␣No␣noise,␣TX␣rise-time,␣RX␣model,␣clock␣recovery,␣or␣BER␣extrapolation␣is␣applied␣here.
␣␣␣␣"""
    if bit_rate_gbps <= 0 or swing_vpp <= 0:
        raise ValueError("Data␣rate␣and␣signal␣swing␣must␣both␣be␣greater␣than␣zero.")
    if total_bits < 1024:
        raise ValueError("Use␣at␣least␣1,024␣symbols␣for␣a␣meaningful␣eye␣view.")
    if signal not in SIGNALS:
        raise ValueError(f"Unknown␣signal␣type:␣{signal}")
    if abs(ffe_pre) + abs(ffe_post) >= 1.0:
        raise ValueError("FFE␣pre␣+␣post␣taps␣must␣stay␣below␣100␣%.")
    spec = SIGNALS[signal]

    f = np.asarray(frequency_hz, dtype=float)
    h = np.asarray(s21, dtype=complex)
    block_bits = int(min(block_bits or max(2048, 1_048_576 // oversample), total_bits))  # constant memory for any oversampling
    block_bits += block_bits % 2  # even, needed for Manchester
    n_blocks = max(1, int(math.ceil(total_bits / block_bits)))
    actual_bits = n_blocks * block_bits

    baud_gbd = bit_rate_gbps / spec["bps"]
    ui_s = 1.0 / (baud_gbd * 1e9)
    ui_ps = ui_s * 1e12
    fs = oversample / ui_s
    n = block_bits * oversample
    two_ui = 2 * oversample

    # Channel response on the rFFT grid.
    rfreq = np.fft.rfftfreq(n, d=1.0 / fs)
    measured_max = float(f[-1])
    h_i = _interp_response(rfreq, f, h)
    taper_start = 0.90 * measured_max
    taper = np.ones_like(rfreq)
    edge = (rfreq > taper_start) & (rfreq < measured_max)
    taper[edge] = 0.5 * (1.0 + np.cos(np.pi * (rfreq[edge] - taper_start) / (measured_max - taper_start)))
    taper[rfreq >= measured_max] = 0.0
    h_i = h_i * taper

    ctle_peak_hz = ctle_peak_ghz * 1e9 if ctle_peak_ghz > 0 else 0.5 / ui_s
    h_i = h_i * ctle_response(rfreq, ctle_boost_db, ctle_peak_hz)
    # Finite TX edge (no real driver has an infinite-bandwidth edge): Gaussian, 10-90 % rise time.
    # None = automatic 25 % of the UI; 0 = ideal (infinitely fast) edge.
    tx_rise_eff = 0.25 * ui_ps if tx_rise_ps is None else max(float(tx_rise_ps), 0.0)
    if tx_rise_eff > 0:
        h_i = h_i * np.exp(-2.0 * (np.pi * rfreq * tx_rise_eff * 1e-12 / 2.563) ** 2)

    amp = swing_vpp / 2.0
```

```
    ffe = (float(ffe_pre), float(ffe_post))
    pre_ui = 8
    p_c, d_pk_c = _pulse_response(h_i, n, oversample, amp, spec["kind"], ffe, pre_ui)
    d_pk = d_pk_c - pre_ui * oversample  # position of the main cursor inside one block

    def cursor(k: int) -> float:
        return float(p_c[(d_pk_c + k * oversample) % n])

    n_dfe = int(max(0, dfe_taps))
    dfe_v = np.array([cursor(k) for k in range(1, n_dfe + 1)])

    rng = np.random.default_rng(seed)
    hist = np.zeros(two_ui * v_bins, dtype=np.int64)
    hist_vis = np.zeros(two_ui * (v_bins + 1))  # display-only density: the trace drawn between samples
    t_off = (np.arange(two_ui, dtype=np.int64) * v_bins)[None, :]
    deviations: list[np.ndarray] = []
    center = 0.0
    shift = 0
    vmax = 1.0
    thresholds_v = np.array([0.0])
    cross_v = 0.0

    for b in range(n_blocks):
        sym = _generate_symbols(spec, rng, block_bits)
        tx = np.repeat(sym * amp, oversample)
        if spec["kind"] == "rz":
            tx.reshape(block_bits, oversample)[:, oversample // 2:] = 0.0
        tx = apply_ffe(tx, oversample, *ffe)
        rx = np.fft.irfft(np.fft.rfft(tx) * h_i, n=n)
        if n_dfe:
            corr_sym = np.zeros(block_bits)
            for k, c in enumerate(dfe_v, start=1):
                corr_sym += c * np.roll(sym, k)
            rx = rx - np.roll(np.repeat(corr_sym, oversample), d_pk - oversample // 2)

        if b == 0:
            # Decision thresholds: midpoints between the mean sampled voltage of each transmitted level.
            sampled = rx[(np.arange(block_bits) * oversample + d_pk) % n]
            lv = np.unique(np.round(sym, 6))
            means = np.sort([float(sampled[np.isclose(sym, v)].mean()) for v in lv])
            if len(means) >= 2:
                thresholds_v = 0.5 * (means[:-1] + means[1:])
            cross_v = float(thresholds_v[len(thresholds_v) // 2])
        phases = _zero_crossings(rx - cross_v, oversample)
        if b == 0:
            if len(phases):
                ang = phases * (2.0 * np.pi / oversample)
                mean_ang = math.atan2(float(np.mean(np.sin(ang))), float(np.mean(np.cos(ang))))
                center = (mean_ang % (2.0 * np.pi)) * oversample / (2.0 * np.pi)
            shift = int(round(center)) % oversample
            vmax = max(1.1 * float(np.max(np.abs(rx))), 0.05 * swing_vpp)
        if len(phases):
            dev = (phases - center + oversample / 2.0) % oversample - oversample / 2.0
            deviations.append(dev.astype(np.float32))

        # Roll so threshold crossings land at 0 / 1 / 2 UI, eye centre at 0.5 / 1.5 UI.
        r2 = np.roll(rx, -shift).reshape(block_bits, oversample)
        win = np.concatenate([r2, np.roll(r2, -1, axis=0), np.roll(r2, -2, axis=0)[:, :1]], axis=1)
        vi_all = np.clip(((win + vmax) / (2.0 * vmax) * v_bins).astype(np.int64), 0, v_bins - 1)
        vi = vi_all[:, :two_ui]
        hist += np.bincount((vi + t_off).ravel(), minlength=two_ui * v_bins)
        # Like a persistence scope, connect consecutive samples so steep edges are drawn as continuous
        # curves instead of isolated dots (each trace column keeps a total weight of one).
        lo_b = np.minimum(vi_all[:, :-1], vi_all[:, 1:])
        hi_b = np.maximum(vi_all[:, :-1], vi_all[:, 1:])
```

```
        wgt = (1.0 / (hi_b - lo_b + 1)).ravel()
        base = np.arange(two_ui, dtype=np.int64)[None, :] * (v_bins + 1)
        size = two_ui * (v_bins + 1)
        hist_vis += (np.bincount((base + lo_b).ravel(), weights=wgt, minlength=size)
                     - np.bincount((base + hi_b + 1).ravel(), weights=wgt, minlength=size))

        if progress is not None:
            progress((b + 1) / n_blocks)

    dens = hist.reshape(two_ui, v_bins).astype(float)
    dens_vis = np.maximum(np.cumsum(hist_vis.reshape(two_ui, v_bins + 1), axis=1)[:, :v_bins
        ], 0.0)
    v_edges = np.linspace(-vmax, vmax, v_bins + 1)
    centers = 0.5 * (v_edges[:-1] + v_edges[1:])

    openings = _eye_openings(dens, centers, thresholds_v)
    height = openings.min(axis=0) if len(openings) else np.zeros(two_ui)
    best_col = int(np.argmax(height[:oversample]))

    if deviations:
        dev_all = np.concatenate(deviations)
        rms_ps = float(np.std(dev_all) / oversample * ui_ps)
        q01, q99 = np.percentile(dev_all, [0.1, 99.9])
        pp_ps = float((q99 - q01) / oversample * ui_ps)
        crossings = int(len(dev_all))
    else:
        rms_ps = pp_ps = float("nan")
        crossings = 0

    ks = np.arange(-8, 25)
    fine = np.arange(-8 * oversample, 25 * oversample)
    eq_text = ((f"TX rise {tx_rise_eff:.1f} ps{' (auto, 25 % UI)' if tx_rise_ps is None else
        ''}, " if tx_rise_eff > 0 else "TX edge ideal, ")
               + f"TX FFE pre {ffe_pre*100:.0f} % / post {ffe_post*100:.0f} %, "
               + (f"CTLE {ctle_boost_db:g} dB @ {ctle_peak_hz/1e9:.3g} GHz, " if
                   ctle_boost_db > 0 else "CTLE off, ")
               + (f"DFE {n_dfe} tap(s)" if n_dfe else "DFE off"))
    return {
        "density": dens,
        "density_vis": dens_vis,
        "v_min": -vmax,
        "v_max": vmax,
        "ui_ps": ui_ps,
        "oversample": oversample,
        "bits": actual_bits,
        "sample_phase_ui": best_col / oversample,
        "eye_height_v": float(max(0.0, height[best_col])),
        "eye_height_raw_v": float(height[best_col]),
        "eye_width_ps": float(max(0.0, ui_ps - pp_ps)) if np.isfinite(pp_ps) else float("nan"
            ),
        "jitter_rms_ps": rms_ps,
        "jitter_pp_ps": pp_ps,
        "crossing_count": crossings,
        "bit_rate_gbps": bit_rate_gbps,
        "baud_gbd": baud_gbd,
        "swing_vpp": swing_vpp,
        "signal": signal,
        "thresholds_v": thresholds_v,
        "n_eyes": len(thresholds_v),
        "eye_heights_v": openings[:, best_col] if len(openings) else np.zeros(1),
        "eq_text": eq_text,
        "dfe_n": n_dfe,
        "dfe_taps_v": dfe_v,
        "pulse": {"x_ui": fine / oversample, "y": p_c[(d_pk_c + fine) % n], "k": ks,
                  "cursors": np.array([cursor(int(k)) for k in ks])},
    }
```

6.10. Display of the Eye

The graphical eye is drawn from the display density. To make faint features visible next to a very bright rail, the intensity is compressed with a power law, $I' = (I/I_{\max})^{0.4}$ (Matplotlib's `PowerNorm`), after Gaussian smoothing with $\sigma = (0.9\,O/64,\ 1.2)$ bins in time and voltage. In the *With noise + RJ* view, the display density is first convolved along the voltage axis with a Gaussian of the user's noise σ_n and along the time axis (circularly) with a Gaussian of the random jitter σ_j; this is the statistical picture of Chapter 7, and it does not alter any reported number. The best sampling phase and decision thresholds are overlaid, a secondary axis shows time in picoseconds, and the eye mask (if enabled) is drawn on top.

Source: `si_analyzer.py` — `SIAnalyzerApp._redraw_eye`

```
def _redraw_eye(self) -> None:
    r = self.eye_result
    if r is None:
        return
    fig = self.fig_eye
    fig.clf()
    ax = fig.add_subplot(111)
    self.ax_eye = ax

    dens = np.array(r.get("density_vis", r["density"]), dtype=float)
    os_ = r["oversample"]
    noisy = self.eye_view_var.get().startswith("With")
    if noisy:  # statistical view: convolve with the voltage-noise and random-jitter
        Gaussians
        dv = (r["v_max"] - r["v_min"]) / dens.shape[1]
        sv = max(self._fv(self.noise_var, 0.0), 0.0) / 1e3 / dv
        sj = max(self._fv(self.rj_var, 0.0), 0.0) / r["ui_ps"] * os_
        if sv > 0.05:
            dens = gaussian_filter1d(dens, sv, axis=1, mode="nearest")
        if sj > 0.05:
            dens = gaussian_filter1d(dens, sj, axis=0, mode="wrap")
    dens = gaussian_filter(dens, sigma=(0.9 * os_ / 64.0, 1.2), mode="nearest")
    norm = PowerNorm(gamma=0.4, vmin=0.0, vmax=float(dens.max()) or 1.0)
    im = ax.imshow(
        dens.T, origin="lower", aspect="auto", cmap=self.cmap_var.get(), norm=norm,
            interpolation="bilinear",
        extent=(0.0, 2.0, r["v_min"], r["v_max"]),
    )
    cbar = fig.colorbar(im, ax=ax, pad=0.02, fraction=0.04)
    cbar.set_label("Hit␣density␣(relative)")
    cbar.outline.set_edgecolor(BORDER)
    cbar.set_ticks([])

    for i, t in enumerate(r["thresholds_v"]):
        ax.axhline(float(t), color=RED, linewidth=0.9, linestyle="--", alpha=0.8,
                   label="Decision␣threshold(s)" if i == 0 else None)
    for k in (0, 1):
        ax.axvline(r["sample_phase_ui"] + k, color="#ffffff", linewidth=0.8, linestyle=":",
            alpha=0.7,
                   label="Best␣sample␣phase" if k == 0 else None)
    ax.set_xlim(0, 2)
    ax.set_ylim(r["v_min"], r["v_max"])
    ax.set_title(f"{r['signal']}␣eye␣density␣from␣measured␣S21␣-␣{r['bits'],}␣symbols" + ("␣
        (noise␣+␣RJ␣applied)" if noisy else ""))
    ax.set_xlabel("Time␣(UI)")
    ax.set_ylabel("Received␣voltage␣(V)")
    ui_ps = r["ui_ps"]
    top = ax.secondary_xaxis("top", functions=(lambda x: x * ui_ps, lambda x: x / ui_ps))
    top.set_xlabel("Time␣(ps)", color=MUTED)
    top.tick_params(colors=MUTED)
    self._draw_mask_overlay(ax, r)
    ax.legend(loc="upper␣right")
    ax.text(
        0.015, 0.03,
        f"Height␣{r['eye_height_raw_v']*1e3:.1f}␣mV␣␣␣Width␣{r['eye_width_ps']:.1f}␣ps␣␣␣"
        f"RMS␣spread␣{r['jitter_rms_ps']:.2f}␣ps",
```

```
        transform=ax.transAxes, color=FG, fontsize=9,
        bbox=dict(boxstyle="round,pad=0.4", facecolor=CARD, edgecolor=BORDER, alpha=0.9),
    )
    self.canvas_eye.draw_idle()
```

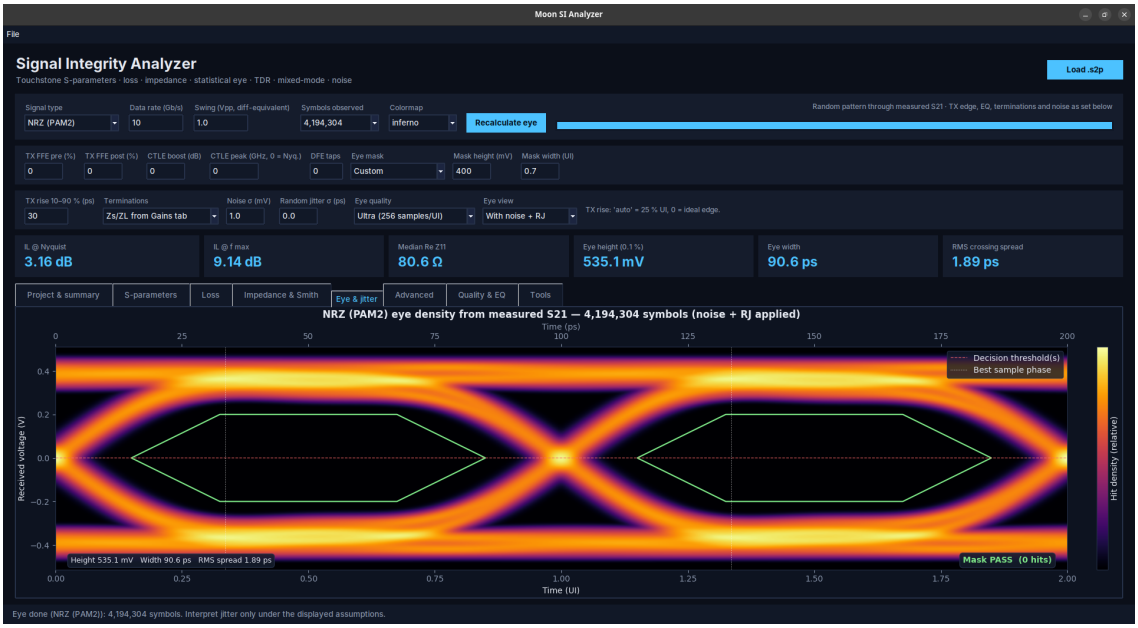


Figure 6.2: Eye diagram of an NRZ signal through a measured channel.



Figure 6.3: PAM4 eye with three sub-eyes and their thresholds.

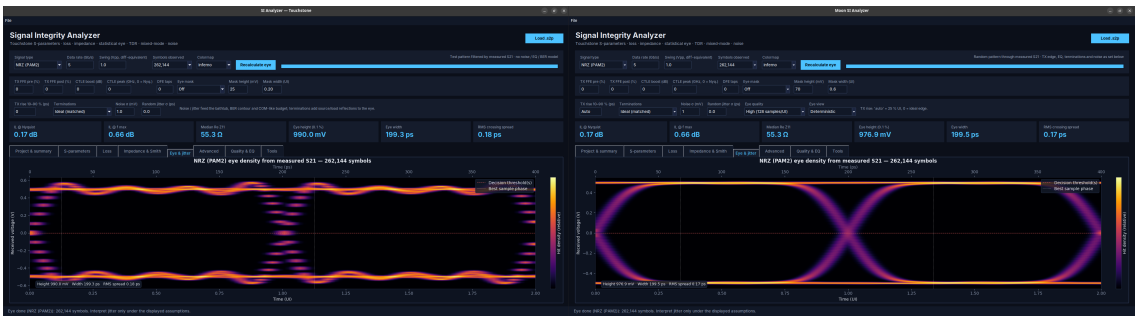


Figure 6.4: Effect of the trace-interpolated display density on steep edges.

7

Statistical Assessment of Channel Quality

The eye of Chapter 6 is deterministic: it contains inter-symbol interference but no noise. Real receivers, however, fail because of rare events in the presence of noise and random jitter, at bit-error ratios of 10^{-12} or lower, far beyond what any simulation of random data can observe directly. This chapter documents the statistical tools that extend the eye to those probabilities, the mask check, and the channel-quality figures (limit lines, insertion-loss deviation, a COM-like budget, causality and crosstalk).

7.1. Bit-Error Probability from the Eye Density

7.1.1. Model

The sample density of the eye (Section 6.7) describes the *deterministic* distribution of the received voltage at each sampling phase. Additive Gaussian voltage noise with standard deviation σ_n turns every deterministic sample into a Gaussian cloud around it. For a decision threshold T, a sample at voltage v above the threshold is decided wrongly if the noise pushes it below T, and vice versa:

$$P_e(v{>}T) = Q\!\left(\frac{v-T}{\sigma_n}\right), \qquad P_e(v{<}T) = Q\!\left(\frac{T-v}{\sigma_n}\right), \qquad Q(x) = \tfrac{1}{2}\,\mathrm{erfc}\!\left(\tfrac{x}{\sqrt{2}}\right). \tag{7.1}$$

7.1.2. Horizontal bathtub

Let $p_t(v)$ be the normalized density at time column t (averaged over the two displayed UI). For each threshold T_k, samples are assigned to the cluster above or below it using the neighbouring thresholds as limits, and the bit-error probability at sampling phase t is the sum over thresholds and bins,

$$\mathrm{BER}(t) = \sum_k \Big[\sum_{v\in\mathcal{U}_k} p_t(v)\, Q\!\Big(\tfrac{v-T_k}{\sigma_n}\Big) + \sum_{v\in\mathcal{L}_k} p_t(v)\, Q\!\Big(\tfrac{T_k-v}{\sigma_n}\Big) \Big], \tag{7.2}$$

where $\mathcal{U}_k$ and $\mathcal{L}_k$ are the clusters above and below T_k. For NRZ this is exactly the average error probability; for multi-level signals it is a *symbol*-error estimate. The verification suite compares the result of Equation (7.2) for a two-rail eye with the closed form $Q(a/\sigma_n)$ and requires a relative error below 10^{-6}.

7.1.3. Random jitter

Random timing jitter with standard deviation σ_j smears the bathtub curve in time. It is applied as a circular convolution of $\mathrm{BER}(t)$ with a Gaussian whose width is σ_j expressed in samples, $s_j = \sigma_j O/T$ (applied only if $s_j > 0.05$). The curve is limited to the range $[10^{-18}, 0.5]$ for plotting.

7.1.4. Vertical bathtub and BER contour

Keeping the sampling phase fixed and varying the threshold y of the centre eye gives the vertical bathtub,

$$\mathrm{BER}(y) = \sum_{v \le T_c} p(v)\, Q\!\left(\frac{y - v}{\sigma_n}\right) + \sum_{v > T_c} p(v)\, Q\!\left(\frac{v - y}{\sigma_n}\right), \tag{7.3}$$

and evaluating it for every phase and 161 threshold values gives the BER contour. The contour is computed as two matrix products, $\mathbf{P}_{lo}\mathbf{Q}_{lo} + \mathbf{P}_{up}\mathbf{Q}_{up}$, where the rows of $\mathbf{P}$ are the densities per time column and the columns of $\mathbf{Q}$ the error probabilities per threshold, followed by the same jitter convolution along time.

7.1.5. Eye opening at a target BER

The width and height of the eye opening at a target BER (for example 10^{-12}) are the extent of the contiguous region around the minimum in which the curve stays below the target:

$$W_{BER} = \frac{\left|\{t : \mathrm{BER}(t) < \mathrm{BER}_{tgt}\}\right|}{O}\ \mathrm{UI}, \qquad H_{BER} = \left|\{y : \mathrm{BER}(y) < \mathrm{BER}_{tgt}\}\right| \Delta y. \tag{7.4}$$

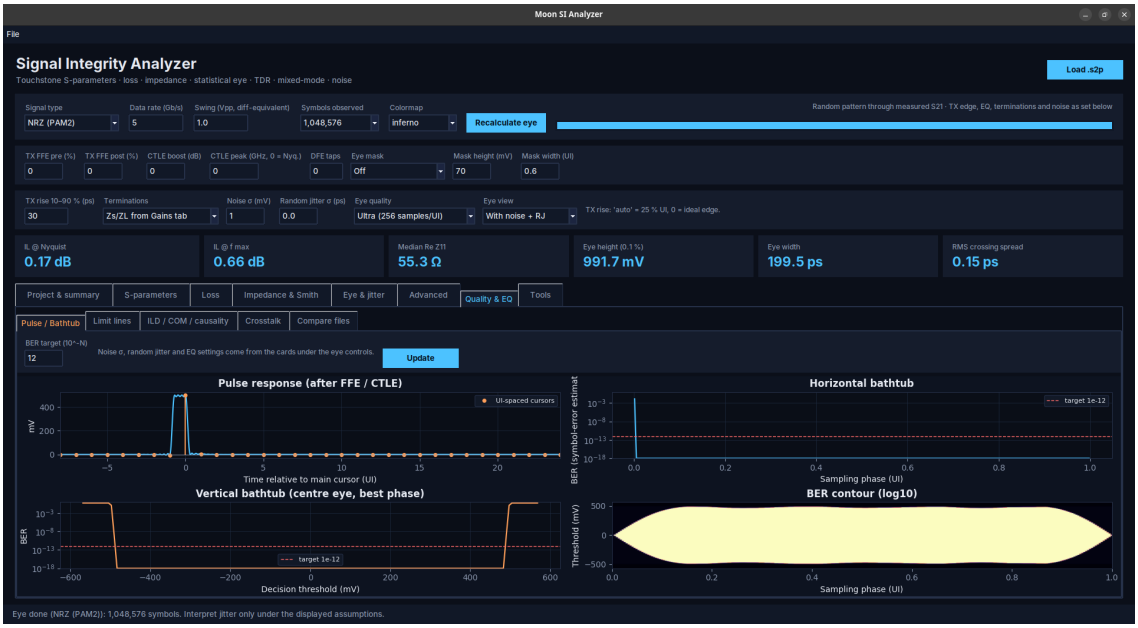


Figure 7.1: Pulse response, bathtub curves and BER contour.

Source: `si_analyzer.py` — `ber_analysis`

```
def ber_analysis(r: dict[str, Any], sigma_v: float, rj_ps: float, target: float) -> dict[str, Any]:
    """Statistical BER estimate from the eye density plus Gaussian voltage noise and random jitter.

    Horizontal bathtub uses all decision thresholds; the vertical bathtub and the contour use the centre
    threshold. For multi-level signals the result is a symbol-error estimate, not a bit-error rate.
    """
    dens = r["density"]
    os_ = r["oversample"]
    nb = dens.shape[1]
    edges_v = np.linspace(r["v_min"], r["v_max"], nb + 1)
    centers = 0.5 * (edges_v[:-1] + edges_v[1:])
```

```
    p = 0.5 * (dens[:os_] + dens[os_:2 * os_])
    p = p / np.maximum(p.sum(axis=1, keepdims=True), 1.0)
    sigma = max(float(sigma_v), 1e-9)
    thr = np.sort(np.asarray(r["thresholds_v"], dtype=float))
    bounds = np.concatenate([[-np.inf], thr, [np.inf]])

    def q(x: np.ndarray) -> np.ndarray:
        return 0.5 * erfc(x / math.sqrt(2.0))

    sj = rj_ps / r["ui_ps"] * os_
    ber_t = np.zeros(os_)
    for k in range(1, len(bounds) - 1):
        lo = (centers > bounds[k - 1]) & (centers <= bounds[k])
        up = (centers > bounds[k]) & (centers <= bounds[k + 1])
        ber_t += p[:, up] @ q((centers[up] - thr[k - 1]) / sigma) + p[:, lo] @ q((thr[k - 1]
            - centers[lo]) / sigma)
    if sj > 0.05:
        ber_t = gaussian_filter1d(ber_t, sj, mode="wrap")
    ber_t = np.clip(ber_t, 1e-18, 0.5)
    x_ui = (np.arange(os_) + 0.5) / os_

    def opening(ok: np.ndarray, i0: int) -> tuple[int, int]:
        if not ok[i0]:
            return i0, i0 - 1
        lft = rgt = i0
        while lft > 0 and ok[lft - 1]:
            lft -= 1
        while rgt < len(ok) - 1 and ok[rgt + 1]:
            rgt += 1
        return lft, rgt

    l, rr = opening(ber_t < target, int(np.argmin(ber_t)))
    width_ui = max(0, rr - l + 1) / os_

    t_mid = thr[len(thr) // 2]
    y = np.linspace(r["v_min"], r["v_max"], 161)
    lo, up = centers <= t_mid, centers > t_mid
    q_lo = q((y[None, :] - centers[lo][:, None]) / sigma)
    q_up = q((centers[up][:, None] - y[None, :]) / sigma)
    contour = p[:, lo] @ q_lo + p[:, up] @ q_up
    if sj > 0.05:
        contour = gaussian_filter1d(contour, sj, axis=0, mode="wrap")
    contour = np.clip(contour, 1e-18, 0.5)
    col = min(int(round(r["sample_phase_ui"] * os_)), os_ - 1)
    ber_y = contour[col]
    yl, yr = opening(ber_y < target, int(np.argmin(ber_y)))
    height_v = (yr - yl + 1) * float(y[1] - y[0]) if yr >= yl else 0.0
    return {"x_ui": x_ui, "ber_t": ber_t, "y": y, "ber_y": ber_y, "contour": contour,
            "width_ui": width_ui, "width_ps": width_ui * r["ui_ps"], "height_v": height_v, "
                target": target}
```

7.2. Eye Masks

Many standards define a forbidden region in the centre of the eye; the signal complies if no trace enters it. The program draws a hexagonal mask centred at each eye centre (0.5 and 1.5 UI) and at each threshold voltage, with full height H_m and full width W_m at the middle, and a flat top and bottom of half that width:

$$\left(c \pm \tfrac{W_m}{2},\, v_c\right),\ \left(c \pm \tfrac{W_m}{4},\, v_c \pm \tfrac{H_m}{2}\right). \tag{7.5}$$

The check counts the samples of the statistical histogram that lie inside any mask polygon (using Matplotlib's point-in-polygon test) and the result is *pass* if the count is zero. The interface offers a "Custom" mask and three generic templates ($100\,\mathrm{mV}\times0.40$ UI, $50\,\mathrm{mV}\times0.30$ UI, $25\,\mathrm{mV}\times0.20$ UI); these templates are illustrative and are *not* the masks of any standard. Specification values must be entered by the user.

Source: `si_analyzer.py` — `mask_polygon`

```
def mask_polygon(c_ui: float, c_v: float, h_v: float, w_ui: float, top: float = 0.5) -> np.ndarray:
    a, b, h = w_ui / 2.0, w_ui / 2.0 * top, h_v / 2.0
    return np.array([[c_ui - a, c_v], [c_ui - b, c_v + h], [c_ui + b, c_v + h],
                     [c_ui + a, c_v], [c_ui + b, c_v - h], [c_ui - b, c_v - h]])
```

Source: `si_analyzer.py` — `mask_check`

```
def mask_check(r: dict[str, Any], h_v: float, w_ui: float) -> tuple[list[np.ndarray], int, bool]:
    """Hexagonal eye mask on every sub-eye of both displayed UIs; PASS means zero hits inside."""
    dens = r["density"]
    os_ = r["oversample"]
    nb = dens.shape[1]
    edges_v = np.linspace(r["v_min"], r["v_max"], nb + 1)
    centers = 0.5 * (edges_v[:-1] + edges_v[1:])
    polys = [mask_polygon(c, float(t), h_v, w_ui) for c in (0.5, 1.5) for t in r["thresholds_v"]]
    xx, yy = np.meshgrid((np.arange(2 * os_) + 0.5) / os_, centers, indexing="ij")
    pts = np.column_stack([xx.ravel(), yy.ravel()])
    inside = np.zeros(len(pts), dtype=bool)
    for poly in polys:
        inside |= MplPath(poly).contains_points(pts)
    hits = int(dens.ravel()[inside].sum())
    return polys, hits, hits == 0
```

Source: `si_analyzer.py` — `ExtraTabs._draw_mask_overlay`

```
def _draw_mask_overlay(self, ax: Any, r: dict[str, Any]) -> None:
    r.pop("mask_result", None)
    if self.mask_var.get() == "Off":
        return
    h = max(self._fv(self.mask_h_var, 50.0), 0.0) / 1e3
    w = min(max(self._fv(self.mask_w_var, 0.3), 0.01), 1.0)
    polys, hits, ok = mask_check(r, h, w)
    r["mask_result"] = (hits, ok)
    col = GREEN if ok else RED
    for poly in polys:
        ax.add_patch(Polygon(poly, closed=True, fill=False, edgecolor=col, linewidth=1.6))
    ax.text(0.985, 0.03, f"Mask {'PASS' if ok else 'FAIL'}  ({hits:,} hits)", transform=ax.transAxes, ha="right",
            color=col, fontsize=10, fontweight="bold",
            bbox=dict(boxstyle="round,pad=0.4", facecolor=CARD, edgecolor=BORDER, alpha=0.9))
```

7.3. Limit Lines

Channel specifications often take the form of a frequency-dependent limit on insertion loss (a maximum) or return loss (a minimum). The user enters points as `f_GHz:dB` pairs separated by commas; the limit between points is linearly interpolated and the check covers the frequency range between the first and the last point. The margin is

$$m_{IL}(f) = L_{IL}(f) - \mathrm{IL}(f), \qquad m_{RL}(f) = \min\!\big(\mathrm{RL}_{11}, \mathrm{RL}_{22}\big) - L_{RL}(f), \tag{7.6}$$

and the result is a pass if the smallest margin is not negative. The tab reports the worst margin and the frequency where it occurs. The default points in the interface are placeholders.

Source: `si_analyzer.py` — `parse_limit_points`

```
def parse_limit_points(text: str) -> tuple[np.ndarray, np.ndarray]:
    """'f_GHz:dB, f_GHz:dB, ...' -> sorted arrays (Hz, dB)."""
    pts = []
    for chunk in text.replace(";", ",").split(","):
        if ":" in chunk:
```

```
            a, b = chunk.split(":", 1)
            try:
                pts.append((float(a) * 1e9, float(b)))
            except ValueError:
                pass
    pts.sort()
    return np.array([p[0] for p in pts]), np.array([p[1] for p in pts])
```

7.4. Insertion-Loss Deviation

A smooth channel loses power in a way that follows conductor (skin-effect) and dielectric loss, which grow approximately with $\sqrt{f}$ and f. Resonances and reflections add ripple. The *insertion-loss deviation* (ILD) is the difference between the measured loss and a smooth fit, here a least-squares polynomial in the basis $\{1, \sqrt{f}, f, f^2\}$,

$$\mathrm{IL}_{fit}(f) = c_0 + c_1\sqrt{f} + c_2 f + c_3 f^2, \qquad \mathrm{ILD}(f) = \mathrm{IL}(f) - \mathrm{IL}_{fit}(f). \tag{7.7}$$

The RMS and the peak of the ILD are reported. Large ILD indicates resonant structures (stubs, via residues, plane resonances) that equalizers cannot easily compensate.

7.5. COM-Like Budget

The channel operating margin of Ethernet [**ieee8023bj**] compares the signal amplitude after equalization with the sum of the residual interference and noise at a target error ratio. The program provides a *simplified, peak-distortion* variant, intentionally named “COM-like” to avoid confusion with the standard procedure, based on the equalized pulse response of Section 6.4:

$$A_s = |h_0|, \tag{7.8}$$

$$A_{isi} = \sum_{k<0} |h_k| + \sum_{k>N_{DFE}} |h_k|, \tag{7.9}$$

$$A_{ni} = A_{isi} + Q_{tgt}\sqrt{\sigma_x^2 + \sigma_n^2}, \tag{7.10}$$

$$\mathrm{COM}_{like} = 20\log_{10}\frac{A_s}{A_{ni}}, \tag{7.11}$$

with $Q_{tgt} = 7.03$ for a Gaussian tail of 10^{-12}, the noise σ_n from the interface, and the crosstalk σ_x taken from the integrated crosstalk noise of Section 7.7 if a multi-port file has been analysed. The cursors considered are those in the window $-8 \ldots +24$ UI of the pulse response. The peak-distortion approach is a conservative bound and does not include the optimization of equalizer coefficients, jitter, or the exact transmitter and receiver models of the standard. The tab shows the four terms as a bar chart.

Source: `si_analyzer.py` — `si_analyzer.py`, lines 1893–2061

```
    r = self.eye_result
    if r is None:
        return self._placeholder("bath", "Run␣the␣eye␣calculation␣first␣(needs␣a␣loaded␣
            channel).")
    sigma = max(self._fv(self.noise_var, 1.0), 0.0) / 1e3
    rj = max(self._fv(self.rj_var, 0.0), 0.0)
    target = 10.0 ** (-abs(self._fv(self.ber_var, 12.0)))
    ba = ber_analysis(r, sigma, rj, target)
    pu = r["pulse"]
    fig = self.fig_bath
    fig.clf()
    ax1, ax2, ax3, ax4 = fig.subplots(2, 2).ravel()
    ax1.plot(pu["x_ui"], pu["y"] * 1e3, color=ACCENT)
    ks, cur = pu["k"], pu["cursors"] * 1e3
    ax1.vlines(ks, 0, cur, color=ACCENT2, linewidth=1.4)
    ax1.plot(ks, cur, "o", color=ACCENT2, markersize=4, label="UI-spaced␣cursors")
    if r["dfe_n"]:
```

```
        ax1.plot(np.arange(1, r["dfe_n"] + 1), r["dfe_taps_v"] * 1e3, "s", color=GREEN,
            markersize=5, label="DFE-cancelled")
    ax1.axhline(0, color=MUTED, linewidth=0.7)
    ax1.set_title("Pulse response (after FFE / CTLE)")
    ax1.set_xlabel("Time relative to main cursor (UI)")
    ax1.set_ylabel("mV")
    ax1.set_xlim(-8, 24)
    ax2.semilogy(ba["x_ui"], ba["ber_t"], color=ACCENT)
    ax2.axhline(target, color=RED, linestyle="--", linewidth=1, label=f"target {target:.0e}")
    ax2.set_title("Horizontal bathtub")
    ax2.set_xlabel("Sampling phase (UI)")
    ax2.set_ylabel("BER (symbol-error estimate)")
    ax2.set_ylim(1e-18, 1.0)
    ax3.semilogy(ba["y"] * 1e3, ba["ber_y"], color=ACCENT2)
    ax3.axhline(target, color=RED, linestyle="--", linewidth=1, label=f"target {target:.0e}")
    ax3.set_title("Vertical bathtub (centre eye, best phase)")
    ax3.set_xlabel("Decision threshold (mV)")
    ax3.set_ylabel("BER")
    ax3.set_ylim(1e-18, 1.0)
    z = np.log10(ba["contour"].T)
    ax4.imshow(z, origin="lower", aspect="auto", cmap="magma_r", vmin=-18, vmax=0,
               extent=(0, 1, ba["y"][0] * 1e3, ba["y"][-1] * 1e3))
    try:
        cs = ax4.contour(ba["x_ui"], ba["y"] * 1e3, z, levels=[-3, -6, -9, -12], colors="
            white", linewidths=0.9)
        ax4.clabel(cs, fmt=lambda v: f"1e{int(v)}", fontsize=7)
    except Exception:
        pass
    ax4.set_title("BER contour (log10)")
    ax4.set_xlabel("Sampling phase (UI)")
    ax4.set_ylabel("Threshold (mV)")
    for ax in (ax1, ax2, ax3):
        ax.grid(True)
        ax.legend(loc="best", fontsize=8)
    self._finish("bath")
    lines = [f"Settings: {r['eq_text']}",
             f"Noise  {sigma*1e3:.2f} mV, random jitter  {rj:.2f} ps, BER target {target:.0e
                 }",
             f"Eye opening at target: width {ba['width_ui']:.3f} UI ({ba['width_ps']:.2f} ps)
                 , height {ba['height_v']*1e3:.2f} mV"]
    if "mask_result" in r:
        lines.append(f"Eye mask: {'PASS' if r['mask_result'][1] else 'FAIL'} ({r['mask_result
            '][0]:,} hits)")
    self._set_section("eqmask", "EQ / bathtub / mask", lines)

# -- limit lines ------------------------------------------------------------------------
def _draw_lim(self) -> None:
    if self.channel is None:
        return self._placeholder("lim", "Load a Touchstone file first.")
    d, f = self.channel, self.channel.frequency_hz
    g = f / 1e9
    il = -_db(d.s[:, 1, 0])
    fi, li = parse_limit_points(self.il_lim_var.get())
    fr, lr = parse_limit_points(self.rl_lim_var.get())
    fig = self.fig_lim
    fig.clf()
    ax1, ax2 = fig.subplots(1, 2)
    ax1.plot(g, il, color=ACCENT2, label="Insertion loss")
    ax2.plot(g, -_db(d.s[:, 0, 0]), color=ACCENT, label="RL S11")
    ax2.plot(g, -_db(d.s[:, 1, 1]), color=PURPLE, linestyle="--", label="RL S22")
    lines = []
    if len(fi) >= 2:
        m = (f >= fi[0]) & (f <= fi[-1])
        lim = np.interp(f, fi, li)
        ax1.plot(fi / 1e9, li, color=RED, linewidth=2, label="Limit (max IL)")
        if m.any():
            margin = lim[m] - il[m]
            lines.append(f"Insertion loss vs limit: {'PASS' if margin.min() >= 0 else 'FAIL
                '}, worst margin {margin.min():.2f} dB @ {g[m][margin.argmin()]:.4g} GHz")
    if len(fr) >= 2:
        m = (f >= fr[0]) & (f <= fr[-1])
```

```
        lim = np.interp(f, fr, lr)
        ax2.plot(fr / 1e9, lr, color=RED, linewidth=2, label="Limit (min RL)")
        if m.any():
            margin = np.minimum(-_db(d.s[m, 0, 0]), -_db(d.s[m, 1, 1])) - lim[m]
            lines.append(f"Return loss vs limit: {'PASS' if margin.min() >= 0 else 'FAIL'}, "
                f"worst margin {margin.min():.2f} dB @ {g[m][margin.argmin()]:.4g} GHz")
    ax1.set_title("Insertion loss")
    ax2.set_title("Return loss")
    for ax in (ax1, ax2):
        ax.set_xlabel("Frequency (GHz)")
        ax.set_ylabel("dB")
        ax.grid(True)
        ax.legend(loc="best")
    self._finish("lim")
    if lines:
        self._set_section("lim", "Limit lines", lines)

# -- ILD, COM-like budget, causality -------------------------------------------
def _draw_qual(self) -> None:
    if self.channel is None:
        return self._placeholder("qual", "Load a Touchstone file first.")
    d, f = self.channel, self.channel.frequency_hz
    g = f / 1e9
    s21 = d.s[:, 1, 0]
    il = -_db(s21)
    basis = np.column_stack([np.ones_like(g), np.sqrt(g), g, g * g])
    coef = np.linalg.lstsq(basis, il, rcond=None)[0]
    fit = basis @ coef
    ild = il - fit
    t, x = impulse_full(f, s21)
    tau = float(np.nanmedian(group_delay(f, s21)))
    tau = tau if np.isfinite(tau) and tau > 0 else 0.0
    energy = float(np.sum(x ** 2))
    pre = float(np.sum(x[t < 0.5 * tau] ** 2) / energy * 100.0) if energy > 0 else float("nan")
    sig = float(passivity(d.s).max())

    fig = self.fig_qual
    fig.clf()
    ax1, ax2, ax3, ax4 = fig.subplots(2, 2).ravel()
    ax1.plot(g, il, color=ACCENT2, label="IL")
    ax1.plot(g, fit, "--", color=GREEN, label="a + √bf + c f + d f²")
    ax1.set_title("Insertion loss and smooth fit")
    ax1.set_ylabel("dB")
    ax2.plot(g, ild, color=PURPLE, label="ILD")
    ax2.axhline(0, color=MUTED, linewidth=0.7)
    ax2.set_title("Insertion loss deviation (ILD)")
    ax2.set_ylabel("dB")
    win = (t > -0.5e-9 - 2 * tau) & (t < 4 * tau + 1e-9)
    ax3.plot(t[win] * 1e9, x[win] * 1e-9, color=ACCENT, label="h(t) of S21")
    ax3.axvline(0.5 * tau * 1e9, color=RED, linestyle="--", linewidth=1, label="0.5  (pre-arrival limit)")
    ax3.set_title("Impulse response (causality view)")
    ax3.set_xlabel("Time (ns)")
    ax3.set_ylabel("1/ns")
    for ax in (ax1, ax2):
        ax.set_xlabel("Frequency (GHz)")
    for ax in (ax1, ax2, ax3):
        ax.grid(True)
        ax.legend(loc="best", fontsize=8)

    lines = [f"ILD: RMS {np.sqrt(np.mean(ild**2)):.3f} dB, peak {np.max(np.abs(ild)):.3f} dB",
             f"Causality (heuristic): {pre:.3f} % of the S21 impulse energy arrives before 0.5 × the median group delay",
             f"({'looks causal' if pre < 1.0 else 'possible non-causality / truncation ringing'})",
             f"Passivity: max (S) = {sig:.4f} ({'passive' if sig <= 1.0005 else 'ACTIVE - use Enforce passivity'})"]
    r = self.eye_result
```

```
    if r is not None:
        pu = r["pulse"]
        k, cur = pu["k"], pu["cursors"]
        main = float(abs(cur[k == 0][0]))
        isi = float(np.abs(cur[k < 0]).sum() + np.abs(cur[k > r["dfe_n"]]).sum())
        q = 7.03  # Gaussian Q for 1e-12
        xt = self._xt_icn_v * q
        nz = max(self._fv(self.noise_var, 1.0), 0.0) / 1e3 * q
        total = isi + math.sqrt(xt ** 2 + nz ** 2)
        com = 20.0 * math.log10(main / total) if total > 0 and main > 0 else float("nan")
        labels = ["Signal (main cursor)", "Residual ISI", "Crosstalk (Q·ICN)", "Noise (Q· )"]
        vals = [main * 1e3, isi * 1e3, xt * 1e3, nz * 1e3]
        ax4.bar(range(4), vals, color=[GREEN, ACCENT2, PURPLE, RED])
        ax4.set_xticks(range(4), labels, rotation=15, fontsize=8)
        ax4.set_ylabel("mV")
        ax4.set_title(f"COM-like budget: {com:.2f} dB")
        lines.append(f"COM-like (simplified, peak-distortion): {com:.2f} dB with {r['eq_text
            ']}; "
                     f"signal {main*1e3:.1f} mV, ISI {isi*1e3:.1f} mV, crosstalk {xt*1e3:.1f}
                         mV, noise {nz*1e3:.1f} mV. "
                     "Not an IEEE 802.3 COM.")
        ax4.grid(True, axis="y")
    else:
        ax4.axis("off")
        ax4.text(0.5, 0.5, "Run the eye calculation\nfor the COM-like budget", ha="center",
            va="center", color=MUTED, transform=ax4.transAxes)
    self._finish("qual")
    self._set_section("qual", "Channel quality", lines)
```

7.6. Passivity Enforcement

Data that violate passivity (Equation (4.4)) by a small amount can be repaired by limiting the singular values. With the decomposition $\mathbf{S} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^{H}$ at each frequency,

$$\hat{\mathbf{S}} = \mathbf{U}\,\min(\mathbf{\Sigma},\,0.999)\,\mathbf{V}^{H}. \tag{7.12}$$

This is a simple, local repair; it does not guarantee that the *causality* of the data is preserved and it changes the response wherever it acts, so it should be used for small violations only. The verification suite checks that the largest singular value after the repair does not exceed 0.999.

Source: `si_analyzer.py` — `enforce_passivity`

```
def enforce_passivity(s: np.ndarray, limit: float = 0.999) -> np.ndarray:
    """Clip the singular values of S at every frequency to `limit` (simple passivity repair).
        """
    u, sv, vh = np.linalg.svd(s)
    return (u * np.minimum(sv, limit)[:, None, :]) @ vh
```

7.7. Crosstalk

For multi-lane connectors and cables the crosstalk between lanes limits performance. The Crosstalk tab loads a separate file with $N \geq 4$ ports (even), and the user selects the victim lane and the port ordering. Two conventions are supported: “halves” (ports $1 \ldots N/2$ on the near end, $N/2+1 \ldots N$ on the far end) and “odd/even” (odd ports near, even ports far). For a victim lane v and aggressor lane i, with n_i and f_i the near and far port of lane i,

$$\mathrm{NEXT}_i = S_{n_v n_i}, \qquad \mathrm{FEXT}_i = S_{f_v n_i}, \qquad \mathrm{IL}_v = -20\log_{10}|S_{f_v n_v}|. \tag{7.13}$$

The power sums over all aggressors are

$$\mathrm{PSXT} = 10\log_{10}\sum_{i}|X_i|^2, \qquad \mathrm{PSACR} = 20\log_{10}|S_{f_v n_v}| - \mathrm{PSXT}\ \text{(in dB)}, \tag{7.14}$$

so that a positive PSACR means that the wanted signal is stronger than the summed crosstalk (PSACR-N uses the near-end and PSACR-F the far-end crosstalk). To express crosstalk as a noise voltage, the program computes the *integrated crosstalk noise* (ICN). The crosstalk transfer function is weighted by the spectrum of the signalling and the bandwidth of the receiver; for baud frequency f_b and amplitude $A = V_{pp}/2$ the weight is, in the style of IEEE 802.3 **[ieee8023bj]**,

$$W(f) = \frac{A^2}{f_b}\,\operatorname{sinc}^2\!\Big(\frac{f}{f_b}\Big)\,\frac{1}{1+(f/f_t)^4}\,\frac{1}{1+(f/f_r)^8}, \qquad f_t = f_r = 0.75 f_b, \tag{7.15}$$

and the RMS noise is obtained by integrating over frequency (trapezoidal rule) and summing the aggressors,

$$\sigma_x = \sqrt{\sum_{i} \int_0^{f_{max}} W(f)\,|X_i(f)|^2\,df}. \tag{7.16}$$

The NEXT and FEXT contributions are reported separately and combined as $\sqrt{\sigma_{x,N}^2 + \sigma_{x,F}^2}$. Because the weighting is an approximation of the standard one and the amplitude convention (peak, $V_{pp}/2$) is a choice, the numbers should be compared only between designs analysed in the same way. The verification suite checks that doubling the coupling doubles the ICN.

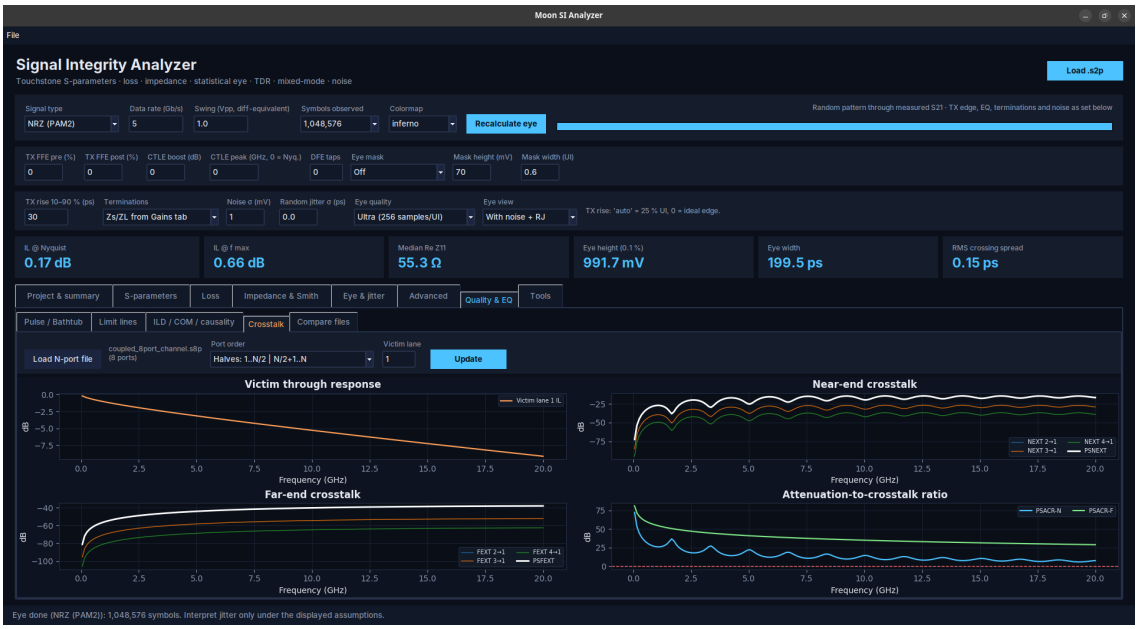


Figure 7.2: Crosstalk analysis of a four-lane connector.

Source: `si_analyzer.py` — `icn_rms`

```
def icn_rms(f: np.ndarray, xs: list[np.ndarray], fb: float, amp: float) -> float:
    """Integrated crosstalk noise (RMS volts) with an IEEE 802.3-style weighting (ft = fr = 0.75 fb)."""
    if not xs:
        return 0.0
    ft = fr = 0.75 * fb
    w = amp ** 2 / fb * np.sinc(f / fb) ** 2 / (1.0 + (f / ft) ** 4) / (1.0 + (f / fr) ** 8)
    return math.sqrt(sum(float(_TRAPZ(w * np.abs(x) ** 2, f)) for x in xs))
```

Source: `si_analyzer.py` — `ExtraTabs._draw_xt`

```
def _draw_xt(self) -> None:
    net = self.xt_net
    if net is None:
        return self._placeholder("xt", "Load an N-port (4, 8, …) crosstalk file with the button above.")
    n = net.nports
    lanes = n // 2
    if self.xt_order_var.get().startswith("Halves"):
        near, far = list(range(lanes)), list(range(lanes, n))
    else:
        near, far = list(range(0, n, 2)), list(range(1, n, 2))
    v = min(max(int(self._fv(self.xt_victim_var, 1.0)) - 1, 0), lanes - 1)
    aggs = [i for i in range(lanes) if i != v]
    f = np.asarray(net.f, dtype=float)
    g = f / 1e9
    s = np.asarray(net.s, dtype=complex)
    il = s[:, far[v], near[v]]
    nexts = [s[:, near[v], near[i]] for i in aggs]
    fexts = [s[:, far[v], near[i]] for i in aggs]
    ps = lambda xs: 10.0 * np.log10(np.maximum(sum(np.abs(x) ** 2 for x in xs), 1e-30))  # noqa: E731
    fig = self.fig_xt
    fig.clf()
    ax1, ax2, ax3, ax4 = fig.subplots(2, 2).ravel()
    ax1.plot(g, _db(il), color=ACCENT2, label=f"Victim lane {v + 1} IL")
    ax1.set_title("Victim through response")
    for i, x in zip(aggs, nexts):
        ax2.plot(g, _db(x), linewidth=1, alpha=0.8, label=f"NEXT {i + 1}→{v + 1}")
    ax2.plot(g, ps(nexts), color="white", linewidth=2, label="PSNEXT")
    ax2.set_title("Near-end crosstalk")
    for i, x in zip(aggs, fexts):
        ax3.plot(g, _db(x), linewidth=1, alpha=0.8, label=f"FEXT {i + 1}→{v + 1}")
    ax3.plot(g, ps(fexts), color="white", linewidth=2, label="PSFEXT")
    ax3.set_title("Far-end crosstalk")
    ax4.plot(g, _db(il) - ps(nexts), color=ACCENT, label="PSACR-N")
    ax4.plot(g, _db(il) - ps(fexts), color=GREEN, label="PSACR-F")
    ax4.axhline(0, color=RED, linestyle="--", linewidth=1)
    ax4.set_title("Attenuation-to-crosstalk ratio")
    for ax in (ax1, ax2, ax3, ax4):
        ax.set_xlabel("Frequency (GHz)")
        ax.set_ylabel("dB")
        ax.grid(True)
        ax.legend(loc="best", fontsize=7, ncol=2)
    self._finish("xt")
    fb = max(self._baud_hz(), 1.0)
    amp = max(self._fv(self.swing_var, 1.0), 0.0) / 2.0
    icn_n, icn_f = icn_rms(f, nexts, fb, amp), icn_rms(f, fexts, fb, amp)
    self._xt_icn_v = math.hypot(icn_n, icn_f)
    self._set_section("xt", "Crosstalk", [
        f"{n}-port file, {len(aggs)} aggressor(s) on victim lane {v + 1}",
        f"ICN (802.3-style weighting, fb = {fb/1e9:.4g} GBd, A = {amp:g} V): NEXT {icn_n*1e3:.3f} mV, FEXT {icn_f*1e3:.3f} mV, "
        f"total {self._xt_icn_v*1e3:.3f} mV RMS (feeds the COM-like budget)"])
    if self.eye_result is not None:
        self._draw_qual()
```

8

Modelling, Calibration and Conditioning

This chapter covers the tools that build models from S-parameters (equivalent circuits, rational approximations and SPICE netlists), remove the influence of measurement fixtures (TRL calibration, cascading and de-embedding), and prepare data for analysis (renormalization, resampling, reciprocity and passivity enforcement).

8.1. Per-Unit-Length Parameters of a Uniform Line

A uniform transmission line is described by the resistance R', inductance L', conductance G' and capacitance C' per unit length. Its complex propagation constant and characteristic impedance are

$$\gamma = \sqrt{(R' + j\omega L')(G' + j\omega C')}, \qquad Z_c = \sqrt{\frac{R' + j\omega L'}{G' + j\omega C'}}, \tag{8.1}$$

and the chain matrix of a section of length ℓ is

$$\begin{bmatrix} A & B \\ C & D \end{bmatrix} = \begin{bmatrix} \cosh\gamma\ell & Z_c \sinh\gamma\ell \\ \dfrac{\sinh\gamma\ell}{Z_c} & \cosh\gamma\ell \end{bmatrix}. \tag{8.2}$$

Given the measured ABCD matrix of such a line (Equations (3.7)–(3.8)) and its length, the parameters are recovered by inverting these relations. With $x = (A + D)/2 = \cosh\gamma\ell$ the quantity $\lambda = e^{-\gamma\ell}$ is the root of $\lambda^2 - 2x\lambda + 1 = 0$ with magnitude not larger than one; the program selects it numerically. The attenuation follows from the magnitude of λ, and the phase $\beta\ell$ from its *unwrapped* argument,

$$\alpha\ell = -\ln|\lambda|, \qquad \beta\ell = -\operatorname{unwrap}(\arg\lambda), \tag{8.3}$$

shifted by a multiple of 2π so that the first value lies near zero (low first frequency). The characteristic impedance is $Z_c = \sqrt{B/C}$ with positive real part. Then

$$R' + j\omega L' = \gamma Z_c, \qquad G' + j\omega C' = \frac{\gamma}{Z_c}, \qquad \varepsilon_{r,\mathrm{eff}} = \left(\frac{\beta c}{\omega}\right)^2, \tag{8.4}$$

with $\gamma = (\alpha\ell + j\beta\ell)/\ell$. The Equivalent-circuit tab plots R', L', G', C', Z_c and $\varepsilon_{r,\mathrm{eff}}$ against frequency. Frequency-dependent R' (skin effect) and G' (dielectric loss) are visible directly. The verification suite

generates a line with known R', L', G', C', converts it to S-parameters and requires the median recovered values to agree within $0.1\,\%$.

Source: `si_analyzer.py` — `extract_rlgc`

```
def extract_rlgc(f: np.ndarray, s: np.ndarray, z0: float, length_m: float) -> dict[str, np.ndarray]:
    """Per-unit-length R, L, G, C, Zc, gamma from the ABCD matrix of a uniform line of known length."""
    abcd = convert_parameters(s, z0)["ABCD"]
    a, b, c, d = abcd[:, 0, 0], abcd[:, 0, 1], abcd[:, 1, 0], abcd[:, 1, 1]
    w = 2.0 * np.pi * np.asarray(f, dtype=float)
    with np.errstate(all="ignore"):
        x = 0.5 * (a + d)
        root = np.sqrt(x * x - 1.0 + 0j)
        l1, l2 = x - root, x + root
        lam = np.where(np.abs(l1) <= np.abs(l2), l1, l2)  # e^{-gamma*l}
        lam = np.where(np.abs(lam) < 1e-12, 1e-12 + 0j, lam)
        alpha_l = -np.log(np.abs(lam))
        beta_l = -np.unwrap(np.angle(lam))
        beta_l = beta_l + 2.0 * np.pi * np.round(-beta_l[0] / (2.0 * np.pi))
        gamma = (alpha_l + 1j * beta_l) / length_m
        zc = np.sqrt(b / c)
        zc = np.where(np.real(zc) < 0, -zc, zc)
        zp, yp = gamma * zc, gamma / zc
        eps_eff = (np.imag(gamma) * SPEED_OF_LIGHT / w) ** 2
    return {"r": zp.real, "l": zp.imag / w, "g": yp.real, "c": yp.imag / w, "zc": zc, "gamma": gamma, "eps_eff": eps_eff}
```

8.2. Lumped π Equivalent Circuit

For electrically short structures a lumped model is more useful than a distributed one. The π model consists of a series impedance $Z_s = R + j\omega L$ and two shunt admittances $Y_1 = G_1 + j\omega C_1$ and $Y_2 = G_2 + j\omega C_2$. Its admittance matrix is

$$\mathbf{Y}_\pi = \begin{bmatrix} Y_1 + Y_s & -Y_s \\ -Y_s & Y_2 + Y_s \end{bmatrix}, \qquad Y_s = \frac{1}{Z_s}. \tag{8.5}$$

Inverting this relation, the elements follow from the measured admittance matrix: $Y_s = -Y_{12}$, $Y_1 = Y_{11} + Y_{12}$, $Y_2 = Y_{22} + Y_{12}$. In the lowest part of the band (default: lowest $5\,\%$ of the points, at least five), the element values are fitted. The series resistance and the shunt conductances are the medians of the real parts, and the reactive elements are the least-squares slopes through the origin,

$$L = \frac{\sum \omega \,\operatorname{Im} Z_s}{\sum \omega^2}, \qquad C_{1,2} = \frac{\sum \omega \,\operatorname{Im} Y_{1,2}}{\sum \omega^2}. \tag{8.6}$$

The fitted model is converted back to S-parameters with $\mathbf{S} = (\mathbf{I} + Z_0\mathbf{Y})^{-1}(\mathbf{I} - Z_0\mathbf{Y})$ and overlaid on the measured $|S_{21}|$ and $|S_{11}|$ so that the range of validity is visible. A lumped model is accurate only while the structure is short compared with the wavelength; for long lines the fit range must be reduced or the distributed model used.

Source: `si_analyzer.py` — `fit_pi_model`

```
def fit_pi_model(f: np.ndarray, s: np.ndarray, z0: float, frac: float = 0.25) -> dict[str, Any]:
    """Lumped pi model (series R+L, shunt G||C at both ends) fitted to the low part of the band."""
    y = convert_parameters(s, z0)["Y"]
    ys = -y[:, 0, 1]
    y1 = y[:, 0, 0] + y[:, 0, 1]
    y2 = y[:, 1, 1] + y[:, 0, 1]
    w = 2.0 * np.pi * np.asarray(f, dtype=float)
    k = min(len(f), max(5, int(round(frac * len(f)))))
    ww = w[:k]
```

```

    def slope(v: np.ndarray) -> float:
        ok = np.isfinite(v)
        return float(np.sum(ww[ok] * v[ok]) / np.sum(ww[ok] ** 2)) if ok.any() else float("
            nan")

    with np.errstate(all="ignore"):
        zs = 1.0 / ys
    r = float(np.nanmedian(zs.real[:k]))
    l_s = slope(zs.imag[:k])
    g1, g2 = float(np.nanmedian(y1.real[:k])), float(np.nanmedian(y2.real[:k]))
    c1, c2 = slope(y1.imag[:k]), slope(y2.imag[:k])
    with np.errstate(all="ignore"):
        ys_m = 1.0 / (r + 1j * w * l_s)
        y1m, y2m = g1 + 1j * w * c1, g2 + 1j * w * c2
    ym = np.empty_like(s)
    ym[:, 0, 0], ym[:, 0, 1], ym[:, 1, 0], ym[:, 1, 1] = y1m + ys_m, -ys_m, -ys_m, y2m + ys_m
    eye = np.broadcast_to(np.eye(2), s.shape)
    sm = _solve(eye + z0 * ym, eye - z0 * ym)
    return {"r": r, "l": l_s, "g1": g1, "c1": c1, "g2": g2, "c2": c2, "s_model": sm, "k": k}
```

Source: `si_analyzer.py` — `AdvancedTabs._draw_eq`

```
def _draw_eq(self) -> None:
    if self.channel is None:
        return self._placeholder("eq", "Load␣a␣Touchstone␣file␣first.")
    d, f = self.channel, self.channel.frequency_hz
    g = f / 1e9
    z1, z2 = self._z0s()
    z0 = 0.5 * (z1 + z2)
    frac = min(max(self._fv(self.fitpct_var, 25.0), 1.0), 100.0) / 100.0
    length = self._fv(self.len_var, 0.0) * 1e-3
    pi = fit_pi_model(f, d.s, z0, frac)
    rlgc = extract_rlgc(f, d.s, z0, length) if length > 0 else None

    fig = self.fig_eq
    fig.clf()
    axes = fig.subplots(2, 4).ravel()
    rlgc_plots = ((" R␣Ω(/m)", "r", 1.0), (" L␣(nH/m)", "l", 1e9), (" G␣(S/m)", "g", 1.0), (" C
        ␣(pF/m)", "c", 1e12))
    for ax, (title, key, scale) in zip(axes[:4], rlgc_plots):
        if rlgc is None:
            ax.text(0.5, 0.5, "Enter␣a␣line␣length\n(mm)␣>␣0", ha="center", va="center",
                color=MUTED, transform=ax.transAxes)
            ax.axis("off")
            continue
        ax.plot(g, rlgc[key] * scale, color=ACCENT)
        ax.set_title(title)
    if rlgc is not None:
        axes[4].plot(g, np.real(rlgc["zc"]), color=ACCENT2, label="Re␣Zc")
        axes[4].plot(g, np.imag(rlgc["zc"]), color=PURPLE, linestyle="--", label="Im␣Zc")
        axes[4].set_title("Characteristic␣impedance␣Ω()")
        axes[4].legend(loc="best")
        axes[5].plot(g, rlgc["eps_eff"], color=GREEN)
        axes[5].set_title(" r,eff␣from␣ ")
    else:
        axes[4].axis("off")
        axes[5].axis("off")
    axes[6].plot(g, _db(d.s[:, 1, 0]), color=ACCENT2, label="Measured")
    axes[6].plot(g, _db(pi["s_model"][:, 1, 0]), "--", color=GREEN, label=" ␣model")
    axes[6].set_title("|S21|␣(dB)")
    axes[7].plot(g, _db(d.s[:, 0, 0]), color=ACCENT, label="Measured")
    axes[7].plot(g, _db(pi["s_model"][:, 0, 0]), "--", color=GREEN, label=" ␣model")
    axes[7].set_title("|S11|␣(dB)")
    for ax in axes:
        if ax.axison:
            ax.set_xlabel("Frequency␣(GHz)")
            ax.grid(True)
    for ax in axes[6:]:
        ax.legend(loc="best", fontsize=8)
```

```
    self._finish("eq")

    lines = [
        f" model fitted on the lowest {pi['k']} points ({g[pi['k'] - 1]:.3g} GHz): "
        f"R = {pi['r']*1e3:.2f} Ωm, L = {pi['l']*1e9:.3f} nH",
        f"Shunt ends: C1 = {pi['c1']*1e12:.3f} pF, G1 = {pi['g1']*1e3:.4g} mS; C2 = {pi['c2
            ']*1e12:.3f} pF, G2 = {pi['g2']*1e3:.4g} mS",
    ]
    if rlgc is not None:
        k = pi["k"]
        lines.append(
            f"Per length (median, same range): R {np.nanmedian(rlgc['r'][:k]):.4g} Ω/m, L {
                np.nanmedian(rlgc['l'][:k])*1e9:.4g} nH/m, "
            f" G {np.nanmedian(rlgc['g'][:k]):.4g} S/m, C {np.nanmedian(rlgc['c'][:k])*1e12
                :.4g} pF/m, "
            f"|Zc| {np.nanmedian(np.abs(rlgc['zc'][:k])):.2f} Ω")
    self._set_section("eq", "Equivalent circuit", lines)
```

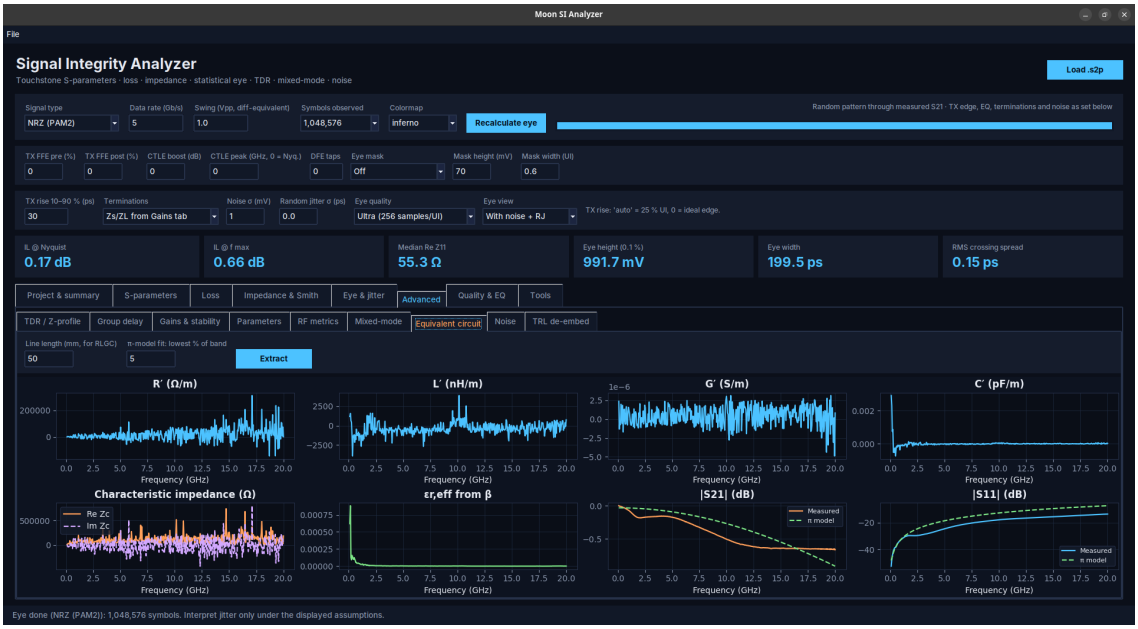


Figure 8.1: Per-unit-length parameters and π-model comparison.

8.3. Vector Fitting and SPICE Export

To use a channel in a circuit simulator it must be expressed in a form a time-domain solver can integrate. A rational approximation in the Laplace variable $s = j\omega$,

$$H(s) \approx \sum_{m=1}^{N} \frac{r_m}{s - p_m} + d + s\,h, \tag{8.7}$$

with poles p_m (real or in complex-conjugate pairs) and residues r_m, can be realized as a network of resistors, capacitors, inductors and controlled sources. *Vector fitting* [**gustavsen1999**] finds the poles and residues iteratively: starting from an initial pole set, it solves a linear least-squares problem for a scaling function, uses its zeros as relocated poles, and repeats until convergence; a final linear problem gives the residues. The program calls the implementation of scikit-rf, with the number of real poles and complex pairs set in the interface (defaults 6 and 16, which fit a $0.1\,\mathrm{m}$ lossy line to an RMS error of about 2×10^{-3}; with 4 and 8 the error is about 0.56, showing that long, delay-dominated channels need many poles).

After a fit the tool reports the RMS error and the number of passivity-violation bands found by the passivity test, offers a passivity enforcement step, and exports a SPICE subcircuit named `channel_fit`.

Source: `si_analyzer.py` — `ExtraTabs._vf_fit`

```
def _vf_fit(self) -> None:
    if self.channel is None:
        return messagebox.showinfo("Load a channel", "Load a Touchstone file first.")
    try:
        from skrf.vectorFitting import VectorFitting
        net = self._channel_network()
        vf = VectorFitting(net)
        vf.vector_fit(n_poles_real=int(self._fv(self.vf_real_var, 3)), n_poles_cmplx=int(self
            ._fv(self.vf_cmplx_var, 4)))
        self.vf, self.vf_net = vf, net
        err = float(np.max(vf.get_rms_error()))
        viol = vf.passivity_test()
        self.vf_status_var.set(f"RMS error {err:.2e}; passivity violation bands: {len(viol)}"
            )
    except Exception as exc:
        self.vf_status_var.set(f"Vector fit failed: {exc}")
        messagebox.showerror("Vector fitting failed", str(exc))
        return
    self._draw_vf()
```

Source: `si_analyzer.py` — `ExtraTabs._vf_passive`

```
def _vf_passive(self) -> None:
    if self.vf is None:
        return messagebox.showinfo("Fit first", "Run Fit first.")
    try:
        self.vf.passivity_enforce()
        self.vf_status_var.set("Passivity enforced; RMS error " + f"{float(np.max(self.vf.
            get_rms_error())):.2e}")
    except Exception as exc:
        self.vf_status_var.set(f"Passivity enforcement failed: {exc}")
        return
    self._draw_vf()
```

Source: `si_analyzer.py` — `ExtraTabs._vf_export`

```
def _vf_export(self) -> None:
    if self.vf is None:
        return messagebox.showinfo("Fit first", "Run Fit first.")
    path = filedialog.asksaveasfilename(defaultextension=".sp", filetypes=[("SPICE netlist",
        "*.sp *.cir"), ("All files", "*.*")])
    if path:
        try:
            self.vf.write_spice_subcircuit_s(path, fitted_model_name="channel_fit")
            self.vf_status_var.set(f"Wrote SPICE subcircuit 'channel_fit' to {Path(path).name
                }")
        except Exception as exc:
            messagebox.showerror("Export failed", str(exc))
```

8.4. TRL Calibration and De-embedding

A vector network analyzer measures a DUT through cables, probes or fixtures. Thru-reflect-line (TRL) calibration removes their effect using three simple standards, a *thru* (zero-length connection), a *reflect* (high-reflection, e.g. a short or open, whose exact value need not be known) and a *line* of known characteristic impedance and a length different from the thru [**engen1979**]. Each fixture half is represented by an error two-port $\mathbf{X}$ (port 1 side) and $\mathbf{Y}$ (port 2 side); the measured transfer matrices of the thru and the line are

$$\mathbf{M}_{thru} = \mathbf{X}\,\mathbf{Y}, \qquad \mathbf{M}_{line} = \mathbf{X}\,\mathbf{L}\,\mathbf{Y}, \tag{8.8}$$

where $\mathbf{L} = \mathrm{diag}(e^{-\gamma\ell}, e^{\gamma\ell})$ is the transfer matrix of the line. Eliminating $\mathbf{Y}$ gives $\mathbf{M}_{line}\mathbf{M}_{thru}^{-1} = \mathbf{X}\mathbf{L}\mathbf{X}^{-1}$, an eigenvalue problem whose eigenvectors give the columns of $\mathbf{X}$ up to scale factors, and the reflect

standard resolves the remaining ambiguities. The corrected DUT is $\mathbf{T}_{DUT} = \mathbf{X}^{-1}\mathbf{T}_{meas}\mathbf{Y}^{-1}$ in transfer-matrix form.

The electrical length of the line must differ from the thru by between about 20° and 160° over the whole band; at 180° the method is singular and beyond it the sign of the result is ambiguous. The calibration itself is performed by the TRL class of scikit-rf. The program adds the file handling: loading the four measurements, converting a one-port reflect measurement into a two-port with zero transmission, interpolating the standards onto the frequency grid of the DUT, running the calibration, plotting raw versus corrected responses, and optionally using the corrected DUT as the active channel. In the verification suite the algorithm recovers a random fixture pair to within 10^{-9} (observed error of the order of 10^{-14}).

Source: `si_analyzer.py` — `AdvancedTabs._trl_run`

```
def _trl_run(self) -> None:
    missing = [k for k in ("thru", "reflect", "line", "dut") if k not in self.trl_nets]
    if missing:
        messagebox.showinfo("Missing standards", "Still to load: " + ", ".join(missing))
        return
    try:
        dut = self.trl_nets["dut"]
        if dut.nports != 2:
            raise ValueError("The raw DUT must be a 2-port file.")

        def prep(net: Any) -> Any:
            if net.nports == 1:
                from skrf.network import two_port_reflect  # location differs between scikit-rf versions
                net = two_port_reflect(net, net)
            if net.nports != 2:
                raise ValueError("Standards must be 1-port (reflect) or 2-port files.")
            if len(net.f) != len(dut.f) or not np.allclose(net.f, dut.f):
                net = net.interpolate(dut.frequency)
            return net

        thru, refl, line = prep(self.trl_nets["thru"]), prep(self.trl_nets["reflect"]), prep(self.trl_nets["line"])
        est = -1 if self.reflect_var.get().startswith("Short") else 1
        cal, errors = None, []
        for kwargs in ({"n_reflects": 1, "reflect_est": est}, {"ideals": None, "n_reflects": 1, "reflect_est": est}):
            try:
                from skrf.calibration import TRL  # location differs between scikit-rf versions
                cal = TRL(measured=[thru, refl, line], **kwargs)
                break
            except Exception as exc:
                errors.append(str(exc))
        if cal is None:
            raise RuntimeError("; ".join(errors))
        cal.run()
        corrected = cal.apply_cal(dut)
    except Exception as exc:
        self.trl_status.set(f"TRL failed: {exc}")
        messagebox.showerror("TRL calibration failed", str(exc))
        return
    self.trl_result = corrected
    self.trl_status.set(f"TRL done: {len(corrected.f)} points. Plots show raw vs corrected.")
    fig = self.fig_trl
    fig.clf()
    ax1, ax2 = fig.subplots(1, 2)
    g = dut.f / 1e9
    for ax, (i, j), title in ((ax1, (1, 0), "|S21|"), (ax2, (0, 0), "|S11|")):
        ax.plot(g, _db(dut.s[:, i, j]), color=MUTED, label="Raw (with fixtures)")
        ax.plot(g, _db(corrected.s[:, i, j]), color=ACCENT2, label="TRL corrected")
        ax.set_title(title + " (dB)")
        ax.set_xlabel("Frequency (GHz)")
        ax.grid(True)
        ax.legend(loc="best")
    self._finish("trl")
```

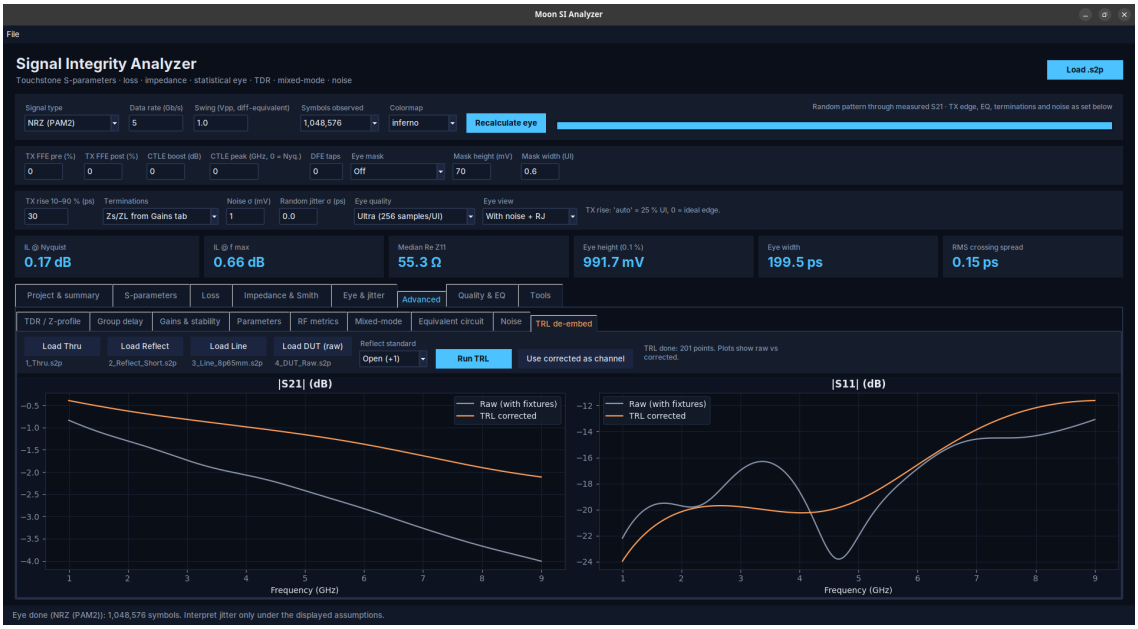


Figure 8.2: Raw and TRL-corrected responses of a DUT.

8.5. Cascading and De-embedding with Known Networks

When the fixture is known as a Touchstone file (for example from simulation) the T-matrix representation makes cascading trivial: the T-matrix of a cascade is the product,

$$\mathbf{T}_{AB} = \mathbf{T}_A \, \mathbf{T}_B, \tag{8.9}$$

and removing a known network is a multiplication with its inverse: $\mathbf{T}_{DUT} = \mathbf{T}_A^{-1}\mathbf{T}_{meas}$ removes a fixture at port 1 and $\mathbf{T}_{DUT} = \mathbf{T}_{meas}\mathbf{T}_B^{-1}$ at port 2. The Cascade tool offers five operations: channel followed by file, file followed by channel, de-embedding of a file at port 1, de-embedding at port 2, and swapping the ports. The file is interpolated onto the frequency grid of the channel if the grids differ. Every operation is previewed in the plot before it is applied and can be undone.

Source: `si_analyzer.py` — `ExtraTabs._casc_result`

```
def _casc_result(self) -> Any:
    main = self._channel_network()
    op = self.casc_op_var.get()
    if op == "Flip␣ports":
        return main.flipped()
    if self.casc_net is None:
        raise ValueError("Load␣a␣2-port␣file␣first.")
    other = self.casc_net
    if len(other.f) != len(main.f) or not np.allclose(other.f, main.f):
        other = other.interpolate(main.frequency)
    if op == CASCADE_OPS[0]:
        return main ** other
    if op == CASCADE_OPS[1]:
        return other ** main
    if op == CASCADE_OPS[2]:
        return other.inv ** main
    return main ** other.inv
```

Source: `si_analyzer.py` — `ExtraTabs._casc_apply`

```
def _casc_apply(self) -> None:
    if self.channel is None:
        return
    try:
```

```
        res = self._casc_result()
    except Exception as exc:
        messagebox.showerror("Operation failed", str(exc))
        return
    self._push_history()
    z0 = np.asarray(res.z0, dtype=complex)
    if z0.ndim == 1:
        z0 = np.repeat(z0[:, None], 2, axis=1)
    self.channel = ChannelData(Path(f"{self.channel.path.stem}-{self.casc_op_var.get().split(':')[0].split(' ')[0].lower()}.s2p"),
                               np.asarray(res.f, dtype=float), np.asarray(res.s, dtype=complex), z0)
    self.eye_result = None
    self._refresh_analysis()
    self._set_status("Applied: " + self.casc_op_var.get())
```

8.6. Conditioning Tools

The Conditioning tool performs five operations on the active channel, each stored in the undo history:

Renormalization. Re-expresses the network with a new reference impedance Z_0' (Section 3.4).

Resampling. Cubic interpolation onto a uniform grid (Section 5.4).

Add DC point. Extrapolation to zero frequency (Section 5.4).

Reciprocity enforcement. Replaces $\mathbf{S}$ by $\frac{1}{2}(\mathbf{S} + \mathbf{S}^T)$, which removes asymmetry noise from a reciprocal network; it is the closest reciprocal matrix in the Frobenius norm.

Passivity enforcement. Singular-value limiting (Section 7.6).

All operations create a new channel named from the old one with a suffix describing the operation. The previous channel is pushed onto the undo stack, and the whole analysis is refreshed.

Source: si_analyzer.py — UltimateTabs._apply_arrays

```
def _apply_arrays(self, f: np.ndarray, s: np.ndarray, z0: np.ndarray, label: str) -> None:
    d = self.channel
    self._push_history()
    if len(z0) != len(f):
        z0 = np.repeat(z0[:1], len(f), axis=0)
    self.channel = ChannelData(Path(f"{d.path.stem}-{label}.s2p"), np.asarray(f, dtype=float),
                               np.asarray(s, dtype=complex), z0)
    self.eye_result = None
    self._refresh_analysis()
    self._set_status(f"Conditioning applied: {label}")
```

Source: si_analyzer.py — UltimateTabs._cond_renorm

```
def _cond_renorm(self) -> None:
    if self.channel is None:
        return
    try:
        net = self._channel_network()
        net.renormalize(max(self._fv(self.z0_new_var, 50.0), 1e-3))
        z0 = np.asarray(net.z0, dtype=complex)
        if z0.ndim == 1:
            z0 = np.repeat(z0[:, None], 2, axis=1)
        self._apply_arrays(net.f, net.s, z0, f"z{self._fv(self.z0_new_var, 50.0):g}")
    except Exception as exc:
        messagebox.showerror("Renormalization failed", str(exc))
```

Source: si_analyzer.py — UltimateTabs._cond_recip

```
def _cond_recip(self) -> None:
    if self.channel is None:
        return
    s = 0.5 * (self.channel.s + self.channel.s.transpose(0, 2, 1))
    self._apply_arrays(self.channel.frequency_hz, s, self.channel.z0, "recip")
```

9

The Graphical User Interface

This chapter documents how the interface is built and how it connects to the numerical layer. It is organized from the outside in: window layout, theming, the pattern used to build every tab, interaction features (markers, undo), and the file-related functions (menu, sessions, exports, batch processing, logo).

9.1. Window Layout

The main window (Figure 1.1) is built from top to bottom:

1. **Menu bar** with the *File* menu (open, open with port mapping, recent files, sessions, quit).
2. **Header** with the logo (if present), the title, and the *Load .s2p* button.
3. **Control card 1**: signal type, data rate, swing, number of symbols, colour map, and the *Recalculate eye* button with the progress bar.
4. **Control card 2**: TX FFE, CTLE, DFE and the eye mask.
5. **Control card 3**: TX rise time, terminations, noise and random-jitter standard deviations, eye quality and eye view.
6. **KPI strip**: six tiles (insertion loss at Nyquist and at the highest frequency, median real part of Z_{11}, eye height, eye width, RMS crossing spread).
7. **Notebook** with the tabs of Table 9.1.
8. **Status bar** at the bottom, with the outcome of the last action.

Table 9.1: Tabs of the main window and the chapters that describe them.

Tab	Content	Chapter
Project & summary	Text report of the channel, the eye and all analyses	9
S-parameters	Magnitude and unwrapped phase of $S_{11}, S_{21}, S_{12}, S_{22}$	4
Loss	Insertion loss with the Nyquist marker	4
Impedance & Smith	Impedance and S11 on the Smith chart	4
Eye & jitter	Eye density with measurements and mask	6
Advanced	TDR/Z-profile, group delay, gains & stability, parameters, RF metrics, mixed-mode, equivalent circuit, noise, TRL	4–8
Quality & EQ	Pulse/bathtub, limit lines, ILD/COM/causality, crosstalk, file comparison	7
Tools	Cascade/de-embed, vector fit/SPICE, gating, Smith+/matching, conditioning, batch, export/session	5, 8, 9

9.2. Theming

A single palette (Section 2.3) defines every colour. Tk's `clam` theme is selected and then restyled through `ttk.Style` for frames, labels, buttons, entries, combo boxes, notebooks, progress bars and the status bar. The Matplotlib toolbars, which are classic Tk widgets, are re-coloured widget by widget and their icons regenerated for the dark background. Fonts are chosen from a preference list, using the first one installed on the system, with the Tk defaults as fallback. Inner notebooks (the sub-tabs of Advanced, Quality & EQ and Tools) use a smaller tab padding and an orange selection colour to distinguish them from the main tabs.

9.3. The Tab-Building Pattern

Every analysis tab is built with the same pattern, implemented once. A tab consists of an optional *control bar* (a card with labelled fields and buttons) above a Matplotlib figure with its toolbar. The helper creates the figure, stores it as `fig_<name>` and its canvas as `canvas_<name>`, and a method `_draw_<name>` redraws it. The convention allows generic code such as the placeholder, the redraw dispatcher and the figure export to address any tab by name.

Source: si_analyzer.py — AdvancedTabs._adv_tab

```
def _adv_tab(self, title: str, name: str, controls: Callable[[tk.Widget], None] | None = None
    , nb: Any = None) -> None:
    parent_nb = nb if nb is not None else self.adv_nb
    tab = ttk.Frame(parent_nb)
    parent_nb.add(tab, text=title)
    if controls is not None:
        bar = ttk.Frame(tab, style="Card.TFrame", padding=(12, 8))
        bar.pack(fill="x", pady=(0, 6))
        controls(bar)
    fig = Figure(figsize=(9, 6), dpi=100, constrained_layout=True)
    setattr(self, f"fig_{name}", fig)
    setattr(self, f"canvas_{name}", self._make_plot(tab, fig))
```

Source: si_analyzer.py — AdvancedTabs._adv_field

```
@staticmethod
def _adv_field(parent: tk.Widget, label: str, factory: Callable[[tk.Widget], tk.Widget]) ->
    tk.Widget:
    box = ttk.Frame(parent, style="Card.TFrame")
    box.pack(side="left", padx=(0, 14))
    ttk.Label(box, text=label, style="CardMuted.TLabel").pack(anchor="w")
    widget = factory(box)
    widget.pack(anchor="w", pady=(2, 0))
    return widget
```

Source: si_analyzer.py — AdvancedTabs._placeholder

```
def _placeholder(self, name: str, message: str) -> None:
    fig = getattr(self, f"fig_{name}")
    fig.clf()
    ax = fig.add_subplot(111)
    ax.axis("off")
    ax.text(0.5, 0.5, message, ha="center", va="center", color=MUTED, fontsize=12, transform=
        ax.transAxes)
    getattr(self, f"canvas_{name}").draw_idle()
```

Source: si_analyzer.py — AdvancedTabs._redraw

```
def _redraw(self, name: str) -> None:
    getattr(self, f"_draw_{name}")()
    self._update_summary()
```

A labelled field is a small frame containing a label above its input widget. In an early version of the main control card the widgets were created with the wrong parent, so that the label and the input were

placed side by side and the labels appeared shifted below the inputs. The fix is to create the widget *inside* the field's frame through a factory function:

Source: si_analyzer.py — si_analyzer.py, lines 3052–3060

```
def field(label: str, factory: Callable[[tk.Widget], tk.Widget]) -> tk.Widget:
    # The widget is created INSIDE the box so the label and the widget
    # stack vertically (label above, input below) instead of sitting side by side.
    box = ttk.Frame(controls, style="Card.TFrame")
    box.pack(side="left", padx=(0, 16))
    ttk.Label(box, text=label, style="CardMuted.TLabel").pack(anchor="w")
    widget = factory(box)
    widget.pack(anchor="w", pady=(2, 0))
    return widget
```

9.4. Plots and Interaction

9.4.1. Embedding Matplotlib

`_make_plot` embeds a figure into a Tk frame, adds the navigation toolbar (zoom, pan, save), re-themes the toolbar, and connects the marker handler of the next subsection.

Source: si_analyzer.py — SIAnalyzerApp._make_plot

```
@staticmethod
def _make_plot(parent: ttk.Frame, figure: Figure) -> FigureCanvasTkAgg:
    holder = tk.Frame(parent, bg=PLOT_BG)
    holder.pack(fill="both", expand=True)
    canvas = FigureCanvasTkAgg(figure, master=holder)
    canvas.get_tk_widget().configure(bg=PLOT_BG, highlightthickness=0)
    canvas.get_tk_widget().pack(fill="both", expand=True)
    toolbar = NavigationToolbar2Tk(canvas, holder, pack_toolbar=False)
    # Re-theme toolbar widgets and recolor their icons for the dark background.
    toolbar.configure(background=PANEL)
    for child in toolbar.winfo_children():
        try:
            child.configure(background=PANEL, foreground=FG, activebackground=CARD_HI,
                highlightbackground=PANEL,
                            selectcolor=CARD_HI)
        except tk.TclError:
            pass
        try:
            toolbar._set_image_for_button(child)  # matplotlib private API; harmless if
                missing
        except Exception:
            pass
    try:
        toolbar._message_label.configure(background=PANEL, foreground=MUTED)
    except Exception:
        pass
    toolbar.update()
    toolbar.pack(side="bottom", fill="x")

    def on_click(event: Any) -> None:
        ax = event.inaxes
        if ax is None or not getattr(event, "dblclick", False) or getattr(toolbar, "mode", ""
            ):
            return
        if event.button == 3:  # right double-click clears the markers of this axes
            for art in getattr(ax, "_si_markers", []):
                try:
                    art.remove()
                except Exception:
                    pass
            ax._si_markers = []
            canvas.draw_idle()
            return
        if event.button != 1:
            return
```

```
        best = None
        for line in ax.get_lines():
            if str(line.get_label()).startswith("_si"):
                continue
            try:
                xs, ys = np.asarray(line.get_xdata(), dtype=float), np.asarray(line.get_ydata(), dtype=float)
                if len(xs) < 2:
                    continue
                xy = ax.transData.transform(np.column_stack([xs, ys]))
                dist = np.hypot(xy[:, 0] - event.x, xy[:, 1] - event.y)
                i = int(np.nanargmin(dist))
            except Exception:
                continue
            if best is None or dist[i] < best[0]:
                best = (float(dist[i]), xs[i], ys[i])
        if best is None:
            return
        _, x0, y0 = best
        mk = ax.plot([x0], [y0], "o", color="white", markersize=6, zorder=10, label="_si_marker", scalex=False, scaley=False)[0]
        ann = ax.annotate(f"{x0:.4g}, {y0:.4g}", (x0, y0), xytext=(8, 8), textcoords="offset points", color=FG, fontsize=8,
                          zorder=11, bbox=dict(boxstyle="round,pad=0.3", facecolor=CARD, edgecolor=BORDER, alpha=0.95))
        ax._si_markers = list(getattr(ax, "_si_markers", [])) + [mk, ann]
        canvas.draw_idle()

    canvas.mpl_connect("button_press_event", on_click)
    return canvas
```

9.4.2. Markers

A double-click with the left button places a marker on the curve nearest to the cursor in *display* coordinates (so that curves with very different scales are treated fairly), and annotates it with its x and y values; a double-click with the right button removes all markers of that plot. Markers are ignored while a toolbar mode (zoom or pan) is active, and marker lines carry a label starting with an underscore so that they are excluded from legends and from subsequent snapping. The markers belong to the axes and disappear when the plot is redrawn.

9.4.3. Summary text

The *Project & summary* tab shows a text report that assembles sections produced by the analyses: channel description, eye estimate, RF metrics, group delay, stability, impedance profile, equivalent circuit, noise, equalization/bathtub, limits, channel quality, crosstalk and matching. Sections are stored in a dictionary and re-assembled in a fixed order, so that any tab can update its own section independently. Lines beginning with “##” are rendered as headings.

Source: `si_analyzer.py` — `AdvancedTabs._set_section`

```
def _set_section(self, key: str, title: str, lines: list[str]) -> None:
    self.adv_summary[key] = f"## {title}\n" + "\n".join(lines)
```

Source: `si_analyzer.py` — `AdvancedTabs._advanced_summary`

```
def _advanced_summary(self) -> str:
    data = getattr(self, "adv_summary", None) or {}
    return "\n\n".join(data[k] for k in ("rf", "gd", "stab", "tdr", "eq", "noise", "eqmask", "lim", "qual", "xt", "sm") if k in data)
```

Source: `si_analyzer.py` — `SIAnalyzerApp._set_summary`

```
def _set_summary(self, text: str) -> None:
    t = self.summary_text
    t.configure(state="normal")
    t.delete("1.0", "end")
```

```
    for line in text.split("\n"):
        if line.startswith("## "):
            t.insert("end", line[3:] + "\n", "h")
        else:
            t.insert("end", line + "\n")
    t.configure(state="disabled")
```

9.4.4. Key performance indicators

The six tiles are updated whenever the channel, the signal or the eye changes. The insertion loss at the Nyquist frequency is interpolated linearly from the loss curve, and reads “out of range” if the Nyquist frequency lies outside the measured band.

Source: `si_analyzer.py` — `SIAnalyzerApp._update_kpis`

```
def _update_kpis(self) -> None:
    if self.channel is None:
        return
    d = self.channel
    il = -20.0 * np.log10(np.maximum(np.abs(d.s[:, 1, 0]), 1e-15))
    z11 = impedance_from_reflection(d.s[:, 0, 0], np.real(d.z0[:, 0]))
    self.kpis["il_max"].configure(text=f"{il[-1]:.2f} dB")
    self.kpis["z11"].configure(text=f"{np.nanmedian(np.real(z11)):.1f} Ω")
    nyq = self._nyquist_hz()
    if nyq is not None and d.frequency_hz[0] <= nyq <= d.frequency_hz[-1]:
        self.kpis["nyq"].configure(text=f"{np.interp(nyq, d.frequency_hz, il):.2f} dB")
    else:
        self.kpis["nyq"].configure(text="out of range")
    r = self.eye_result
    if r is not None:
        self.kpis["height"].configure(text=f"{r['eye_height_raw_v']*1e3:.1f} mV")
        self.kpis["width"].configure(text=f"{r['eye_width_ps']:.1f} ps")
        self.kpis["jitter"].configure(text=f"{r['jitter_rms_ps']:.2f} ps")
    else:
        for k in ("height", "width", "jitter"):
            self.kpis[k].configure(text="-")
```

9.5. File Menu, Recent Files and Port Mapping

The menu is styled with the application palette. The list of recent files is rebuilt whenever a file is opened and entries whose files no longer exist are omitted. *Open with port mapping* always shows the mapping dialog, using the number of ports read from the file extension (`.sNp`); files that need a mapping (neither 2 nor 4 ports) open it automatically. The dialog offers a single-ended or a differential choice with combo boxes, validates that the selected ports are distinct, and returns a tuple that is passed to the reader (Section 3.3).

Source: `si_analyzer.py` — `UltimateTabs._build_menu`

```
def _build_menu(self) -> None:
    self.prefs = load_prefs()
    opts = dict(tearoff=0, bg=PANEL, fg=FG, activebackground=ACCENT, activeforeground="#04111
        c", borderwidth=0)
    bar = tk.Menu(self, **opts)
    fm = tk.Menu(bar, **opts)
    fm.add_command(label="…Open    Ctrl+O", command=self._choose_s2p)
    fm.add_command(label="Open with port …mapping", command=lambda: self._choose_s2p(
        force_map=True))
    self._recent_menu = tk.Menu(fm, **opts)
    fm.add_cascade(label="Recent files", menu=self._recent_menu)
    fm.add_separator()
    fm.add_command(label="Save …session", command=self._save_session)
    fm.add_command(label="Load …session", command=self._load_session)
    fm.add_separator()
    fm.add_command(label="Quit", command=self.destroy)
    bar.add_cascade(label="File", menu=fm)
    self.config(menu=bar)
    self.bind("<Control-o>", lambda _e: self._choose_s2p())
    self._rebuild_recent()
```

Source: `si_analyzer.py` — `UltimateTabs._ask_port_map`

```
def _ask_port_map(self, nports: int, path: str | Path) -> tuple | None:
    win = tk.Toplevel(self)
    win.title("Port mapping")
    win.configure(bg=BG)
    win.transient(self)
    ttk.Style(self).configure("TRadiobutton", background=BG, foreground=FG)
    ports = [str(i + 1) for i in range(nports)]
    mode = tk.StringVar(value="se")
    se_vars = [tk.StringVar(value=ports[0]), tk.StringVar(value=ports[min(1, nports - 1)])]
    diff_vars = [tk.StringVar(value=ports[min(k, nports - 1)]) for k in range(4)]
    result: dict[str, Any] = {"map": None}
    ttk.Label(win, text=f"{Path(path).name} has {nports} ports.\nChoose which ports form the channel to analyse.",
              padding=(14, 12)).pack(anchor="w")
    frm = ttk.Frame(win, padding=(14, 0))
    frm.pack(fill="x")
    ttk.Radiobutton(frm, text="Single-ended    input port", variable=mode, value="se").grid(row=0, column=0, sticky="w", pady=4)
    ttk.Combobox(frm, values=ports, textvariable=se_vars[0], width=4, state="readonly").grid(row=0, column=1, padx=6)
    ttk.Label(frm, text="output port").grid(row=0, column=2)
    ttk.Combobox(frm, values=ports, textvariable=se_vars[1], width=4, state="readonly").grid(row=0, column=3, padx=6)
    if nports >= 4:
        ttk.Radiobutton(frm, text="Differential    input pair (+, -)", variable=mode, value="diff").grid(row=1, column=0, sticky="w", pady=4)
        for k, col in ((0, 1), (1, 2)):
            ttk.Combobox(frm, values=ports, textvariable=diff_vars[k], width=4, state="readonly").grid(row=1, column=col, padx=6)
        ttk.Label(frm, text="output pair (+, -)").grid(row=2, column=0, sticky="e")
        for k, col in ((2, 1), (3, 2)):
            ttk.Combobox(frm, values=ports, textvariable=diff_vars[k], width=4, state="readonly").grid(row=2, column=col, padx=6)

    def ok() -> None:
        if mode.get() == "se":
            a, b = int(se_vars[0].get()) - 1, int(se_vars[1].get()) - 1
            if a == b:
                return messagebox.showerror("Invalid mapping", "Input and output ports must differ.", parent=win)
            result["map"] = ("se", a, b)
        else:
            sel = [int(v.get()) - 1 for v in diff_vars]
            if len(set(sel)) != 4:
                return messagebox.showerror("Invalid mapping", "The four differential ports must all differ.", parent=win)
            result["map"] = ("diff", *sel)
        win.destroy()

    row = ttk.Frame(win, padding=14)
    row.pack(fill="x")
    ttk.Button(row, text="OK", style="Accent.TButton", command=ok).pack(side="right")
    ttk.Button(row, text="Cancel", command=win.destroy).pack(side="right", padx=8)
    win.grab_set()
    self.wait_window(win)
    return result["map"]
```

9.6. Sessions and Exports

Sessions. A session file is a JSON document holding the value of every control (the names are listed in a class constant) and the path of the channel file. Loading restores the values and reloads the file; if the file has been moved the settings are still restored and the user is told to load the channel manually.

Figure export. Every figure of the application (the four main ones and every tab figure, found by the naming convention `fig_*`) is saved as a PNG file of 150 dpi into a chosen folder.

CSV export. The active channel is written as columns: frequency, magnitude (dB) and phase (degrees) of the four S-parameters, insertion loss, and S21 group delay.

HTML report. A single self-contained HTML file with the summary text and all figures embedded as base64-encoded PNG images, in the dark colour scheme of the application.

Source: `si_analyzer.py` — `ExtraTabs._save_session`

```
def _save_session(self) -> None:
    import json
    path = filedialog.asksaveasfilename(defaultextension=".json", filetypes=[("Session", "*.
        json")])
    if not path:
        return
    data = {k: getattr(self, k).get() for k in self._SESSION_VARS if hasattr(self, k)}
    data["file"] = str(self.channel.path) if self.channel is not None else ""
    Path(path).write_text(json.dumps(data, indent=2), encoding="utf-8")
    self.export_status_var.set(f"Session␣saved␣to␣{Path(path).name}")
```

Source: `si_analyzer.py` — `ExtraTabs._export_csv`

```
def _export_csv(self) -> None:
    if self.channel is None:
        return messagebox.showinfo("Nothing␣to␣export", "Load␣a␣channel␣first.")
    import csv
    path = filedialog.asksaveasfilename(defaultextension=".csv", filetypes=[("CSV", "*.csv")
        ])
    if not path:
        return
    d, f = self.channel, self.channel.frequency_hz
    cols: dict[str, np.ndarray] = {"freq_GHz": f / 1e9}
    for (i, j, nm) in ((0, 0, "S11"), (1, 0, "S21"), (0, 1, "S12"), (1, 1, "S22")):
        cols[f"{nm}_dB"] = _db(d.s[:, i, j])
        cols[f"{nm}_deg"] = np.degrees(np.angle(d.s[:, i, j]))
    cols["IL_dB"] = -_db(d.s[:, 1, 0])
    cols["groupdelay_S21_ps"] = group_delay(f, d.s[:, 1, 0]) * 1e12
    with open(path, "w", newline="") as fh:
        w = csv.writer(fh)
        w.writerow(cols.keys())
        w.writerows(zip(*cols.values()))
    self.export_status_var.set(f"Wrote␣{len(f)}␣rows␣to␣{Path(path).name}")
```

Source: `si_analyzer.py` — `ExtraTabs._export_html`

```
def _export_html(self) -> None:
    import base64
    import html
    import io
    path = filedialog.asksaveasfilename(defaultextension=".html", filetypes=[("HTML", "*.html
        ")])
    if not path:
        return
    parts = ["<html><head><meta␣charset='utf-8'><title>SI␣report</title></head>"
             "<body␣style='font-family:sans-serif;background:#0b0f19;color:#e6eaf2;max-width
                 :1100px;margin:auto'>",
             "<h1>Signal␣Integrity␣report</h1><pre␣style='background:#151c2c;padding:12px;
                 white-space:pre-wrap'>"
             + html.escape(self.summary_text.get("1.0", "end")) + "</pre>"]
    for name, fig in self._all_figures():
        buf = io.BytesIO()
        fig.savefig(buf, format="png", dpi=110)
        parts.append(f"<h3>{html.escape(name)}</h3><img␣style='max-width:100%'␣src='data:
            image/png; base64,"
                     + base64.b64encode(buf.getvalue()).decode() + "'>")
    parts.append("</body></html>")
    Path(path).write_text("".join(parts), encoding="utf-8")
    self.export_status_var.set(f"Wrote␣report␣{Path(path).name}")
```

9.7. Batch Processing

Design reviews often need the same figures of merit for many files (for example, variants of a layout). The Batch tool takes a list of files or all Touchstone files in a folder and computes one row per file: ports, highest frequency, insertion loss at the Nyquist frequency and at the highest frequency, worst return loss of each port, median real part of Z_{11}, minimum and maximum impedance of the TDR profile within the automatic window, largest singular value of $\mathbf{S}$, and, optionally, the eye height and width with the current signal and equalization settings at 16 384 symbols and 64 samples per UI to keep the run fast. Results are shown in a table and exported to CSV. A file that fails (for instance, because it needs a port mapping) produces a row with the reason instead of aborting the batch.

Source: `si_analyzer.py` — `batch_row`

```
def batch_row(path: Path, nyq_hz: float, pairing: str, rate: float, swing: float, signal: str,
              eq: dict[str, float], with_eye: bool, tr_ps: float, window: str) -> list[str]:
    ch = read_touchstone(path, pairing)
    f, s = ch.frequency_hz, ch.s
    il = -_db(s[:, 1, 0])
    z1 = float(np.median(np.real(ch.z0[:, 0])))
    zd = impedance_from_reflection(s[:, 0, 0], np.real(ch.z0[:, 0]))
    t, st, _ = time_response(f, s[:, 0, 0], tr_ps * 1e-12, window)
    tau = float(np.nanmedian(np.abs(group_delay(f, s[:, 1, 0]))))
    span = max(6.0 * (tau if np.isfinite(tau) else 0.0) + 8.0 * tr_ps * 1e-12, 20.0 / f[-1])
    z = impedance_from_step(st, z1)[t <= span]
    nyq = f"{np.interp(nyq_hz, f, il):.2f}" if f[0] <= nyq_hz <= f[-1] else "n/a"
    row = [path.name, str(ch.ports), f"{f[-1]/1e9:.4g}", nyq, f"{il[-1]:.2f}", f"{(-_db(s[:, 0, 0])).min():.2f}",
           f"{(-_db(s[:, 1, 1])).min():.2f}", f"{np.nanmedian(np.real(zd)):.1f}", f"{np.nanmin(z):.1f}",
           f"{np.nanmax(z):.1f}", f"{passivity(s).max():.3f}", "", ""]
    if with_eye:
        r = make_eye_and_jitter(f, s[:, 1, 0], rate, swing, total_bits=16_384, signal=signal,
                                **{**eq, "oversample": 64})
        row[-2], row[-1] = f"{r['eye_height_raw_v']*1e3:.1f}", f"{r['eye_width_ps']:.1f}"
    return row
```

Source: `si_analyzer.py` — `UltimateTabs._batch_run`

```
def _batch_run(self) -> None:
    if not self._batch_files_list:
        return messagebox.showinfo("Nothing queued", "Add files or a folder first.")
    self.batch_tree.delete(*self.batch_tree.get_children())
    self._batch_rows.clear()
    nyq = self._nyquist_hz() or 0.0
    eq = self._eq_settings()
    with_eye = self.batch_eye_var.get().startswith("Eye: on")
    tr = max(self._fv(self.tr_var, 35.0), 0.0)
    for k, path in enumerate(self._batch_files_list, start=1):
        self.batch_status_var.set(f"Processing {k}/{len(self._batch_files_list)}: {path.name} ")
        self.update_idletasks()
        try:
            row = batch_row(path, nyq, self.pair_var.get(), self._fv(self.rate_var, 5.0),
                self._fv(self.swing_var, 1.0),
                            self.signal_var.get(), eq, with_eye, tr, self.win_var.get())
        except Exception as exc:
            note = "needs port mapping (File  Open with port mapping)" if isinstance(exc,
                NeedPortMap) else "error: " + str(exc)[:60]
            row = [path.name, note] + [""] * (len(BATCH_COLUMNS) - 2)
        self._batch_rows.append(row)
        self.batch_tree.insert("", "end", values=row)
    self.batch_status_var.set(f"Done: {len(self._batch_rows)} file(s).")
```

9.8. The Logo

If a file named `logo.png` exists in the folder of the program (or in the working directory), the program uses it as the window icon (scaled to at most 256×256 pixels) and as a header image (scaled to fit $220 \times$

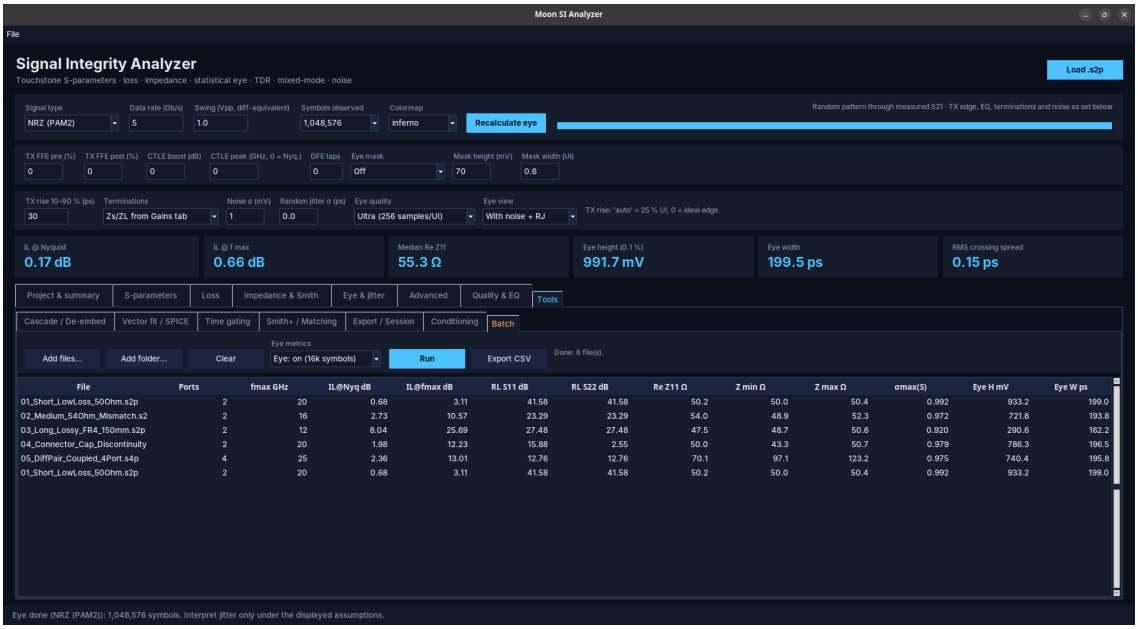


Figure 9.1: Batch processing of several channels.

46 pixels). Pillow is used for loading and Lanczos resampling. Any failure to load the image (missing file, unreadable file, missing library) is silently ignored so that a logo problem can never prevent the program from starting. When the program is packaged, the file must be added as a data file (Chapter 10).

Source: `si_analyzer.py` — `prepare_logo`

```
def prepare_logo(search_dirs: list[Path] | None = None) -> tuple[Any, Any] | None:
    """Find␣logo.png␣next␣to␣this␣script␣(or␣in␣the␣working␣directory)␣and␣return␣(icon,␣
        header)␣PIL␣images."""
    try:
        from PIL import Image
    except Exception:
        return None
    dirs = search_dirs or [Path(__file__).resolve().parent, Path.cwd()]
    for folder in dirs:
        path = folder / LOGO_NAME
        if path.is_file():
            try:
                img = Image.open(path).convert("RGBA")
            except Exception:
                return None
            resample = getattr(getattr(Image, "Resampling", Image), "LANCZOS")
            icon, header = img.copy(), img.copy()
            icon.thumbnail((256, 256), resample)
            header.thumbnail((220, 46), resample)
            return icon, header
    return None
```

Source: `si_analyzer.py` — `SIAnalyzerApp._load_logo`

```
def _load_logo(self) -> None:
    """Use␣logo.png␣(same␣folder␣as␣this␣script)␣as␣the␣window␣icon␣and␣in␣the␣header;␣
        skipped␣if␣absent."""
    self.logo_header = None
    self._logo_icon = None
    try:
        from PIL import ImageTk
        images = prepare_logo()
        if images is None:
            return
        self._logo_icon = ImageTk.PhotoImage(images[0])
```

```
        self.logo_header = ImageTk.PhotoImage(images[1])
        self.iconphoto(True, self._logo_icon)
    except Exception:
        self.logo_header = None  # a broken logo must never stop the application
```

9.9. Entry Point

The program can be started with an optional path argument, which is loaded shortly after the window appears. This is convenient for file associations and for scripting.

Source: si_analyzer.py — main

```
def main() -> None:
    import sys
    app = SIAnalyzerApp()
    if len(sys.argv) > 1 and Path(sys.argv[1]).exists():
        app.after(300, lambda: app.load_s2p(sys.argv[1]))  # python si_analyzer.py channel.
            s2p
    app.mainloop()
```

10

Installation, Usage and Deployment

This chapter explains how to install and run Moon SI Analyzer, how to use it from the command line and as a Python library, how a typical analysis session proceeds, how to package the program for distribution, and how to resolve common problems.

10.1. Requirements

- **Python** 3.10 or newer (the verification campaign of Chapter 11 used the version of Table 10.1).
- **Python packages:** NumPy, SciPy, Matplotlib and scikit-rf [**scikitrf**] (Pillow is installed together with Matplotlib).
- **Tk:** the `tkinter` module. It is part of the standard Windows and macOS installers of Python; on Debian and Ubuntu it is a separate package (`python3-tk`).
- **Display:** a graphical desktop. A resolution of at least 1280×800 is recommended; the window is 1320×880 by default with a minimum size of 1000×700.

Table 10.1: Software versions used for the verification run.

Package	Version
Python	3.12.3
NumPy	2.5.4
SciPy	1.18.1
Matplotlib	3.11.2
scikit-rf	2.2.0

10.2. Installation

The program is a single file. A virtual environment is recommended so that its dependencies do not interfere with other projects.

```
python3 -m venv venv
source venv/bin/activate            # Windows: venv\Scripts\activate
pip install numpy scipy matplotlib scikit-rf
sudo apt install python3-tk         # Debian / Ubuntu only
```

Place `si_analyzer.py` (and optionally `logo.png`) in one folder and run it as described below. Because the source is one file, updating the program means replacing that file.

10.3. Running the Program

```
python si_analyzer.py               # start empty, use File > Open (Ctrl+O)
```

```
python si_analyzer.py channel.s2p     # start and load a file
```

On start the program reads its preferences file (Section 2.10); nothing is written to the working folder. Files are opened from the *File* menu, the *Load* button, the recent-files list, or the command line.

10.4. Using the Numerical Layer as a Library

Because the numerical functions do not depend on the interface (Section 2.2), they can be used in scripts and notebooks. The module imports `tkinter` at start-up, so Tk must be installed even for headless use, but no window is created when the module is only imported. The following example reads a channel, computes an equalized eye and the statistical opening at a BER of 10^{-12}.

```
import numpy as np
import si_analyzer as si

channel = si.read_touchstone("channel.s2p")        # 2-port, or 4-port (differential Sdd)
f, s = channel.frequency_hz, channel.s             # Hz and an (Nf, 2, 2) complex array

il = -20 * np.log10(np.abs(s[:, 1, 0]))            # insertion loss in dB
print("IL at the last point: %.2f dB" % il[-1])

eye = si.make_eye_and_jitter(
    f, s[:, 1, 0], bit_rate_gbps=10, swing_vpp=1.0, total_bits=262_144,
    signal="NRZ (PAM2)", tx_rise_ps=30, ffe_post=0.10, ctle_boost_db=4, dfe_taps=2)
print(eye["eq_text"])
print("height %.1f mV, width %.1f ps" % (eye["eye_height_raw_v"] * 1e3, eye["eye_width_ps"]))

ber = si.ber_analysis(eye, sigma_v=1e-3, rj_ps=0.5, target=1e-12)
print("opening at 1e-12: %.3f UI, %.1f mV" % (ber["width_ui"], ber["height_v"] * 1e3))
```

The functions return plain NumPy arrays and dictionaries; the keys of the eye result are described in Section 6.9.

10.5. A Typical Session

1. **Load** the channel (Ctrl+O). Check the project summary: number of points, frequency range, median real part of Z_{11} and the passivity statement (Chapter 4).
2. **Inspect the frequency response:** S-parameters, loss and the Smith chart. Look for resonances and the loss at the Nyquist frequency of the intended data rate.
3. **Find physical problems:** in Advanced → TDR / Z-profile, read the discontinuity list and use the distance axis to map features to the layout (set the effective permittivity first).
4. **Judge the signal:** choose the signal type, the data rate and the swing in the first control card. Set a realistic TX rise time in picoseconds, and press *Recalculate eye*.
5. **Equalize:** set the FFE, CTLE and DFE in the second card and recalculate. Compare the eye height, the per-eye heights for PAM and the COM-like budget.
6. **Add noise and jitter** in the third card and open Quality & EQ → Pulse / Bathtub for the opening at the target BER; enable an eye mask to see pass/fail.
7. **Check compliance:** enter limit lines for the insertion and return loss in Quality & EQ → Limit lines.
8. **Document:** export the figures, the CSV and the HTML report, and save the session.

10.6. Typical Tasks

Differential channel from a four-port file. Open the `.s4p`; the differential block $\mathbf{S}_{dd}$ is analysed. If the results look wrong, change the pairing in Advanced → Mixed-mode (instruments differ in port numbering) and press *Apply pairing*.

Single lane of a multi-lane connector. Open the file with *File* → *Open with port mapping*, choose single-ended or differential ports, and use the Crosstalk tab with the same file to obtain NEXT, FEXT and ICN for a victim lane.

Removing a test fixture. With a known fixture file use Tools → Cascade / De-embed. With TRL standards use Advanced → TRL de-embed (Section 8.4).

Preparing a model for SPICE. Use Tools → Vector fit / SPICE, raise the number of poles until the error is acceptable, enforce passivity and export the subcircuit.

Comparing design variants. Load the reference as the channel and add the others in Quality & EQ → Compare files, or process all of them in Tools → Batch.

10.7. Packaging for Distribution

The following procedure is a recommended way to build a stand-alone application with PyInstaller [**pyinstaller**]; adapt it to the build scripts of the project. PyInstaller builds for the operating system on which it runs, so each platform (Windows, Linux, macOS) must be built on that platform.

```
pip install pyinstaller
pyinstaller --noconfirm --windowed --name "Moon SI Analyzer" \
            --add-data "logo.png:." si_analyzer.py
# On Windows the separator in --add-data is ';' instead of ':'
```

The `--add-data` option copies the logo next to the program inside the bundle, where the program searches for it (Section 9.8). If the frozen program reports missing modules or data files of scikit-rf, add `--collect-all skrf` to the command. The resulting folder is archived (`.zip` for Windows and macOS, `.tar.gz` for Linux) for the release page. Before releasing, run the verification suite (Chapter 11) against the same source file and test the frozen program with at least one two-port and one four-port file.

10.8. Files Created by the Program

- `~/.si_analyzer.json`: preferences (recent files, last folder).
- Files that the user explicitly exports: PNG figures, CSV, HTML report, SPICE netlist, session JSON, batch CSV.

No other files are created, and no network access is performed.

10.9. Troubleshooting

Symptom	Probable cause	Remedy
`No module named tkinter`	Tk not installed	Install `python3-tk` (Linux) or the python.org installer
Dialog asks for a port mapping	File has a port count other than 2 or 4	Choose the ports that form the channel
Differential results look wrong for a 4-port file	Port numbering convention differs	Change the pairing in Mixed-mode
Eye looks identical at all data rates	TX rise is `auto` (25 % of the UI)	Enter a fixed rise time in picoseconds
Eye shows ringing at the edges	Ideal TX edge with a band-limited channel	Use a realistic TX rise time
Eye calculation is slow	Many symbols or Ultra quality	Reduce the symbols or choose Normal/High quality
TDT or TDR has a slow drift	Data do not start at DC	Add a DC point in Tools → Conditioning
Values are `NaN` at some frequencies	Singular matrix (open, short, $S_{21} = 0$)	Expected; the point is skipped in plots
Passivity reported as ACTIVE	Measurement or solver error	Inspect the data; optionally enforce passivity
Vector-fit error is large	Too few poles for a long channel	Increase real poles and complex pairs

Symptom	Probable cause	Remedy
TRL fails or gives noise	Line length outside 20°–160°	Choose a different line or restrict the band
Logo not shown	File name or location wrong	Place `logo.png` next to `si_analyzer.py`

11

Verification and Validation

A reference document that presents equations is only as reliable as the code that implements them. This chapter describes the verification campaign that checks the implementation against independent references, states its results, reports the defects it uncovered, and explains how the checks are repeated after every change.

11.1. Method

Verification asks whether the code solves its equations correctly; *validation* asks whether the equations describe reality. The campaign of this chapter is mainly verification, supplemented by physical-plausibility checks. Each check compares a function of the numerical layer with an independent reference of one of four kinds:

1. a **library reference**: the same quantity computed by scikit-rf (parameter conversions, calibration);
2. a **closed-form solution**: for example the group delay of a lossless line, the Gaussian error probability $Q(a/\sigma)$, or a known RLGC set converted to S-parameters and recovered again;
3. a **circuit solution**: the source–network–load circuit solved directly with Kirchhoff's laws (terminations), or an impedance chain evaluated by hand (L-section matching);
4. a **mathematical identity**: orthogonality of the mixed-mode transformation, the equivalence of the stability criteria ($\mu > 1$ versus $K > 1 \wedge |\Delta| < 1$), or the equality of the sum of keep-gate and remove-gate responses with the original.

The suite is a stand-alone script that imports the program with the graphical toolkit replaced by a stub, so it runs without a display. It uses synthetic channels with known properties, generated with the media models of scikit-rf (lossy and lossless lines, a $70\,\Omega$ section, random fixture error boxes).

11.2. Results

Table 11.1 lists every check, its worst-case result and the acceptance criterion. All checks pass.

Table 11.1: Results of the automated verification suite (`tools/validate.py`).

ID	Area	Check	Result	Criterion	Status
V1	Network parameters	ABCD from S versus scikit-rf (max abs error)	2.28e-14	<= 1.00e-09	**pass**
V2	Network parameters	Z from S versus scikit-rf (max relative error)	3.73e-16	<= 1.00e-09	**pass**
V3	Network parameters	Y from S versus scikit-rf (max relative error)	6.08e-16	<= 1.00e-09	**pass**

ID	Area	Check	Result	Criterion	Status
V4	Network parameters	G = H^-1 consistency, max |G H - I|	1.08e-12	<= 1.00e-09	**pass**
V5	Mixed-mode	Orthogonality of the transform, max |M M^T - I|	2.22e-16	<= 1.00e-12	**pass**
V6	Mixed-mode	Two uncoupled identical lines: |Sdd21 - S21|	2.48e-16	<= 1.00e-12	**pass**
V7	Mixed-mode	Two uncoupled identical lines: max mode conversion |Scd|	2.94e-17	<= 1.00e-12	**pass**
V8	Frequency domain	Group delay of a 0.1 m line versus l*sqrt(er)/c (relative error)	3.88e-15	<= 1.00e-03	**pass**
V9	Frequency domain	Equivalence (K>1 and |Delta|<1) <=> (mu>1) on 4000 random S matrices (mismatches)	0	<= 0	**pass**
V10	Frequency domain	Matched terminations: GT = |S21|^2 (max abs error)	0.00e+00	<= 1.00e-12	**pass**
V11	Frequency domain	Passive line: max singular value of S minus 1 (must be <= 0)	-2.69e-02	<= 1.00e-09	**pass**
V12	Eye / terminations	Terminated voltage transfer versus direct circuit solution at one frequency	1.24e-16	<= 1.00e-09	**pass**
V13	Eye / terminations	Matched source and load reduce the transfer to S21 (max abs error)	0.00e+00	<= 1.00e-12	**pass**
V14	Time domain	TDR: position of the 70 ohm plateau centre (error, ps)	35.4	<= 60 ps (about 2x rise time)	**pass**
V15	Time domain	TDT of a lossless matched line settles to unity (|final-1|)	0.014	<= 0.020	**pass**
V16	Equivalent circuit	RLGC extraction: L' (relative error, median)	0.00e+00	<= 1.00e-03	**pass**
V17	Equivalent circuit	RLGC extraction: C' (relative error, median)	0.00e+00	<= 1.00e-03	**pass**
V18	Equivalent circuit	RLGC extraction: R' (relative error, median)	4.77e-15	<= 1.00e-03	**pass**
V19	Calibration	TRL de-embedding of a random fixture pair (max abs error)	2.60e-14	<= 1.00e-09	**pass**
V20	Conditioning	Passivity enforcement: sigma_max after clipping a 5 % gain error	0.9990	<= 0.9990	**pass**
V21	Conditioning	DC extrapolation keeps the response passive (sigma_max at DC)	0.9519	<= 1.0000	**pass**
V22	Conditioning	Uniform resampling: error against the original at original nodes (max)	1.01e-03	<= 5e-3 (linear read-back)	**pass**
V23	Smith / matching	Number of L-section solutions for 40+60j ohm -> 50 ohm (both topologies exist)	4	>= 4	**pass**
V24	Smith / matching	Input impedance of the shunt-first topology (max |Zin - 50|)	1.00e-14	<= 1.00e-09	**pass**
V25	Smith / matching	Input impedance of the series-first topology (max |Zin - 50|)	0.00e+00	<= 1.00e-09	**pass**
V26	Eye engine	PRBS7 period balance: |ones - zeros| (maximal-length LFSR gives 1)	1	<= 1	**pass**
V27	Eye engine	PRBS15 length equals 2^15 - 1	0	<= 0	**pass**
V28	Eye engine	CTLE has unity gain at its peak frequency (|H|-1)	0.00e+00	<= 1.00e-09	**pass**

ID	Area	Check	Result	Criterion	Status
V29	Eye engine	3-tap FFE preserves the sum of absolute tap weights (|sum|c|-1|)	0.00e+00	<= 1.00e-12	**pass**
V30	Eye engine	Lossless channel, ideal edge: eye height relative to swing	0.981	>= 0.900	**pass**
V31	Eye engine	Lossy channel: eye height gain from FFE + CTLE + DFE (equalised / unequalised)	19.67	> 1.00	**pass**
V32	BER	Two-rail eye + Gaussian noise: BER at the eye centre versus Q(a/sigma) (relative error)	2.14e-15	<= 1.00e-06	**pass**
V33	Crosstalk	ICN scales linearly with the coupling amplitude (|ICN(2X)/ICN(X) - 2|)	0.00e+00	<= 1.00e-09	**pass**
V34	Time domain	Keep-gate + remove-gate reconstruct the original response (max abs error)	3.12e-03	<= 1.00e-02	**pass**

Selected checks are reproduced below to show the principle. The first verifies the terminated-channel transfer function (Equation (6.13)) against a direct solution of the source–two-port–load circuit at one frequency:

Source: `si_analyzer.py` — `validate.py`, *lines 72–77*

```
k = 100; Zs, ZL = 70.0, 80.0
Zm = 50 * np.linalg.solve(np.eye(2) - S[k], np.eye(2) + S[k])
A = np.array([[Zm[0, 0] + Zs, Zm[0, 1]], [Zm[1, 0], Zm[1, 1] + ZL]]); I = np.linalg.solve(A, [1, 0])
v_circuit = -I[1] * ZL / 0.5            # load voltage / (E/2); minus sign: I2 flows into the port
v_formula = m.terminated_response(S, (Zs - 50) / (Zs + 50), (ZL - 50) / (ZL + 50))[k]
check("V12", "Eye / terminations", "Terminated voltage transfer versus direct circuit solution at one frequency", abs(v_circuit - v_formula), 1e-9)
```

The second verifies the bit-error model against the closed form $Q(a/\sigma_n)$ for an idealized two-rail eye:

Source: `si_analyzer.py` — `validate.py`, *lines 165–172*

```
dens = np.zeros((128, 400)); vmin, vmax = -0.6, 0.6
edges = np.linspace(vmin, vmax, 401); cen = 0.5 * (edges[:-1] + edges[1:])
ihi, ilo = np.argmin(np.abs(cen - 0.25)), np.argmin(np.abs(cen + 0.25)); dens[:, ihi] = 1e6; dens[:, ilo] = 1e6
rr = {"density": dens, "oversample": 64, "v_min": vmin, "v_max": vmax, "thresholds_v": np.array([0.0]), "ui_ps": 100.0, "sample_phase_ui": 0.5}
sig = 0.05; ba = m.ber_analysis(rr, sig, 0.0, 1e-12)
from scipy.special import erfc
exact = 0.5 * erfc((cen[ihi] / sig) / math.sqrt(2))
check("V32", "BER", "Two-rail eye + Gaussian noise: BER at the eye centre versus Q(a/sigma) (relative error)", abs(ba["ber_t"][32] - exact) / exact, 1e-6)
```

11.3. Defects Found During Verification and Development

DC discontinuity in the time-domain transform. The first version extrapolated below the first measured frequency by holding *both* the real and the imaginary part. For a delay line whose first point is at $50\,\mathrm{MHz}$ the imaginary part is about -0.2, which created a step at DC and a slowly decaying error: the settled value of the transmission step of a lossless matched line was $9\,\%$ off. The check “TDT settles to unity” exposed it. The corrected extrapolation (real part held, imaginary part ramped to zero) reduced the error to $1.4\,\%$ and was applied to every transform (TDR, impulse, gating and the eye engine). The residual error is the cost of band-limitation with a Hann window.

Isolated dots in the eye display. On low-loss channels at low data rates the transitions of the eye appeared as separate dots instead of curves. The statistics were correct; the cause was that the display histogram contained only samples, while a fast edge jumps over many voltage bins between two samples. The remedy (Section 6.7) is the display-only persistence density.

Wrongly assumed ideal edge. Results at different data rates looked identical because the `auto` rise time scales with the UI. It is documented in Section 6.3 and offered as an explicit input.

Mis-specified test cases. Three of the 34 checks failed at first because the *test* was wrong, not the code: a port pairing that did not match the synthetic wiring, a TDR reference taken at the beginning instead of the centre of a plateau, and a threshold that was too tight for the Gibbs ringing of an ideal edge. They were corrected after analysis; this is mentioned here because verification code needs the same scrutiny as the code under test.

11.4. Regression Procedure

After any change to the numerical layer, run

```
python tools/validate.py si_analyzer.py results.json
```

and regenerate this documentation (the table above is produced from `results.json`). A change that breaks a check must be understood before it is accepted. When a new analysis is added, a check against an independent reference should be added at the same time.

11.5. What Has Not Been Validated

Honesty about the extent of the evidence is part of a reliable reference. The following have *not* been verified in this campaign:

- Agreement with commercial signal-integrity tools or with a measured eye on an oscilloscope. The eye engine implements a well-defined mathematical model, and its building blocks are verified, but the end-to-end result has not been compared with instrument data.
- Behaviour with large real-world measurement files (hundreds of thousands of points) and with every vendor's Touchstone variants; scikit-rf's parser is relied upon.
- The graphical interface itself: it has been used interactively but is not covered by automated tests.
- The packaged applications on all platforms.

Users should treat the program as an engineering aid whose assumptions are stated, and should confirm critical decisions with an independent tool or measurement.

12

Assumptions, Limitations and Future Work

Every model simplifies. This chapter collects the assumptions of the analyses in one place, lists the known limitations, gives guidance on interpreting results, and proposes directions for future development.

12.1. Global Assumptions

- The channel is **linear and time-invariant**; non-linear drivers, receivers and time-varying behaviour are not modelled.
- The data are **passive, reciprocal and causal** unless stated otherwise; the program checks and reports deviations but does not correct causality.
- Reference impedances are **real**; complex references are not treated by the conversion functions, and unequal port impedances are replaced by their average.
- Frequency data are **band-limited**; the response above the highest measured frequency is taken as zero (with a taper) and the response below the first point is extrapolated.
- The transmitted data are **ideal**: symbols are independent and uniformly random (or PRBS), amplitudes are exact, and there is no transmitter jitter or noise.

12.2. Limitations by Analysis

Analysis	Limitation
Eye diagram	Deterministic ISI only in the measurements; random noise and jitter appear only in the statistical views. No clock recovery, receiver sampling aperture, or adaptation of equalizers; FFE has three taps, the CTLE has a fixed one-zero/two-pole form, the DFE assumes error-free decisions. Eye height uses the $0.1\,\%$ tails.
Bathtub and BER	Gaussian noise and Gaussian random jitter only; the noise is independent of the signal; multi-level BER is a symbol-error estimate; the error probability of tails beyond the observed deterministic distribution comes from the Gaussian model, not from the data.
Masks	Generic hexagonal masks; standard masks must be entered by hand; the mask is pass only for zero hits on the sample histogram.

Analysis	Limitation
COM-like	Simplified peak-distortion budget; not the IEEE 802.3 COM; no optimization of equalizer coefficients.
ICN	Weighting in the style of IEEE 802.3 with $f_t = f_r = 0.75 f_b$ and amplitude $V_{pp}/2$; compare only between designs analysed identically.
TDR/Z-profile	Computed from a band-limited, windowed transform. Accuracy of absolute impedance decreases after the first discontinuities because of attenuation and multiple reflections. Resolution is limited by f_{max} and the rise time.
Causality check	Heuristic energy test; it cannot prove causality. No causality enforcement.
Passivity enforcement	Local singular-value clipping; does not preserve causality or accuracy beyond small violations.
Equivalent circuit	Uniform-line assumption for RLGC; π model only for electrically short structures; constant R and G (no skin-effect or dispersive model).
Vector fitting	Quality depends on the number of poles and on delay; delay is not extracted before fitting.
TRL	Requires a line of suitable electrical length and a reflect of high magnitude; uses the scikit-rf implementation.
Mixed-mode	Single-ended ports with equal real reference impedance; port numbering conventions must be selected by the user.
Noise	Noise parameters are read only from two-port files; passive-network noise figure at 290 K when the block is absent.
Batch	Eye metrics use 16 384 symbols and 64 samples per UI for speed.

12.3. Interpreting Results Responsibly

- Compare designs *relative to each other* using identical settings; the absolute values depend on the transmitter and receiver models that the program only approximates.
- Check the *passivity and causality statements* first. Time-domain and eye results from active or acausal data are not trustworthy.
- State the *assumptions printed in the summary* (equalization settings, TX rise, terminations) whenever a figure is shared.
- Treat compliance statements (mask, limit lines) as *screening*; use the standard's own procedure for formal compliance.

12.4. Future Work

The following extensions would increase the usefulness of the program while remaining within its architecture.

1. **Standard masks and limits** for common interfaces, entered from the specifications, with provenance recorded in the file.
2. **Full COM** following the Ethernet procedure, with optimization of FFE and CTLE settings.
3. **Causality enforcement** and a causality metric based on the Hilbert-transform relations, and passivity enforcement by Hamiltonian perturbation.
4. **Delay extraction** before vector fitting to reduce the number of poles for long channels.
5. **Adaptive equalization** (LMS FFE/DFE) and a clock-and-data-recovery model with jitter transfer.
6. **Frequency-dependent RLGC models** (skin-effect and dielectric dispersion) for the equivalent-circuit tool.

7. **Complex reference impedances** and unequal port impedances in all conversions.
8. **Automated GUI tests** and a continuous-integration pipeline that runs the verification suite on every commit.
9. **Comparison with instrument data**: a measured-eye import and an agreement report.
10. **Parallel eye computation** across blocks for faster results at high symbol counts.

A

API Reference

This appendix lists the functions and classes of `si_analyzer.py`. Table A.1 is generated automatically from the source (names, parameters and the first sentence of each docstring), so it always matches the program. Names starting with an underscore are internal helpers; they are listed because the listings of the chapters refer to them.

Table A.1: Top-level functions of `si_analyzer.py` (parameters shown without annotations and defaults).

Function	Call	Purpose	Line
`read_touchstone`	`read_touchstone(path, pairing, port_map)`	Load an N-port Touchstone file through scikit-rf.	105
`impedance_from_reflection`	`impedance_from_reflection(gamma, reference_z)`	Convert a one-port reflection coefficient to impedance.	157
`_pam`	`_pam(levels)`	Specification of an L-level PAM signal (levels, bits per symbol, thresholds).	171
`_prbs_sequence`	`_prbs_sequence(order)`	One full period of a maximal-length LFSR sequence (x^n + x^k + 1).	197
`_generate_symbols`	`_generate_symbols(spec, rng, n)`	Return n symbol levels in [-1, 1] for the selected signal type.	211
`_zero_crossings`	`_zero_crossings(rx, oversample)`	Return crossing positions (in samples, modulo one UI) of the zero threshold.	240
`_eye_openings`	`_eye_openings(dens, centers, thresholds)`	Per-sub-eye, per-column opening (0.1 % tails): array of shape (n_eyes, columns).	252
`_statistical_eye_height`	`_statistical_eye_height(dens, centers, thresholds)`	Per-column eye opening; for multi-level signals the smallest sub-eye wins.	270
`ctle_response`	`ctle_response(freq, boost_db, peak_hz)`	One-zero / two-pole CTLE normalised to unity gain at its peak frequency (low frequencies are attenuated).	276
`apply_ffe`	`apply_ffe(wave, os_, pre, post)`	3-tap TX FFE [-pre, 1-|pre|-|post|, -post] applied on a symbol-rate waveform (circular).	290
`_pulse_response`	`_pulse_response(h_i, n, os_, amp, kind, ffe, pre_ui)`	Single-symbol response of TX FFE + channel + CTLE.	297

Function	Call	Purpose	Line
make_eye_and_jitter	make_eye_and_jitter(frequency_hz, s21, bit_rate_gbps, swing_vpp, total_bits, oversample, block_bits, v_bins, seed, signal, progress, ffe_pre, ffe_post, ctle_boost_db, ctle_peak_ghz, dfe_taps, tx_rise_ps)	Filter random symbol blocks with measured S21 and accumulate a 2-UI eye density.	309
ber_analysis	ber_analysis(r, sigma_v, rj_ps, target)	Statistical BER estimate from the eye density plus Gaussian voltage noise and random jitter.	511
mask_polygon	mask_polygon(c_ui, c_v, h_v, w_ui, top)	Vertices of the hexagonal eye mask.	572
mask_check	mask_check(r, h_v, w_ui)	Hexagonal eye mask on every sub-eye of both displayed UIs; PASS means zero hits inside.	578
_db	_db(x)	Magnitude in decibels with a 10^{-15} floor.	603
_pdb	_pdb(x)	Power ratio -> dB, NaN where the ratio is not a positive finite number.	607
_interp_response	_interp_response(fg, f, h)	Interpolate a measured complex response onto the grid ‘fg‘ (starting at DC).	616
_solve	_solve(a, b)	Batched linear solve that returns NaN for singular frequency points instead of raising.	635
group_delay	group_delay(f, s)	Group delay in seconds: -d(phase)/d(omega).	650
stability_metrics	stability_metrics(s)	Rollett K, determinant Δ, Edwards–Sinsky μ, μ', MSG, MAG and Mason’s U.	657
power_gains	power_gains(s, gs, gl)	Transducer (GT), available (GA) and operating (GP) power gain for given terminations.	672
passivity	passivity(s)	Largest singular value of S at each frequency (<= 1 for a passive network).	685
convert_parameters	convert_parameters(s, z0)	S -> Z, Y, ABCD, T, H, G for a 2-port with equal real reference impedance z0.	691
time_response	time_response(f, h, rise_s, window, n_freq, ext)	Step and impulse response of a measured transfer/reflection function.	722
impedance_from_step	impedance_from_step(rho, z0)	Converts a reflection step response to an impedance profile.	754
extract_rlgc	extract_rlgc(f, s, z0, length_m)	Per-unit-length R, L, G, C, Zc, gamma from the ABCD matrix of a uniform line of known length.	760
fit_pi_model	fit_pi_model(f, s, z0, frac)	Lumped pi model (series R+L, shunt G||C at both ends) fitted to the low part of the band.	782
mixed_mode	mixed_mode(s4, pairing)	Single-ended 4-port -> mixed-mode 2x2 blocks (Sdd, Sdc, Scd, Scc).	813
parse_touchstone_noise	parse_touchstone_noise(path)	Read the optional noise block (freq, NFmin dB, |Gopt|, angle, Rn/R0) of a 2-port Touchstone file.	824
noise_figure_db	noise_figure_db(noise, gs)	Noise figure for a given source reflection coefficient from the four noise parameters.	863

Function	Call	Purpose	Line
rf_metrics_lines	rf_metrics_lines(f, s)	Text report: IL, -3 dB bandwidth, RL, VSWR, mismatch loss, passivity, reciprocity.	873
_format_values	_format_values(v, fmt)	Converts complex data to magnitude, dB, phase, real or imaginary part.	905
impulse_full	impulse_full(f, h, n_freq, ext)	Two-sided impulse response (t from -T/2 to +T/2) of a measured response, Hann-windowed.	1544
gate_response	gate_response(f, h, t1, t2, edge, keep, n_freq, ext)	Time-gate a response: keep (or remove) the window [t1, t2] with raised-cosine edges.	1560
enforce_passivity	enforce_passivity(s, limit)	Clip the singular values of S at every frequency to 'limit' (simple passivity repair).	1588
icn_rms	icn_rms(f, xs, fb, amp)	Integrated crosstalk noise (RMS volts) with an IEEE 802.3-style weighting (ft = fr = 0.75 fb).	1594
parse_limit_points	parse_limit_points(text)	'f_GHz:dB, f_GHz:dB, ...' -> sorted arrays (Hz, dB).	1603
_x_text	_x_text(x, w)	Formats a reactance as an inductance or capacitance.	1617
_b_text	_b_text(b, w)	Formats a susceptance as a capacitance or inductance.	1621
l_match	l_match(z_load, r0, w)	Lumped L-section solutions matching z_load to r0 at angular frequency w.	1625
stability_circles	stability_circles(s)	Source- and load-plane stability circles (centre, radius) for one 2x2 S matrix.	1641
available_gain_circle	available_gain_circle(s, ga_lin)	Constant available-gain circle in the source plane.	1653
noise_circle	noise_circle(fmin_lin, gopt, rn_norm, f_lin)	Constant noise-figure circle in the source plane.	1665
prepare_logo	prepare_logo(search_dirs)	Find logo.png next to this script (or in the working directory) and return (icon, header) PIL images.	2555
load_prefs	load_prefs()	Reads the persistent preferences file.	2577
save_prefs	save_prefs(prefs)	Writes the persistent preferences file.	2585
terminated_response	terminated_response(s, gs, gl)	Voltage transfer to the load with arbitrary source/load reflection coefficients.	2593
find_discontinuities	find_discontinuities(t, z, z0, span, max_n)	Strongest impedance excursions (time, Z) inside the viewed window, in time order.	2605
extrapolate_dc	extrapolate_dc(f, s, k)	Prepend a DC point: real part extrapolated in f^2 from the first k points, imaginary part = 0.	2616
resample_uniform	resample_uniform(f, s, n)	Cubic resampling of S-parameters onto a uniform frequency grid.	2631
batch_row	batch_row(path, nyq_hz, pairing, rate, swing, signal, eq, with_eye, tr_ps, window)	Computes one row of the batch table for one file.	2646
main	main()	Program entry point (optionally opens the file given on the command line).	3631

A.1. Classes

Table A.2: Classes of the program.

Class	Purpose
`ChannelData`	Data model of the active channel (frequency, S-parameters, reference impedances, ports, noise, mapping note).
`NeedPortMap`	Exception signalling that a file needs a port mapping chosen by the user.
`AdvancedTabs`	Mixin: the Advanced tab (TDR, group delay, gains, parameters, RF metrics, mixed-mode, equivalent circuit, noise, TRL).
`ExtraTabs`	Mixin: equalization settings, masks, bathtub, limits, quality, crosstalk, comparison, cascade, vector fitting, gating, Smith+, export.
`UltimateTabs`	Mixin: menu, preferences, port mapping, conditioning, batch.
`SIAnalyzerApp`	Main window: styling, control cards, main tabs, loading, KPIs, eye display and calculation.

A.2. Naming Conventions of the Interface Code

Table A.3: Naming conventions in the mixins.

Pattern	Meaning
`_draw_<name>`	Redraws the tab named `<name>` from the current state (never changes the data).
`fig_<name>`, `canvas_<name>`	Matplotlib figure and Tk canvas of the tab.
`<something>_var`	Tk `StringVar` bound to an input control; read with `_fv` for numbers.
`_build_<group>_tabs`	Constructs the widgets of a group of tabs (called once at start-up).
`_refresh_<group>`	Redraws a group of tabs after the channel changed.
`_set_section(key, title, lines)`	Publishes a section of the summary report.
`_push_history`, `_undo`	Undo stack of channels.
`_apply_*`, `_casc_apply`, `_gate_apply`	Operations that replace the active channel (always with undo).

A.3. Tab Registry

Table A.4: Analysis tabs: figure name, draw method and group.

Tab	Draw method	Figure	Group
TDR / Z-profile	`_draw_tdr`	`fig_tdr`	Advanced
Group delay	`_draw_gd`	`fig_gd`	Advanced
Gains & stability	`_draw_gains`	`fig_gains`	Advanced
Parameters	`_draw_params`	`fig_params`	Advanced
RF metrics	`_draw_rf`	`fig_rf`	Advanced
Mixed-mode	`_draw_mm`	`fig_mm`	Advanced
Equivalent circuit	`_draw_eq`	`fig_eq`	Advanced
Noise	`_draw_noise`	`fig_noise`	Advanced
TRL de-embed	(in `_trl_run`)	`fig_trl`	Advanced
Pulse / Bathtub	`_draw_bath`	`fig_bath`	Quality & EQ
Limit lines	`_draw_lim`	`fig_lim`	Quality & EQ

Tab	Draw method	Figure	Group
ILD / COM / causality	`_draw_qual`	`fig_qual`	Quality & EQ
Crosstalk	`_draw_xt`	`fig_xt`	Quality & EQ
Compare files	`_draw_cmp`	`fig_cmp`	Quality & EQ
Cascade / De-embed	`_draw_casc`	`fig_casc`	Tools
Vector fit / SPICE	`_draw_vf`	`fig_vf`	Tools
Time gating	`_draw_gate`	`fig_gate`	Tools
Smith+ / Matching	`_draw_sm`	`fig_sm`	Tools
Conditioning	`_draw_cond`	`fig_cond`	Tools

A.4. Keys of the Eye Result

The dictionary returned by `make_eye_and_jitter` contains the following entries.

Table A.5: Keys of the eye-engine result.

Key	Meaning
`density`	Sample histogram, array $(2O \times B)$, used for all measurements
`density_vis`	Display histogram with trace interpolation
`v_min`, `v_max`	Voltage range of the histogram (V)
`ui_ps`, `oversample`, `bits`	UI in ps, samples per UI, symbols observed
`sample_phase_ui`	Best sampling phase (UI)
`eye_height_raw_v`, `eye_height_v`	Eye height (V), raw and clipped at zero
`eye_heights_v`	Opening of every sub-eye at the best phase
`eye_width_ps`, `jitter_rms_ps`, `jitter_pp_ps`, `crossing_count`	Timing measurements
`bit_rate_gbps`, `baud_gbd`, `swing_vpp`, `signal`	Settings used
`thresholds_v`, `n_eyes`	Decision thresholds and their number
`eq_text`	Description of the equalization and edge settings
`dfe_n`, `dfe_taps_v`	Number and values of the DFE taps
`pulse`	Pulse response: `x_ui`, `y`, cursor index `k` and values `cursors`

B

User-Interface Reference

This appendix lists the input controls of the interface, their default values and their effects. Numerical entries accept ordinary decimal numbers; an invalid entry falls back to the default of the corresponding calculation.

B.1. Main Control Cards

Table B.1: Controls of the three cards above the notebook.

Control	Default	Effect
Signal type	NRZ (PAM2)	Signal for the eye (Table 6.1); also sets the Nyquist frequency
Data rate (Gb/s)	5.0	Bit rate R_b; $T = \beta_s / R_b$
Swing (Vpp)	1.0	Peak-to-peak transmitted amplitude, $A = V_{pp}/2$
Symbols observed	262,144	Number of symbols accumulated in the eye (16 384 to 4 194 304)
Colormap	inferno	Colour map of the eye density
TX FFE pre / post (%)	0 / 0	Pre- and post-cursor FFE tap magnitudes (0–45)
CTLE boost (dB)	0	CTLE peaking; 0 disables
CTLE peak (GHz)	0	Peak frequency; 0 means the Nyquist frequency
DFE taps	0	Number of post-cursor DFE taps (0–24)
Eye mask	Off	Off, Custom or a generic template
Mask height (mV), width (UI)	50, 0.3	Dimensions of the hexagonal mask
TX rise 10–90 % (ps)	auto	`auto` = 25 % of the UI; 0 = ideal edge
Terminations	Ideal (matched)	Or “Zs/ZL from Gains tab” to include source/load reflections
Noise σ (mV)	1.0	Gaussian voltage noise for bathtub, BER, COM-like and the noisy eye view
Random jitter σ (ps)	0.0	Gaussian jitter for bathtub, BER and the noisy eye view
Eye quality	High (128)	Samples per UI: 64, 128 or 256
Eye view	Deterministic	Or “With noise + RJ” (display only)

B.2. Advanced Tab Controls

Table B.2: Controls inside the Advanced tab.

Control	Default	Effect
Rise time 10–90 % (ps)	35	Gaussian edge of the TDR/TDT step
$\varepsilon_{r,\mathrm{eff}}$	4.0	Distance axis of the TDR profile, $v = c/\sqrt{\varepsilon_{r,\mathrm{eff}}}$
Window	Hann	Frequency window of the time transform
View span (ns)	0 (auto)	Visible time range
Source R, X (Ω)	50, 0	Source impedance (gains, noise, terminated eye)
Load R, X (Ω)	50, 0	Load impedance
Parameter set	S	S, Z, Y, ABCD, T, H or G
Display	Magnitude (dB)	Magnitude (dB), magnitude, phase, real, imaginary
Differential pairs	1-2 / 3-4	Port pairing of 4-port files
Line length (mm)	50	For the RLGC extraction
π fit, lowest % of band	5	Range used for the lumped model
Reflect standard	Short (-1)	Estimated reflection of the TRL reflect

B.3. Quality & EQ and Tools Controls

Table B.3: Controls of the Quality & EQ and Tools tabs.

Control	Default	Effect
BER target (10^{-N})	12	Target for the eye opening
Max insertion loss	0:1, 5:10, 10:20	Limit points `f_GHz:dB` (placeholder)
Min return loss	0:12, 5:10, 10:6	Limit points (placeholder)
Port order (crosstalk)	Halves	Near/far port arrangement
Victim lane	1	Lane analysed as victim
Metric (compare)	Insertion loss	Quantity overlaid for several files
Operation (cascade)	Channel $\rightarrow$ file	Cascade, de-embed or flip
Real poles / complex pairs	6 / 16	Vector-fit model order
Gate parameter	S11	S11, S22, S21 or S12
Gate start / stop / edge (ns)	0 / 1 / 0.1	Time gate
Gate mode	Keep gate	Keep or remove the gate
Smith+ frequency (GHz)	5	Frequency of the circles and the match
Gain circles (dB)	$-1, -2, -3$	Available-gain circles
Noise circles (dB)	0.5, 1, 2	Offsets above NF_{min}
New Z_0 (Ω)	50	Renormalization target
Uniform points	801	Resampling grid
Eye metrics (batch)	Eye: off	Include eye height/width in the batch table

B.4. Keyboard and Mouse

Action	Effect
Ctrl+O	Open a Touchstone file
Double-click on a plot	Place a marker on the nearest curve point
Right double-click on a plot	Clear the markers of that plot
Toolbar zoom / pan	Standard Matplotlib navigation (markers are suspended while active)

Action	Effect
Command line argument	Path of a file to load at start-up